

HanOS

0.1.2.250201

THINK BIG, DO SMALL

1 Todo List	1
2 Data Structure Index	3
2.1 Data Structures	3
3 File Index	7
3.1 File List	7
4 Data Structure Documentation	13
4.1 acpi_gas_t Struct Reference	13
4.2 acpi_sdt_hdr_t Struct Reference	13
4.3 acpi_sdt_t Struct Reference	14
4.4 addrspace_t Struct Reference	14
4.5 ata_device_t Struct Reference	15
4.6 ata_identify_t Struct Reference	16
4.7 atapi_command_t Union Reference	16
4.8 auxval_t Struct Reference	16
4.9 backcmd_t Struct Reference	17
4.10 bmp_header_t Struct Reference	17
4.11 cmd_t Struct Reference	18
4.12 cmos_rtc_t Struct Reference	18
4.13 command_help_t Struct Reference	18
4.14 computer_info_t Struct Reference	19
4.15 cpu_t Struct Reference	19
4.16 cpuid_feature_t Struct Reference	20
4.17 dirent_t Struct Reference	20
4.18 edid_info_t Struct Reference	20
4.19 elf_hdr_t Struct Reference	21
4.20 elf_nhdr_t Struct Reference	22
4.21 elf_phdr_t Struct Reference	22
4.22 elf_shdr_t Struct Reference	23
4.23 elf_sym_t Struct Reference	23
4.24 event_t Struct Reference	23
4.25 execcmd_t Struct Reference	24
4.26 fat32_bs_info_t Struct Reference	24
4.27 fat32_entry_t Struct Reference	24
4.28 fat32_ident_item_t Struct Reference	25
4.29 fat32_ident_t Struct Reference	25
4.30 fat_bs_t Struct Reference	26
4.31 fat_dir_entry_t Struct Reference	26
4.32 fat_extbs_16_t Struct Reference	27
4.33 fat_extbs_32_t Struct Reference	27
4.34 fat_lfn_entry_t Struct Reference	27

4.35 fb_info_t Struct Reference	28
4.36 file_dup_t Struct Reference	28
4.37 font_psf1_t Struct Reference	29
4.38 font_psf2_t Struct Reference	29
4.39 gdt_entry_t Struct Reference	29
4.40 gdt_register_t Struct Reference	30
4.41 gdt_table_t Struct Reference	30
4.42 gfx_gtt_t Struct Reference	31
4.43 gfx_mem_manager_t Struct Reference	31
4.44 gfx_mem_range_t Struct Reference	32
4.45 gfx_object_t Struct Reference	32
4.46 gfx_pci_t Struct Reference	32
4.47 hpet_sdt_t Struct Reference	33
4.48 hpet_t Struct Reference	33
4.49 hpet_timer_t Struct Reference	34
4.50 ht_item_t Struct Reference	34
4.51 ht_t Struct Reference	35
4.52 ide_channel_regs_t Struct Reference	35
4.53 ide_device_t Struct Reference	36
4.54 idt_entry_t Struct Reference	36
4.55 idt_register_t Struct Reference	36
4.56 image_t Struct Reference	37
4.57 keyboard_t Struct Reference	37
4.58 klog_info_t Struct Reference	38
4.59 listcmd_t Struct Reference	39
4.60 lock_t Struct Reference	39
4.61 madt_record_hdr_t Struct Reference	39
4.62 madt_record_ioapic_t Struct Reference	40
4.63 madt_record_iso_t Struct Reference	40
4.64 madt_record_lapic_t Struct Reference	41
4.65 madt_record_nmi_t Struct Reference	42
4.66 madt_t Struct Reference	42
4.67 mbr_t Struct Reference	43
4.68 mem_info_t Struct Reference	43
4.69 mem_map_t Struct Reference	44
4.70 memory_metadata_t Struct Reference	44
4.71 out_fct_wrap_type Struct Reference	44
4.72 partition_t Struct Reference	45
4.73 pci_bar_t Struct Reference	45
4.74 pci_device_desc_t Struct Reference	45
4.75 pci_device_t Struct Reference	46
4.76 pipecmd_t Struct Reference	46

4.77 pipefs_file_t Struct Reference	47
4.78 pipefs_ident_item_t Struct Reference	47
4.79 pipefs_ident_t Struct Reference	48
4.80 prdt_t Struct Reference	48
4.81 ramfs_file_t Struct Reference	48
4.82 ramfs_ident_item_t Struct Reference	49
4.83 ramfs_ident_t Struct Reference	49
4.84 redircmd_t Struct Reference	50
4.85 rsdp_t Struct Reference	50
4.86 rusage_t Struct Reference	51
4.87 scache_t Struct Reference	52
4.88 sigaction_t Struct Reference	53
4.89 siginfo_t Struct Reference	53
4.90 sigset_t Struct Reference	55
4.91 signal Union Reference	55
4.92 slab_t Struct Reference	55
4.93 smp_info_t Struct Reference	56
4.94 stack_t Struct Reference	56
4.95 stat_t Struct Reference	57
4.96 symbol_t Struct Reference	57
4.97 sys_seg_desc_t Struct Reference	58
4.98 task_regs_t Struct Reference	58
4.99 task_t Struct Reference	59
4.100 term_info_t Struct Reference	60
4.101 termios_t Struct Reference	61
4.102 timespec_t Struct Reference	61
4.103 timeval_t Struct Reference	61
4.104 timezone_t Struct Reference	62
4.105 tm_t Struct Reference	62
4.106 tss_t Struct Reference	62
4.107 ttyfs_file_t Struct Reference	63
4.108 ttyfs_ident_item_t Struct Reference	63
4.109 ttyfs_ident_t Struct Reference	64
4.110 ustar_file_t Struct Reference	64
4.111 vfs_dirent_t Struct Reference	65
4.112 vfs_fsinfo_t Struct Reference	65
4.113 vfs_inode_t Struct Reference	66
4.114 vfs_node_desc_t Struct Reference	67
4.115 vfs_stat_t Struct Reference	68
4.116 vfs_timespec_t Struct Reference	69
4.117 vfs_tnode_t Struct Reference	69
4.118 winsize_t Struct Reference	70

5 File Documentation	71
5.1 kernel/base/hash.c File Reference	71
5.1.1 Detailed Description	72
5.2 kernel/base/hash.h File Reference	72
5.2.1 Detailed Description	72
5.3 kernel/base/image.c File Reference	72
5.3.1 Detailed Description	73
5.4 kernel/base/image.h File Reference	73
5.4.1 Detailed Description	74
5.5 kernel/base/klib.h File Reference	74
5.5.1 Detailed Description	75
5.5.2 Macro Definition Documentation	75
5.5.2.1 ASSERT	75
5.6 kernel/base/klog.c File Reference	75
5.6.1 Detailed Description	76
5.7 kernel/base/klog.h File Reference	76
5.7.1 Detailed Description	77
5.8 kernel/base/kmalloc.c File Reference	77
5.8.1 Detailed Description	78
5.9 kernel/base/kmalloc.h File Reference	78
5.9.1 Detailed Description	79
5.10 kernel/base/lock.c File Reference	80
5.10.1 Detailed Description	80
5.11 kernel/base/lock.h File Reference	80
5.11.1 Detailed Description	81
5.12 kernel/base/time.c File Reference	81
5.12.1 Detailed Description	82
5.13 kernel/base/time.h File Reference	82
5.13.1 Detailed Description	84
5.14 kernel/base/vector.h File Reference	84
5.14.1 Detailed Description	84
5.14.2 Macro Definition Documentation	85
5.14.2.1 vec_erase	85
5.14.2.2 vec_erase_all	85
5.14.2.3 vec_erase_val	85
5.14.2.4 vec_push_back	85
5.14.2.5 vec_struct	86
5.15 kernel/device/display/edid.h File Reference	86
5.15.1 Detailed Description	86
5.16 kernel/device/display/fb.c File Reference	86
5.16.1 Detailed Description	87
5.17 kernel/device/display/fb.h File Reference	88

5.17.1 Detailed Description	89
5.18 kernel/device/display/font.h File Reference	89
5.18.1 Detailed Description	89
5.19 kernel/device/display/gfx.c File Reference	90
5.19.1 Detailed Description	90
5.20 kernel/device/display/gfx.h File Reference	91
5.20.1 Detailed Description	91
5.21 kernel/device/display/gfx_reg.h File Reference	92
5.21.1 Detailed Description	94
5.22 kernel/device/display/term.c File Reference	94
5.22.1 Detailed Description	95
5.23 kernel/device/display/term.h File Reference	95
5.23.1 Detailed Description	96
5.24 kernel/device/keyboard/keyboard.h File Reference	97
5.24.1 Detailed Description	98
5.25 kernel/device/keyboard/keycode.h File Reference	98
5.25.1 Detailed Description	99
5.26 kernel/device/storage/ata.c File Reference	99
5.26.1 Detailed Description	100
5.27 kernel/device/storage/ata.h File Reference	100
5.27.1 Detailed Description	102
5.28 kernel/fs/fat32.c File Reference	102
5.28.1 Detailed Description	103
5.28.2 Variable Documentation	103
5.28.2.1 fat32	103
5.29 kernel/fs/fat32.h File Reference	103
5.29.1 Detailed Description	105
5.30 kernel/fs/filebase.c File Reference	105
5.30.1 Detailed Description	106
5.31 kernel/fs/filebase.h File Reference	106
5.31.1 Detailed Description	107
5.32 kernel/fs/pipefs.c File Reference	107
5.32.1 Detailed Description	108
5.32.2 Variable Documentation	108
5.32.2.1 pipefs	108
5.33 kernel/fs/pipefs.h File Reference	108
5.33.1 Detailed Description	109
5.34 kernel/fs/ramfs.c File Reference	110
5.34.1 Detailed Description	111
5.34.2 Variable Documentation	111
5.34.2.1 ramfs	111
5.35 kernel/fs/ramfs.h File Reference	111

5.35.1 Detailed Description	112
5.36 kernel/fs/ttyfs.c File Reference	112
5.36.1 Detailed Description	114
5.36.2 Variable Documentation	114
5.36.2.1 ttyfs	114
5.37 kernel/fs/ttyfs.h File Reference	115
5.37.1 Detailed Description	116
5.38 kernel/fs/vfs.c File Reference	116
5.38.1 Detailed Description	117
5.39 kernel/fs/vfs.h File Reference	117
5.39.1 Detailed Description	119
5.39.2 Macro Definition Documentation	120
5.39.2.1 S_IRWXG	120
5.39.2.2 S_IRWXO	120
5.39.2.3 S_IRWXU	120
5.40 kernel/kconfig.h File Reference	120
5.40.1 Detailed Description	121
5.41 kernel/kmain.c File Reference	121
5.41.1 Detailed Description	122
5.42 kernel/mm/alloc.c File Reference	122
5.42.1 Detailed Description	123
5.42.2 Macro Definition Documentation	123
5.42.2.1 CAPACITY_SIZE	123
5.43 kernel/mm/alloc.h File Reference	123
5.43.1 Detailed Description	124
5.44 kernel/mm/mm.h File Reference	124
5.44.1 Detailed Description	126
5.44.2 Macro Definition Documentation	126
5.44.2.1 VMM_FLAGS_MMIO	126
5.45 kernel/mm/pmm.c File Reference	126
5.45.1 Detailed Description	127
5.46 kernel/mm/slab.c File Reference	127
5.46.1 Detailed Description	128
5.46.2 Macro Definition Documentation	128
5.46.2.1 GET_SLAB	128
5.47 kernel/mm/slab.h File Reference	129
5.47.1 Detailed Description	130
5.48 kernel/mm/vmm.c File Reference	130
5.48.1 Detailed Description	131
5.49 kernel/proc/elf.c File Reference	131
5.49.1 Detailed Description	132
5.50 kernel/proc/elf.h File Reference	132

5.50.1 Detailed Description	133
5.51 kernel/proc/eventbus.c File Reference	134
5.51.1 Detailed Description	134
5.52 kernel/proc/eventbus.h File Reference	134
5.52.1 Detailed Description	135
5.53 kernel/proc/sched.c File Reference	135
5.53.1 Detailed Description	137
5.54 kernel/proc/sched.h File Reference	137
5.54.1 Detailed Description	138
5.55 kernel/proc/signal.c File Reference	138
5.55.1 Detailed Description	138
5.56 kernel/proc/signal.h File Reference	139
5.56.1 Detailed Description	141
5.57 kernel/proc/syscall.c File Reference	141
5.57.1 Detailed Description	143
5.58 kernel/proc/syscall.h File Reference	143
5.58.1 Detailed Description	145
5.59 kernel/proc/task.c File Reference	145
5.59.1 Detailed Description	146
5.60 kernel/proc/task.h File Reference	146
5.60.1 Detailed Description	147
5.61 kernel/proc/wait.h File Reference	148
5.61.1 Detailed Description	148
5.62 kernel/symbols.h File Reference	148
5.62.1 Detailed Description	149
5.63 kernel/sys/acpi.c File Reference	149
5.63.1 Detailed Description	150
5.64 kernel/sys/acpi.h File Reference	150
5.64.1 Detailed Description	151
5.65 kernel/sys/apic.c File Reference	151
5.65.1 Detailed Description	152
5.65.2 Function Documentation	152
5.65.2.1 apic_enable()	152
5.65.2.2 apic_init()	152
5.65.2.3 apic_read_reg()	152
5.65.2.4 apic_send_eoi()	153
5.65.2.5 apic_send_ipi()	153
5.65.2.6 apic_write_reg()	153
5.66 kernel/sys/apic.h File Reference	154
5.66.1 Detailed Description	155
5.66.2 Function Documentation	155
5.66.2.1 apic_enable()	155

5.66.2.2 apic_init()	155
5.66.2.3 apic_read_reg()	156
5.66.2.4 apic_send_eoi()	157
5.66.2.5 apic_send_ipi()	157
5.66.2.6 apic_write_reg()	157
5.67 kernel/sys/cmos.c File Reference	158
5.67.1 Detailed Description	159
5.68 kernel/sys/cmos.h File Reference	159
5.68.1 Detailed Description	160
5.69 kernel/sys/cpu.c File Reference	160
5.69.1 Detailed Description	160
5.70 kernel/sys/cpu.h File Reference	161
5.70.1 Detailed Description	162
5.70.2 Macro Definition Documentation	162
5.70.2.1 read_cr	162
5.70.2.2 write_cr	162
5.71 kernel/sys/gdt.c File Reference	162
5.71.1 Detailed Description	163
5.72 kernel/sys/gdt.h File Reference	163
5.72.1 Detailed Description	164
5.72.2 Macro Definition Documentation	164
5.72.2.1 AC_PR	164
5.72.2.2 AC_ST	165
5.72.2.3 GDT_GR	165
5.72.2.4 GDT_LM	165
5.73 kernel/sys/hpet.c File Reference	165
5.73.1 Detailed Description	166
5.74 kernel/sys/hpet.h File Reference	166
5.74.1 Detailed Description	167
5.75 kernel/sys/idt.c File Reference	167
5.75.1 Detailed Description	168
5.76 kernel/sys/idt.h File Reference	168
5.76.1 Detailed Description	169
5.77 kernel/sys/isr.c File Reference	169
5.77.1 Detailed Description	169
5.78 kernel/sys/isr_base.h File Reference	170
5.78.1 Detailed Description	171
5.78.2 Macro Definition Documentation	171
5.78.2.1 isr_disable_interrupts	171
5.78.2.2 isr_enable_interrupts	172
5.79 kernel/sys/madt.c File Reference	172
5.79.1 Detailed Description	172

5.80 kernel/sys/madt.h File Reference	172
5.80.1 Detailed Description	174
5.81 kernel/sys/mtrr.c File Reference	174
5.81.1 Detailed Description	174
5.82 kernel/sys/mtrr.h File Reference	175
5.82.1 Detailed Description	176
5.83 kernel/sys/panic.h File Reference	176
5.83.1 Detailed Description	177
5.83.2 Macro Definition Documentation	177
5.83.2.1 kpanic	177
5.83.2.2 panic_if	178
5.83.2.3 panic_unless	178
5.84 kernel/sys/pci.c File Reference	178
5.84.1 Detailed Description	179
5.84.2 Macro Definition Documentation	179
5.84.2.1 pci_func_exist	179
5.85 kernel/sys/pci.h File Reference	179
5.85.1 Detailed Description	183
5.85.2 Macro Definition Documentation	183
5.85.2.1 PCI_MAKE_DEVICE_ID	184
5.86 kernel/sys/pit.c File Reference	184
5.86.1 Detailed Description	184
5.87 kernel/sys/pit.h File Reference	185
5.87.1 Detailed Description	185
5.88 kernel/sys/serial.c File Reference	185
5.88.1 Detailed Description	186
5.89 kernel/sys/serial.h File Reference	186
5.89.1 Detailed Description	187
5.90 kernel/sys/smp.c File Reference	187
5.90.1 Detailed Description	188
5.91 kernel/sys/smp.h File Reference	188
5.91.1 Detailed Description	189
5.92 kernel/sys/timer.c File Reference	190
5.92.1 Detailed Description	190
5.93 kernel/sys/timer.h File Reference	191
5.93.1 Detailed Description	192
5.94 kernel/version.h File Reference	192
5.94.1 Detailed Description	193
5.95 libc/ctype.c File Reference	193
5.95.1 Detailed Description	194
5.96 libc/ctype.h File Reference	194
5.96.1 Detailed Description	195

5.97 libc/errno.h File Reference	195
5.97.1 Detailed Description	197
5.97.2 Macro Definition Documentation	197
5.97.2.1 ENETRESET	197
5.97.2.2 EOPNOTSUPP	197
5.98 libc/numeric.c File Reference	197
5.98.1 Detailed Description	198
5.99 libc/numeric.h File Reference	198
5.99.1 Detailed Description	199
5.100 libc/printf.h File Reference	199
5.100.1 Detailed Description	200
5.100.2 Macro Definition Documentation	200
5.100.2.1 printf_	200
5.100.2.2 snprintf	201
5.100.2.3 sprintf	201
5.100.2.4 vprintf	201
5.100.3 Function Documentation	202
5.100.3.1 _putchar()	202
5.100.3.2 fctprintf()	202
5.101 libc/stdio.h File Reference	202
5.101.1 Detailed Description	204
5.102 libc/string.c File Reference	204
5.102.1 Detailed Description	205
5.103 libc/string.h File Reference	205
5.103.1 Detailed Description	206
5.104 libc/sysfunc.c File Reference	206
5.104.1 Detailed Description	208
5.104.2 Macro Definition Documentation	209
5.104.2.1 SYSCALL0	209
5.104.2.2 SYSCALL1	209
5.104.2.3 SYSCALL2	209
5.104.2.4 SYSCALL3	209
5.104.2.5 SYSCALL4	209
5.104.2.6 SYSCALL6	210
5.105 libc/sysfunc.h File Reference	210
5.105.1 Detailed Description	212
5.106 userspace/cat.c File Reference	212
5.106.1 Detailed Description	213
5.107 userspace/echo.c File Reference	213
5.107.1 Detailed Description	214
5.108 userspace/hansh.c File Reference	214
5.108.1 Detailed Description	216

5.108.2 Function Documentation	216
5.108.2.1 execcmd()	216
5.108.3 Variable Documentation	216
5.108.3.1 whitespace	216
5.109 userspace/help.h File Reference	216
5.109.1 Detailed Description	217
5.110 userspace/init.c File Reference	217
5.110.1 Detailed Description	218
5.110.2 Variable Documentation	218
5.110.2.1 argv	218
5.111 userspace/ls.c File Reference	219
5.111.1 Detailed Description	219
5.112 userspace/pwd.c File Reference	219
5.112.1 Detailed Description	220
5.113 userspace/rm.c File Reference	220
5.113.1 Detailed Description	221
5.114 userspace/wc.c File Reference	221
5.114.1 Detailed Description	222
Index	223

Chapter 1

Todo List

File [fb.c](#)

Improve memcpy() by SSE enhancements.

File [kmalloc.c](#)

Memory allocation should be improved for better efficiency.

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

acpi_gas_t	13
acpi_sdt_hdr_t	13
acpi_sdt_t	14
addrspace_t	14
ata_device_t	15
ata_identify_t	16
atapi_command_t	16
auxval_t	16
backcmd_t	17
bmp_header_t	17
cmd_t	18
cmos_rtc_t	18
command_help_t	18
computer_info_t	19
cpu_t	19
cpuid_feature_t	20
dirent_t	20
edid_info_t	20
elf_hdr_t	21
elf_nhdr_t	22
elf_phdr_t	22
elf_shdr_t	23
elf_sym_t	23
event_t	23
execcmd_t	24
fat32_bs_info_t	24
fat32_entry_t	24
fat32_ident_item_t	25
fat32_ident_t	25
fat_bs_t	26
fat_dir_entry_t	26
fat_extbs_16_t	27
fat_extbs_32_t	27
fat_lfn_entry_t	27
fb_info_t	28

file_dup_t	28
font_psf1_t	29
font_psf2_t	29
gdt_entry_t	29
gdt_register_t	30
gdt_table_t	30
gfx_gtt_t	31
gfx_mem_manager_t	31
gfx_mem_range_t	32
gfx_object_t	32
gfx_pci_t	32
hpet_sdt_t	33
hpet_t	33
hpet_timer_t	34
ht_item_t	34
ht_t	35
ide_channel_regs_t	35
ide_device_t	36
idt_entry_t	36
idt_register_t	36
image_t	37
keyboard_t	37
klog_info_t	38
listcmd_t	39
lock_t	39
madt_record_hdr_t	39
madt_record_ioapic_t	40
madt_record_iso_t	40
madt_record_lapic_t	41
madt_record_nmi_t	42
madt_t	42
mbr_t	43
mem_info_t	43
mem_map_t	44
memory_metadata_t	44
out_fct_wrap_type	44
partition_t	45
pci_bar_t	45
pci_device_desc_t	45
pci_device_t	46
pipecmd_t	46
pipefs_file_t	47
pipefs_ident_item_t	47
pipefs_ident_t	48
prdt_t	48
ramfs_file_t	48
ramfs_ident_item_t	49
ramfs_ident_t	49
redircmd_t	50
rsdp_t	50
rusage_t	51
scache_t	52
sigaction_t	53
siginfo_t	53
sigset_t	55
sigval	55
slab_t	55
smp_info_t	56

stack_t	56
stat_t	57
symbol_t	57
sys_seg_desc_t	58
task_regs_t	58
task_t	59
term_info_t	60
termios_t	61
timespec_t	61
timeval_t	61
timezone_t	62
tm_t	62
tss_t	62
ttyfs_file_t	63
ttyfs_ident_item_t	63
ttyfs_ident_t	64
ustar_file_t	64
vfs_dirent_t	65
vfs_fsinfo_t	65
vfs_inode_t	66
vfs_node_desc_t	67
vfs_stat_t	68
vfs_timespec_t	69
vfs_tnode_t	69
winsize_t	70

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

kernel/ kconfig.h	
Kernel configuration header	120
kernel/ kmain.c	
Entry function of HanOS kernel	121
kernel/ symbols.h	
Definition of symbol related data structures	148
kernel/ version.h	
Definition of version information	192
kernel/base/ hash.c	
Implementation of a simple hash table	71
kernel/base/ hash.h	
Hash table definitions	72
kernel/base/ image.c	
Implementation of image handling functions	72
kernel/base/ image.h	
Image handling definitions	73
kernel/base/ klib.h	
Definition of fundamental data structures, macros and functions	74
kernel/base/ klog.c	
Implementation of kernel log related functions	75
kernel/base/ klog.h	
Definition of kernel log related functions	76
kernel/base/ kmallocc.c	
Implementation of kernel memory allocation functions	77
kernel/base/ kmallocc.h	
Definition of memory allocation related functions	78
kernel/base/ lock.c	
Implementation of spin lock related functions	80
kernel/base/ lock.h	
Definition of spin lock related data structures and functions	80
kernel/base/ time.c	
Implementation of time related functions	81
kernel/base/ time.h	
Definition of time related data structures and functions	82
kernel/base/ vector.h	
Vector - fundamental data structure, variable length array	84

kernel/device/display/ edid.h	
EDID (Extended Display Identification Data) definitions	86
kernel/device/display/ fb.c	
Implementation of framebuffer related functions	86
kernel/device/display/ fb.h	
Definition of framebuffer related functions	88
kernel/device/display/ font.h	
Definition of font data matrix	89
kernel/device/display/ gfx.c	
Implementation of graphics device handling	90
kernel/device/display/ gfx.h	
Graphics device handling definitions	91
kernel/device/display/ gfx_reg.h	
Graphics device register definitions	92
kernel/device/display/ term.c	
Implementation of framebuffer terminal related functions	94
kernel/device/display/ term.h	
Definition of framebuffer terminal related functions	95
kernel/device/keyboard/ keyboard.h	
Definition of keyboard related macros and data structures	97
kernel/device/keyboard/ keycode.h	
Keycode definitions for keyboard driver	98
kernel/device/storage/ ata.c	
Implementation of ATA (Advanced Technology Attachment) device handling	99
kernel/device/storage/ ata.h	
ATA (Advanced Technology Attachment) definitions	100
kernel/fs/ fat32.c	
Implementation of FAT32 related functions	102
kernel/fs/ fat32.h	
Definition of FAT32 related data structures and functions	103
kernel/fs/ filebase.c	
Implementation of fundamental VFS file node related functions	105
kernel/fs/ filebase.h	
Definition of fundamental VFS file node related functions	106
kernel/fs/ pipefs.c	
Implementation of PipeFS functions	107
kernel/fs/ pipefs.h	
Definition of PipeFS related data structures and functions	108
kernel/fs/ ramfs.c	
Implementation of RAMFS functions	110
kernel/fs/ ramfs.h	
Definition of RAMFS related data structures and functions	111
kernel/fs/ ttyfs.c	
Implementation of TTYFS functions	112
kernel/fs/ ttyfs.h	
Definition of TTYFS related data structures and functions	115
kernel/fs/ vfs.c	
Implementation of VFS related functions	116
kernel/fs/ vfs.h	
Definition of VFS related data structures and functions	117
kernel/mm/ alloc.c	
Implementation of memory allocation functions	122
kernel/mm/ alloc.h	
Memory allocation definitions	123
kernel/mm/ mm.h	
Definition of memory management related data structures	124
kernel/mm/ pmm.c	
Implementation of physical memory management functions	126

kernel/mm/ slab.c	
Implementation of slab allocator	127
kernel/mm/ slab.h	
Slab allocator definitions	129
kernel/mm/ vmm.c	
Implementation of virtual memory management functions	130
kernel/proc/ elf.c	
Implementation of ELF file loading and handling functions	131
kernel/proc/ elf.h	
Definitions for ELF file format structures and constants	132
kernel/proc/ eventbus.c	
Implementation of the event bus system	134
kernel/proc/ eventbus.h	
Definitions for the event bus system	134
kernel/proc/ sched.c	
Maintain task list and schedule tasks according to system clock ticks	135
kernel/proc/ sched.h	
Definition of scheduling related functions	137
kernel/proc/ signal.c	
Implementation of signal handling functions	138
kernel/proc/ signal.h	
Definitions for signal handling	139
kernel/proc/ syscall.c	
Implementation of system calls	141
kernel/proc/ syscall.h	
Definitions and declarations for system calls	143
kernel/proc/ task.c	
Implementation of task related functions	145
kernel/proc/ task.h	
Definition of task related functions	146
kernel/proc/ wait.h	
Definitions for process wait options and macros	148
kernel/sys/ acpi.c	
Implementation of ACPI (Advanced Configuration and Power Management Interface) functions	149
kernel/sys/ acpi.h	
Definition of ACPI (Advanced Configuration and Power Management Interface) related data structures	150
kernel/sys/ apic.c	
Implementation of APIC (Advanced Programmable Interrupt Controller) functions	151
kernel/sys/ apic.h	
Definition of APIC (Advanced Programmable Interrupt Controller) related data structures	154
kernel/sys/ cmos.c	
Implementation of CMOS related functions	158
kernel/sys/ cmos.h	
Definition of CMOS related data structures	159
kernel/sys/ cpu.c	
Initialize CPUs including BSP and other cores for SMP arch	160
kernel/sys/ cpu.h	
Definition of CPU related data structures and macros	161
kernel/sys/ gdt.c	
Implementation of GDT related functions	162
kernel/sys/ gdt.h	
Definition of GDT related data structures	163
kernel/sys/ hpet.c	
Implementation of HPET (High Precision Event Timer) functions	165
kernel/sys/ hpet.h	
Definition of HPET (High Precision Event Timer) related data structures	166

kernel/sys/idt.c	
Implementation of idt related functions	167
kernel/sys/idt.h	
Definition of IDT related data structures	168
kernel/sys/isr.c	
Implementation of ISR related functions	169
kernel/sys/isr_base.h	
Definition of ISR related data structures	170
kernel/sys/madt.c	
Implementation of ACPI MADT (Multiple APIC Description Table) functions	172
kernel/sys/madt.h	
Definition of ACPI MADT (Multiple APIC Description Table) related data structures	172
kernel/sys/mtrr.c	
MTRR configuration to speed up memory reading & writing operations	174
kernel/sys/mtrr.h	
Definition of MTRR macros and save/restore functions	175
kernel/sys/panic.h	
Implementation of panic related functions	176
kernel/sys/pci.c	
Implementation of PCI related functions	178
kernel/sys/pci.h	
Definition of PCI related data structures and functions	179
kernel/sys/pit.c	
Implementation of PIT related functions	184
kernel/sys/pit.h	
Definition of PIT related data structures	185
kernel/sys/serial.c	
Implementation of serial port communication functions	185
kernel/sys/serial.h	
Definitions for serial port communication	186
kernel/sys/smp.c	
SMP initialization and other support functions	187
kernel/sys/smp.h	
Definition of SMP related data structures and functions	188
kernel/sys/timer.c	
Implementation of APIC timer related functions	190
kernel/sys/timer.h	
Definition of APIC timer related macros, data structures, functions	191
libc/ctype.c	
Implementation of character type and conversion functions	193
libc/ctype.h	
Character type and conversion functions for HanOS standard library	194
libc/errno.h	
Error number definitions for HanOS standard library	195
libc/numeric.c	
Implementation of numeric utility functions	197
libc/numeric.h	
Numeric utility functions for HanOS standard library	198
libc/printf.h	
Printf, sprintf and snprintf implementation for HanOS standard library	199
libc/stdio.h	
Standard I/O definitions and declarations for HanOS standard library	202
libc/string.c	
Implementation of various string manipulation functions	204
libc/string.h	
Definition of string operation related functions	205
libc/sysfunc.c	
Implementation of system call functions	206

libc/ sysfunc.h	
System call function declarations and related macros	210
userspace/ cat.c	
Implementation of the 'cat' command for HanOS userspace	212
userspace/ echo.c	
Implementation of the 'echo' command for HanOS userspace	213
userspace/ hansh.c	
Implementation of the HanOS shell (hansh) for userspace	214
userspace/ help.h	
Definition of help command related data structures	216
userspace/ init.c	
Initialization process for HanOS userspace	217
userspace/ ls.c	
Implementation of the 'ls' command for HanOS userspace	219
userspace/ pwd.c	
Implementation of the 'pwd' command for HanOS userspace	219
userspace/ rm.c	
Implementation of the 'rm' command for HanOS userspace	220
userspace/ wc.c	
Implementation of the 'wc' command for HanOS userspace	221

Chapter 4

Data Structure Documentation

4.1 `acpi_gas_t` Struct Reference

Data Fields

- `uint8_t addr_space_id`
- `uint8_t reg_bit_width`
- `uint8_t reg_bit_offset`
- `uint8_t reserved`
- `uint64_t address`

The documentation for this struct was generated from the following file:

- `kernel/sys/acpi.h`

4.2 `acpi_sdt_hdr_t` Struct Reference

Data Fields

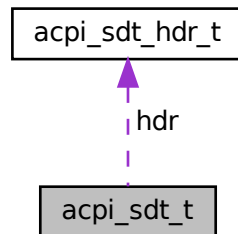
- `char sign [4]`
- `uint32_t length`
- `uint8_t rev`
- `uint8_t checksum`
- `char oem_id [6]`
- `char oem_table_id [8]`
- `uint32_t oem_rev`
- `uint32_t creator_id`
- `uint32_t creator_rev`

The documentation for this struct was generated from the following file:

- `kernel/sys/acpi.h`

4.3 acpi_sdt_t Struct Reference

Collaboration diagram for acpi_sdt_t:



Data Fields

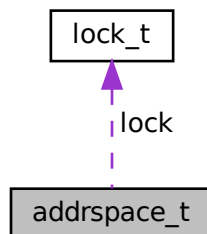
- [acpi_sdt_hdr_t](#) `hdr`
- `uint8_t data []`

The documentation for this struct was generated from the following file:

- `kernel/sys/acpi.h`

4.4 addrspace_t Struct Reference

Collaboration diagram for addrspace_t:



Public Member Functions

- `vec_struct (uint64_t) mem_list`

Data Fields

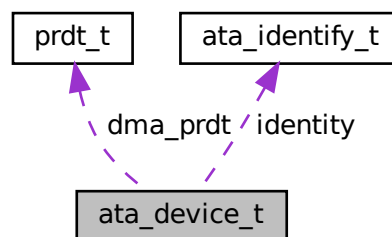
- uint64_t * **PML4**
- [lock_t](#) **lock**
- bool **initialized**

The documentation for this struct was generated from the following file:

- kernel/mm/[mm.h](#)

4.5 ata_device_t Struct Reference

Collaboration diagram for ata_device_t:



Data Fields

- int **io_base**
- int **control**
- int **slave**
- int **is_atapi**
- [ata_identify_t](#) **identity**
- [prdt_t](#) * **dma_prdt**
- uintptr_t **dma_prdt_phys**
- uint8_t * **dma_start**
- uintptr_t **dma_start_phys**
- uint32_t **bar4**
- uint32_t **atapi_lba**
- uint32_t **atapi_sector_size**

The documentation for this struct was generated from the following file:

- kernel/device/storage/[ata.h](#)

4.6 ata_identify_t Struct Reference

Data Fields

- uint16_t **flags**
- uint16_t **unused1** [9]
- char **serial** [20]
- uint16_t **unused2** [3]
- char **firmware** [8]
- char **model** [40]
- uint16_t **sectors_per_int**
- uint16_t **unused3**
- uint16_t **capabilities** [2]
- uint16_t **unused4** [2]
- uint16_t **valid_ext_data**
- uint16_t **unused5** [5]
- uint16_t **size_of_rw_mult**
- uint32_t **sectors_28**
- uint16_t **unused6** [38]
- uint64_t **sectors_48**
- uint16_t **unused7** [152]

The documentation for this struct was generated from the following file:

- kernel/device/storage/[ata.h](#)

4.7 atapi_command_t Union Reference

Data Fields

- uint8_t **command_bytes** [12]
- uint16_t **command_words** [6]

The documentation for this union was generated from the following file:

- kernel/device/storage/[ata.c](#)

4.8 auxval_t Struct Reference

Data Fields

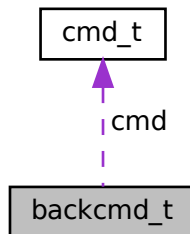
- uint64_t **entry**
- uint64_t **phdr**
- uint64_t **phaddr**
- uint16_t **phentsize**
- uint16_t **phnum**
- uint64_t **shdr**
- uint16_t **shnum**

The documentation for this struct was generated from the following file:

- kernel/proc/[task.h](#)

4.9 backcmd_t Struct Reference

Collaboration diagram for backcmd_t:



Data Fields

- int **type**
- [cmd_t](#) * **cmd**

The documentation for this struct was generated from the following file:

- userspace/[hansh.c](#)

4.10 bmp_header_t Struct Reference

Data Fields

- uint16_t **bf_signature**
- uint32_t **bf_size**
- uint32_t **reserved**
- uint32_t **bf_offset**
- uint32_t **bi_size**
- uint32_t **bi_width**
- uint32_t **bi_height**
- uint16_t **bi_planes**
- uint16_t **bi_bpp**
- uint32_t **bi_compression**
- uint32_t **bi_image_size**
- uint32_t **bi_xcount**
- uint32_t **bi_ycount**
- uint32_t **bi_clr_used**
- uint32_t **bi_clr_important**
- uint32_t **red_mask**
- uint32_t **green_mask**
- uint32_t **blue_mask**

The documentation for this struct was generated from the following file:

- kernel/base/[image.c](#)

4.11 cmd_t Struct Reference

Data Fields

- int **type**

The documentation for this struct was generated from the following file:

- userspace/[hansh.c](#)

4.12 cmos_rtc_t Struct Reference

Data Fields

- uint8_t **seconds**
- uint8_t **minutes**
- uint8_t **hours**
- uint8_t **weekdays**
- uint8_t **day**
- uint8_t **month**
- uint16_t **year**
- uint8_t **century**

The documentation for this struct was generated from the following file:

- kernel/sys/[cmos.h](#)

4.13 command_help_t Struct Reference

Data Fields

- char **command** [256]
- char **desc** [256]

The documentation for this struct was generated from the following file:

- libc/[sysfunc.h](#)

4.14 computer_info_t Struct Reference

Data Fields

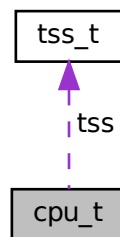
- uint64_t **screen_hor_size**
- uint64_t **screen_ver_size**
- uint64_t **prefer_res_x**
- uint64_t **prefer_res_y**
- uint64_t **actual_res_x**
- uint64_t **actual_res_y**

The documentation for this struct was generated from the following file:

- kernel/[kconfig.h](#)

4.15 cpu_t Struct Reference

Collaboration diagram for cpu_t:



Data Fields

- int64_t **errno**
- [tss_t](#) **tss**
- uint16_t **cpu_id**
- uint16_t **lapic_id**
- bool **is_bsp**
- uint64_t **fpu_storage_size**
- void(* **fpu_save**)(void *)
- void(* **fpu_restore**)(void *)
- uint16_t **proc_id**
- uint8_t **reserved**

The documentation for this struct was generated from the following file:

- kernel/sys/[smp.h](#)

4.16 cpuid_feature_t Struct Reference

Public Types

- enum { **CPUID_REG_EAX**, **CPUID_REG_EBX**, **CPUID_REG_ECX**, **CPUID_REG_EDX** }

Data Fields

- uint32_t **func**
- uint32_t **param**
- enum cpuid_feature_t:: { ... } **reg**
- uint32_t **mask**

The documentation for this struct was generated from the following file:

- kernel/sys/[cpu.h](#)

4.17 dirent_t Struct Reference

Data Fields

- ino_t **d_ino**
- off_t **d_off**
- uint16_t **d_reclen**
- uint8_t **d_type**
- char **d_name**[1024]
- uint64_t **d_ino**
- int64_t **d_off**

The documentation for this struct was generated from the following files:

- kernel/fs/[vfs.h](#)
- libc/[stdio.h](#)

4.18 edid_info_t Struct Reference

Data Fields

- uint8_t **padding** [8]
- uint16_t **manufacturer_id_be**
- uint16_t **edid_id_code**
- uint32_t **serial_num**
- uint8_t **man_week**
- uint8_t **man_year**
- uint8_t **edid_version**
- uint8_t **edid_revision**
- uint8_t **video_input_type**

- uint8_t **max_hor_size**
- uint8_t **max_ver_size**
- uint8_t **gamma_factor**
- uint8_t **dpms_flags**
- uint8_t **chroma_info** [10]
- uint8_t **est_timings1**
- uint8_t **est_timings2**
- uint8_t **est_timings3**
- - struct {
 uint8_t **resolution**
 uint8_t **frequency**
 } **std_timing_id** [8]
- - struct {
 uint16_t **pixel_clock**
 uint8_t **horz_active**
 uint8_t **horz_bank**
 uint8_t **horz_active_blank_msb**
 uint8_t **vert_active**
 uint8_t **vert_blank**
 uint8_t **vert_active_blank_msb**
 uint8_t **horz_sync_sffset**
 uint8_t **horz_sync_pulse**
 uint8_t **vert_sync**
 uint8_t **sync_msb**
 uint8_t **dimension_width**
 uint8_t **dimension_height**
 uint8_t **dimension_msb**
 uint8_t **horz_border**
 uint8_t **vert_border**
 uint8_t **features**
 } **det_timings** [4]
- uint8_t **unused**
- uint8_t **checksum**

The documentation for this struct was generated from the following file:

- kernel/device/display/[edid.h](#)

4.19 elf_hdr_t Struct Reference

Data Fields

- uint32_t **magic**
- uint8_t **elf** [12]
- uint16_t **type**
- uint16_t **machine**
- uint32_t **version**
- uint64_t **entry**

- uint64_t **phoff**
- uint64_t **shoff**
- uint32_t **flags**
- uint16_t **ehsize**
- uint16_t **phentsize**
- uint16_t **phnum**
- uint16_t **shentsize**
- uint16_t **shnum**
- uint16_t **shstrndx**

The documentation for this struct was generated from the following file:

- kernel/proc/[elf.h](#)

4.20 elf_nhdr_t Struct Reference

Data Fields

- uint32_t **name_size**
- uint32_t **desc_size**
- uint32_t **type**

The documentation for this struct was generated from the following file:

- kernel/proc/[elf.h](#)

4.21 elf_phdr_t Struct Reference

Data Fields

- uint32_t **type**
- uint32_t **flags**
- uint64_t **offset**
- uint64_t **vaddr**
- uint64_t **paddr**
- uint64_t **filesz**
- uint64_t **memsz**
- uint64_t **align**

The documentation for this struct was generated from the following file:

- kernel/proc/[elf.h](#)

4.22 elf_shdr_t Struct Reference

Data Fields

- uint32_t **name**
- uint32_t **type**
- uint64_t **flags**
- uint64_t **addr**
- uint64_t **offset**
- uint64_t **size**
- uint32_t **link**
- uint32_t **info**
- uint64_t **addralign**
- uint64_t **entsize**

The documentation for this struct was generated from the following file:

- kernel/proc/[elf.h](#)

4.23 elf_sym_t Struct Reference

Data Fields

- uint32_t **name**
- uint8_t **info**
- uint8_t **other**
- uint16_t **shndx**
- uint64_t **value**
- uint64_t **size**

The documentation for this struct was generated from the following file:

- kernel/proc/[elf.h](#)

4.24 event_t Struct Reference

Data Fields

- task_id_t **pub_tid**
- task_id_t **sub_tid**
- event_type_t **type**
- event_para_t **para**
- uint64_t **timestamp**

The documentation for this struct was generated from the following file:

- kernel/proc/[task.h](#)

4.25 execcmd_t Struct Reference

Data Fields

- int **type**
- char * **argv** [MAXARGS]
- char * **eargv** [MAXARGS]

The documentation for this struct was generated from the following file:

- userspace/[hansh.c](#)

4.26 fat32_bs_info_t Struct Reference

Data Fields

- uint16_t **bytes_per_sector**
- uint8_t **sectors_per_cluster**
- uint16_t **reserved_sector_count**
- uint8_t **num_fats**
- uint32_t **sectors_per_fat**
- uint32_t **root_dir_first_cluster**
- uint32_t **fat_begin_lba**
- uint32_t **cluster_begin_lba**
- uint32_t **total_sectors**

The documentation for this struct was generated from the following file:

- kernel/fs/[fat32.h](#)

4.27 fat32_entry_t Struct Reference

Data Fields

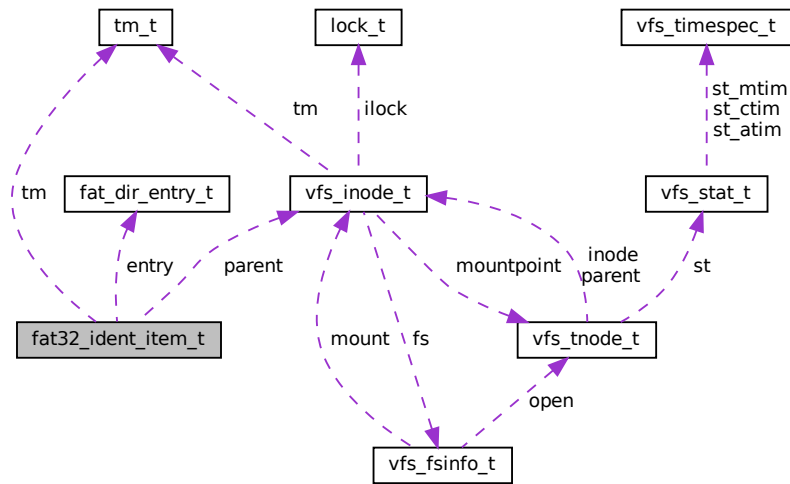
- uint8_t **name** [11]
- uint8_t **attribute**
- uint32_t **cluster_begin**
- uint32_t **file_size_bytes**
- uint32_t **dir_entry_cluster**
- uint64_t **dir_entry_index**

The documentation for this struct was generated from the following file:

- kernel/fs/[fat32.h](#)

4.28 fat32_ident_item_t Struct Reference

Collaboration diagram for fat32_ident_item_t:



Data Fields

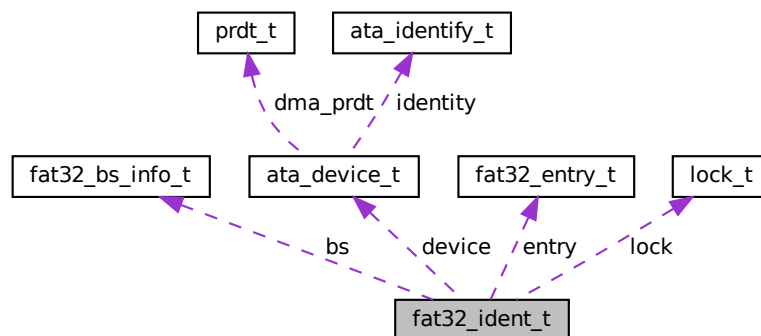
- [fat_dir_entry_t](#) `entry`
- [tm_t](#) `tm`
- `char name [VFS_MAX_NAME_LEN]`
- [vfs_inode_t](#) * `parent`

The documentation for this struct was generated from the following file:

- `kernel/fs/fat32.h`

4.29 fat32_ident_t Struct Reference

Collaboration diagram for fat32_ident_t:



Data Fields

- [ata_device_t](#) * **device**
- [lock_t](#) **lock**
- [fat32_bs_info_t](#) **bs**
- [fat32_entry_t](#) **entry**
- [uint32_t](#) * **fat**
- [uint64_t](#) **fat_len**

The documentation for this struct was generated from the following file:

- [kernel/fs/fat32.h](#)

4.30 fat_bs_t Struct Reference

Data Fields

- [uint8_t](#) **bootjmp** [3]
- [uint8_t](#) **oem_name** [8]
- [uint16_t](#) **bytes_per_sector**
- [uint8_t](#) **sectors_per_cluster**
- [uint16_t](#) **reserved_sector_count**
- [uint8_t](#) **table_count**
- [uint16_t](#) **root_entry_count**
- [uint16_t](#) **total_sectors_16**
- [uint8_t](#) **media_type**
- [uint16_t](#) **table_size_16**
- [uint16_t](#) **sectors_per_track**
- [uint16_t](#) **head_side_count**
- [uint32_t](#) **hidden_sector_count**
- [uint32_t](#) **total_sectors_32**
- [uint8_t](#) **extended_section** [54]

The documentation for this struct was generated from the following file:

- [kernel/fs/fat32.h](#)

4.31 fat_dir_entry_t Struct Reference

Data Fields

- [char](#) **file_name_and_ext** [8+3]
- [uint8_t](#) **attribute**
- [uint8_t](#) **file_data_1** [2]
- [uint16_t](#) **create_time**
- [uint16_t](#) **create_date**
- [uint16_t](#) **last_visit_date**
- [uint16_t](#) **cluster_num_high**
- [uint16_t](#) **modify_time**
- [uint16_t](#) **modify_date**
- [uint16_t](#) **cluster_num_low**
- [uint32_t](#) **file_size_bytes**

The documentation for this struct was generated from the following file:

- [kernel/fs/fat32.h](#)

4.32 fat_extbs_16_t Struct Reference

Data Fields

- uint8_t **bios_drive_num**
- uint8_t **reserved1**
- uint8_t **boot_signature**
- uint32_t **volume_id**
- uint8_t **volume_label** [11]
- uint8_t **fat_type_label** [8]

The documentation for this struct was generated from the following file:

- kernel/fs/[fat32.h](#)

4.33 fat_extbs_32_t Struct Reference

Data Fields

- uint32_t **table_size_32**
- uint16_t **extended_flags**
- uint16_t **fat_version**
- uint32_t **root_cluster**
- uint16_t **fat_info**
- uint16_t **backup_BS_sector**
- uint8_t **reserved_0** [12]
- uint8_t **drive_number**
- uint8_t **reserved_1**
- uint8_t **boot_signature**
- uint32_t **volume_id**
- uint8_t **volume_label** [11]
- uint8_t **fat_type_label** [8]

The documentation for this struct was generated from the following file:

- kernel/fs/[fat32.h](#)

4.34 fat_lfn_entry_t Struct Reference

Data Fields

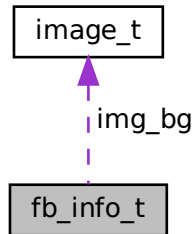
- uint8_t **sequence_number**
- char **name1** [10]
- uint8_t **attribute**
- uint8_t **type**
- uint8_t **dos_checksum**
- char **name2** [12]
- uint16_t **first_cluster**
- char **name3** [4]

The documentation for this struct was generated from the following file:

- kernel/fs/[fat32.h](#)

4.35 fb_info_t Struct Reference

Collaboration diagram for fb_info_t:



Data Fields

- uint8_t * **addr**
- uint8_t * **bgbuffer**
- uint8_t * **swapbuffer**
- uint32_t **width**
- uint32_t **height**
- uint32_t **pitch**
- uint8_t * **backbuffer**
- uint32_t **backbuffer_len**
- [image_t](#) **img_bg**

The documentation for this struct was generated from the following file:

- kernel/device/display/[fb.h](#)

4.36 file_dup_t Struct Reference

Data Fields

- vfs_handle_t **fh**
- vfs_handle_t **newfh**

The documentation for this struct was generated from the following file:

- kernel/proc/[task.h](#)

4.37 font_psf1_t Struct Reference

Data Fields

- uint8_t **magic** [2]
- uint8_t **mode**
- uint8_t **charsize**
- uint8_t **data** []

The documentation for this struct was generated from the following file:

- kernel/device/display/[font.h](#)

4.38 font_psf2_t Struct Reference

Data Fields

- uint32_t **magic**
- uint32_t **version**
- uint32_t **headersize**
- uint32_t **flags**
- uint32_t **numglyph**
- uint32_t **glyph_size**
- uint32_t **height**
- uint32_t **width**
- uint8_t **data** []

The documentation for this struct was generated from the following file:

- kernel/device/display/[font.h](#)

4.39 gdt_entry_t Struct Reference

Data Fields

- uint16_t **limit**
- uint16_t **base_low**
- uint8_t **base_mid**
- uint8_t **access**
- uint8_t **granularity**
- uint8_t **base_high**

The documentation for this struct was generated from the following file:

- kernel/sys/[gdt.h](#)

4.40 gdt_register_t Struct Reference

Data Fields

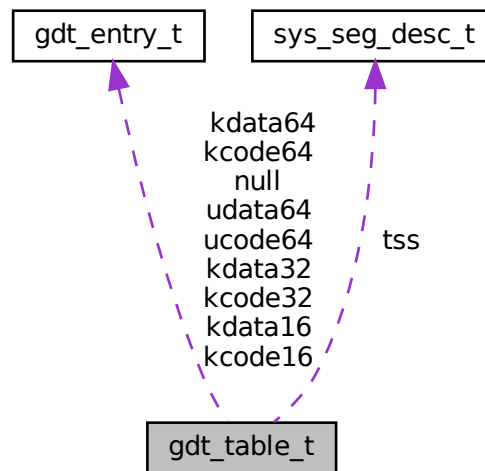
- `uint16_t` **size**
- `uint64_t` **offset**

The documentation for this struct was generated from the following file:

- `kernel/sys/gdt.h`

4.41 gdt_table_t Struct Reference

Collaboration diagram for `gdt_table_t`:



Data Fields

- `gdt_entry_t` **null**
- `gdt_entry_t` **kcode16**
- `gdt_entry_t` **kdata16**
- `gdt_entry_t` **kcode32**
- `gdt_entry_t` **kdata32**
- `gdt_entry_t` **kcode64**
- `gdt_entry_t` **kdata64**
- `gdt_entry_t` **udata64**
- `gdt_entry_t` **ucode64**
- `sys_seg_desc_t` **tss**

The documentation for this struct was generated from the following file:

- `kernel/sys/gdt.h`

4.42 gfx_gtt_t Struct Reference

Data Fields

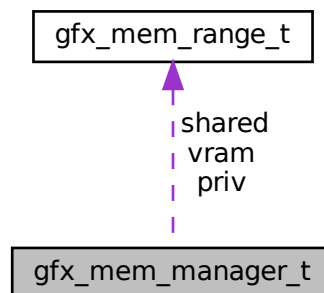
- uint32_t **stolen_mem_size**
- uint32_t **gtt_mem_size**
- uint32_t **stolen_mem_base**
- uint32_t **num_total_entries**
- uint32_t **num_mappable_entries**
- volatile uint32_t * **entries**

The documentation for this struct was generated from the following file:

- kernel/device/display/[gfx.h](#)

4.43 gfx_mem_manager_t Struct Reference

Collaboration diagram for gfx_mem_manager_t:



Data Fields

- [gfx_mem_range_t](#) **vram**
- [gfx_mem_range_t](#) **shared**
- [gfx_mem_range_t](#) **priv**
- volatile uint8_t * **gfx_mem_base**
- volatile uint8_t * **gfx_mem_next**

The documentation for this struct was generated from the following file:

- kernel/device/display/[gfx.h](#)

4.44 gfx_mem_range_t Struct Reference

Data Fields

- uint64_t **base**
- uint64_t **top**
- uint64_t **current**

The documentation for this struct was generated from the following file:

- [kernel/device/display/gfx.h](#)

4.45 gfx_object_t Struct Reference

Data Fields

- volatile uint8_t * **cpu_addr**
- volatile uint64_t **gfx_addr**

The documentation for this struct was generated from the following file:

- [kernel/device/display/gfx.h](#)

4.46 gfx_pci_t Struct Reference

Data Fields

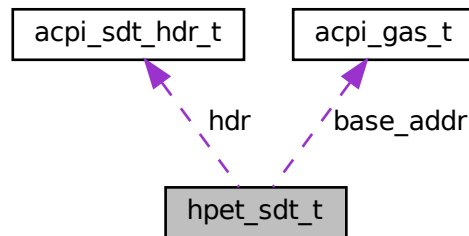
- uint32_t **id**
- volatile void * **aperture_bar**
- volatile void * **mmio_bar**
- volatile uint32_t * **gtt_addr**
- uint16_t **iobase**
- uint32_t **aperture_size**

The documentation for this struct was generated from the following file:

- [kernel/device/display/gfx.h](#)

4.47 hpet_sdt_t Struct Reference

Collaboration diagram for hpet_sdt_t:



Data Fields

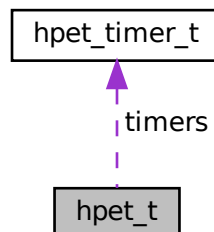
- [acpi_sdt_hdr_t](#) `hdr`
- `uint8_t` `hardware_rev_id`
- `uint8_t` `comparator_count`: 5
- `uint8_t` `counter_size`: 1
- `uint8_t` `reserved`: 1
- `uint8_t` `legacy_replace`: 1
- `uint16_t` `pci_vendor_id`
- [acpi_gas_t](#) `base_addr`
- `uint8_t` `hpet_number`
- `uint16_t` `minimum_tick`
- `uint8_t` `page_protection`

The documentation for this struct was generated from the following file:

- [kernel/sys/hpet.h](#)

4.48 hpet_t Struct Reference

Collaboration diagram for hpet_t:



Data Fields

- volatile uint64_t **general_capabilities**
- volatile uint64_t **unused0**
- volatile uint64_t **general_configuration**
- volatile uint64_t **unused1**
- volatile uint64_t **general_int_status**
- volatile uint64_t **unused2**
- volatile uint64_t **unused3** [2][12]
- volatile uint64_t **main_counter_value**
- volatile uint64_t **unused4**
- [hpet_timer_t](#) **timers** []

The documentation for this struct was generated from the following file:

- kernel/sys/[hpet.h](#)

4.49 hpet_timer_t Struct Reference

Data Fields

- volatile uint64_t **config_and_capabilities**
- volatile uint64_t **comparator_value**
- volatile uint64_t **fsb_interrupt_route**
- volatile uint64_t **unused**

The documentation for this struct was generated from the following file:

- kernel/sys/[hpet.h](#)

4.50 ht_item_t Struct Reference

Data Fields

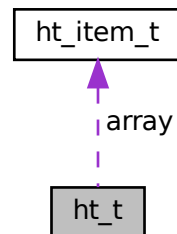
- int64_t **key**
- void * **data**

The documentation for this struct was generated from the following file:

- kernel/base/[hash.h](#)

4.51 ht_t Struct Reference

Collaboration diagram for ht_t:



Data Fields

- uint64_t **size**
- [ht_item_t](#) * **array**

The documentation for this struct was generated from the following file:

- [kernel/base/hash.h](#)

4.52 ide_channel_regs_t Struct Reference

Data Fields

- uint16_t **base**
- uint16_t **ctrl**
- uint16_t **bmide**
- uint16_t **nien**

The documentation for this struct was generated from the following file:

- [kernel/device/storage/ata.h](#)

4.53 ide_device_t Struct Reference

Data Fields

- uint8_t **reserved**
- uint8_t **channel**
- uint8_t **drive**
- uint16_t **type**
- uint16_t **signature**
- uint16_t **capabilities**
- uint32_t **command_sets**
- uint32_t **size**
- uint8_t **model** [41]

The documentation for this struct was generated from the following file:

- kernel/device/storage/[ata.h](#)

4.54 idt_entry_t Struct Reference

Data Fields

- uint16_t **offset_1**
- uint16_t **selector**
- uint8_t **ist**
- uint8_t **type_attributes**
- uint16_t **offset_2**
- uint32_t **offset_3**
- uint32_t **zero**

The documentation for this struct was generated from the following file:

- kernel/sys/[idt.h](#)

4.55 idt_register_t Struct Reference

Data Fields

- uint16_t **size**
- uint64_t **offset**

The documentation for this struct was generated from the following file:

- kernel/sys/[idt.h](#)

4.56 image_t Struct Reference

Data Fields

- uint8_t * **img**
- uint64_t **size**
- uint64_t **pitch**
- uint64_t **bpp**
- uint64_t **img_width**
- uint64_t **img_height**

The documentation for this struct was generated from the following file:

- kernel/base/[image.h](#)

4.57 keyboard_t Struct Reference

Data Fields

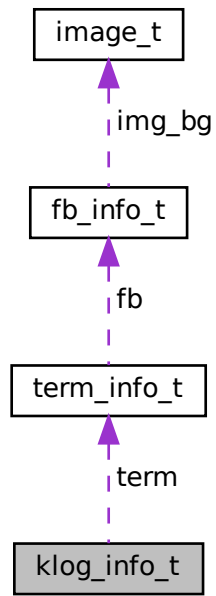
- int32_t **mouse_x_offset**
- int32_t **mouse_y_offset**
- int32_t **mouse_x**
- int32_t **mouse_y**
- uint8_t **mouse_cycle**
- uint8_t **last_keypress**
- bool **key_pressed** [KEY_COUNT]
- uint8_t * **ptr_to_update**

The documentation for this struct was generated from the following file:

- kernel/device/keyboard/[keyboard.h](#)

4.58 klog_info_t Struct Reference

Collaboration diagram for klog_info_t:



Data Fields

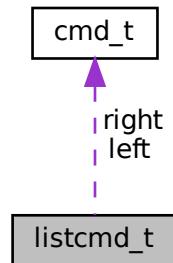
- `uint8_t` **buff** [KLOG_BUFFER_SIZE]
- `int` **start**
- `int` **end**
- `term_info_t` * **term**

The documentation for this struct was generated from the following file:

- `kernel/base/klog.h`

4.59 listcmd_t Struct Reference

Collaboration diagram for listcmd_t:



Data Fields

- int **type**
- [cmd_t](#) * **left**
- [cmd_t](#) * **right**

The documentation for this struct was generated from the following file:

- userspace/[hansh.c](#)

4.60 lock_t Struct Reference

Data Fields

- uint32_t **lock**
- uint64_t **rflags**

The documentation for this struct was generated from the following file:

- kernel/base/[lock.h](#)

4.61 madt_record_hdr_t Struct Reference

Data Fields

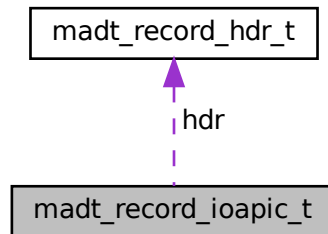
- uint8_t **type**
- uint8_t **len**

The documentation for this struct was generated from the following file:

- kernel/sys/[madt.h](#)

4.62 madt_record_ioapic_t Struct Reference

Collaboration diagram for madt_record_ioapic_t:



Data Fields

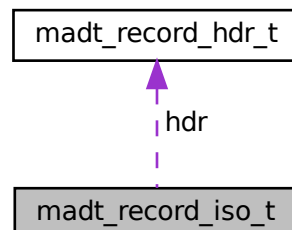
- [madt_record_hdr_t](#) **hdr**
- `uint8_t` **id**
- `uint8_t` **reserved**
- `uint32_t` **addr**
- `uint32_t` **gsi_base**

The documentation for this struct was generated from the following file:

- [kernel/sys/madt.h](#)

4.63 madt_record_iso_t Struct Reference

Collaboration diagram for madt_record_iso_t:



Data Fields

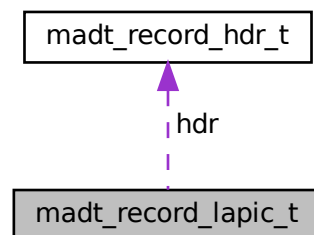
- [madt_record_hdr_t](#) **hdr**
- uint8_t **bus_src**
- uint8_t **irq_src**
- uint32_t **gsi**
- uint16_t **flags**

The documentation for this struct was generated from the following file:

- kernel/sys/[madt.h](#)

4.64 madt_record_lapic_t Struct Reference

Collaboration diagram for madt_record_lapic_t:



Data Fields

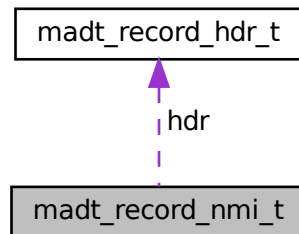
- [madt_record_hdr_t](#) **hdr**
- uint8_t **proc_id**
- uint8_t **apic_id**
- uint32_t **flags**

The documentation for this struct was generated from the following file:

- kernel/sys/[madt.h](#)

4.65 madt_record_nmi_t Struct Reference

Collaboration diagram for madt_record_nmi_t:



Data Fields

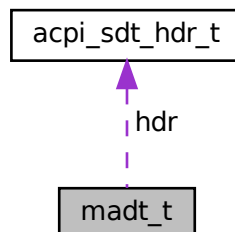
- [madt_record_hdr_t](#) `hdr`
- `uint8_t` `proc_id`
- `uint16_t` `flags`
- `uint8_t` `lint`

The documentation for this struct was generated from the following file:

- [kernel/sys/madt.h](#)

4.66 madt_t Struct Reference

Collaboration diagram for madt_t:



Data Fields

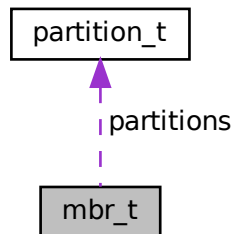
- [acpi_sdt_hdr_t](#) **hdr**
- [uint32_t](#) **lapic_addr**
- [uint32_t](#) **flags**
- [uint8_t](#) **records** []

The documentation for this struct was generated from the following file:

- [kernel/sys/madt.h](#)

4.67 mbr_t Struct Reference

Collaboration diagram for mbr_t:



Data Fields

- [uint8_t](#) **bootstrap** [446]
- [partition_t](#) **partitions** [4]
- [uint8_t](#) **signature** [2]

The documentation for this struct was generated from the following file:

- [kernel/device/storage/ata.h](#)

4.68 mem_info_t Struct Reference

Data Fields

- [uint64_t](#) **phys_limit**
- [uint64_t](#) **total_size**
- [uint64_t](#) **free_size**
- [uint8_t](#) * **bitmap**

The documentation for this struct was generated from the following file:

- [kernel/mm/mm.h](#)

4.69 mem_map_t Struct Reference

Data Fields

- uint64_t **vaddr**
- uint64_t **paddr**
- uint64_t **flags**
- uint64_t **np**

The documentation for this struct was generated from the following file:

- [kernel/mm/mm.h](#)

4.70 memory_metadata_t Struct Reference

Data Fields

- uint64_t **magic**
- uint64_t **checkno**
- uint64_t **numpages**
- uint64_t **size**
- char **filename** [512]
- uint64_t **lineno**

The documentation for this struct was generated from the following file:

- [kernel/base/kmalloc.h](#)

4.71 out_fct_wrap_type Struct Reference

Data Fields

- void(* **fct**)(char character, void *arg)
- void * **arg**

The documentation for this struct was generated from the following file:

- [libc/printf.c](#)

4.72 partition_t Struct Reference

Data Fields

- uint8_t **status**
- uint8_t **chs_start** [3]
- uint8_t **type**
- uint8_t **chs_end** [3]
- uint32_t **lba_start**
- uint32_t **sector_count**

The documentation for this struct was generated from the following file:

- kernel/device/storage/[ata.h](#)

4.73 pci_bar_t Struct Reference

Data Fields

- ```
union {
 void * address
 uint16_t port
} u
```
- uint64\_t **size**
- uint32\_t **flags**

The documentation for this struct was generated from the following file:

- kernel/sys/[pci.h](#)

## 4.74 pci\_device\_desc\_t Struct Reference

### Data Fields

- uint16\_t **vendor\_id**
- uint16\_t **device\_id**
- char **desc** [256]

The documentation for this struct was generated from the following file:

- kernel/sys/[pci.h](#)

## 4.75 pci\_device\_t Struct Reference

### Data Fields

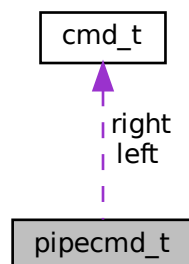
- int64\_t **parent**
- uint8\_t **bus**
- uint8\_t **func**
- uint8\_t **device**
- uint16\_t **device\_id**
- uint16\_t **vendor\_id**
- uint8\_t **rev\_id**
- uint8\_t **subclass**
- uint8\_t **device\_class**
- uint8\_t **prog\_if**
- int **multifunction**
- uint8\_t **irq\_pin**
- int **has\_prt**
- uint32\_t **gsi**
- uint16\_t **gsi\_flags**

The documentation for this struct was generated from the following file:

- kernel/sys/[pci.h](#)

## 4.76 pipecmd\_t Struct Reference

Collaboration diagram for pipecmd\_t:



### Data Fields

- int **type**
- [cmd\\_t](#) \* **left**
- [cmd\\_t](#) \* **right**

The documentation for this struct was generated from the following file:

- userspace/[hansh.c](#)

## 4.77 pipefs\_file\_t Struct Reference

### Data Fields

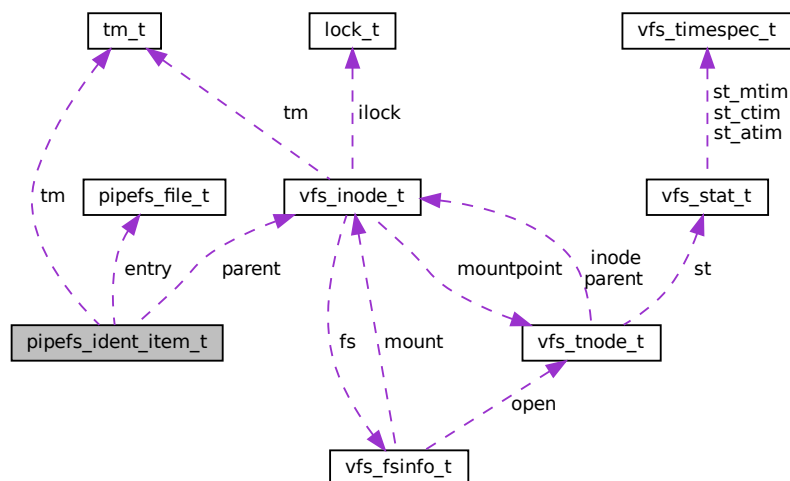
- char **name** [VFS\_MAX\_NAME\_LEN]

The documentation for this struct was generated from the following file:

- [kernel/fs/pipefs.h](#)

## 4.78 pipefs\_ident\_item\_t Struct Reference

Collaboration diagram for pipefs\_ident\_item\_t:



### Data Fields

- [pipefs\\_file\\_t](#) **entry**
- [tm\\_t](#) **tm**
- char **name** [VFS\_MAX\_NAME\_LEN]
- [vfs\\_inode\\_t](#) \* **parent**

The documentation for this struct was generated from the following file:

- [kernel/fs/pipefs.h](#)

## 4.79 pipefs\_ident\_t Struct Reference

### Data Fields

- char **buff** [PIPE\_BUFFER\_SIZE]
- int64\_t **size**
- bool **closed**

The documentation for this struct was generated from the following file:

- [kernel/fs/pipefs.c](#)

## 4.80 prdt\_t Struct Reference

### Data Fields

- uintptr\_t **offset**
- uint16\_t **bytes**
- uint16\_t **last**

The documentation for this struct was generated from the following file:

- [kernel/device/storage/ata.h](#)

## 4.81 ramfs\_file\_t Struct Reference

### Data Fields

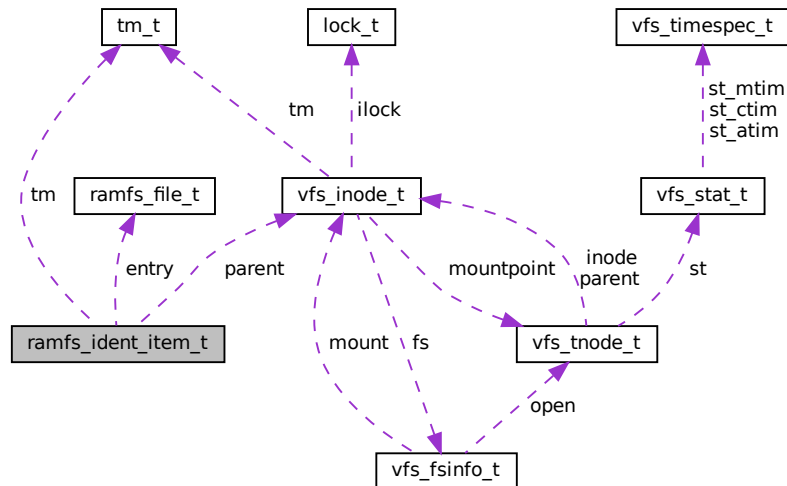
- char **name** [VFS\_MAX\_NAME\_LEN]
- void \* **data**
- uint64\_t **size**

The documentation for this struct was generated from the following file:

- [kernel/fs/ramfs.h](#)

## 4.82 ramfs\_ident\_item\_t Struct Reference

Collaboration diagram for ramfs\_ident\_item\_t:



### Data Fields

- [ramfs\\_file\\_t](#) **entry**
- `vfs_node_type_t` **type**
- [tm\\_t](#) **tm**
- `char` **name** [VFS\_MAX\_NAME\_LEN]
- `char` **path** [VFS\_MAX\_NAME\_LEN]
- [vfs\\_inode\\_t](#) \* **parent**

The documentation for this struct was generated from the following file:

- `kernel/fs/ramfs.h`

## 4.83 ramfs\_ident\_t Struct Reference

### Data Fields

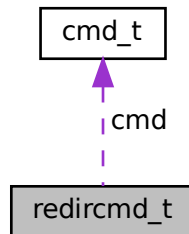
- `uint64_t` **alloc\_size**
- `void *` **data**

The documentation for this struct was generated from the following file:

- `kernel/fs/ramfs.c`

## 4.84 redircmd\_t Struct Reference

Collaboration diagram for redircmd\_t:



### Data Fields

- int **type**
- [cmd\\_t](#) \* **cmd**
- char \* **file**
- char \* **efile**
- int **mode**
- int **fd**

The documentation for this struct was generated from the following file:

- userspace/[hansh.c](#)

## 4.85 rsdp\_t Struct Reference

### Data Fields

- char **sign** [8]
- uint8\_t **checksum**
- char **oem\_id** [6]
- uint8\_t **revision**
- uint32\_t **rsdt\_addr**
- uint32\_t **length**
- uint64\_t **xsdt\_addr**
- uint8\_t **checksum\_ext**
- uint8\_t **reserved** [3]

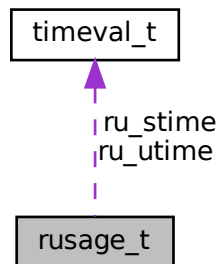
The documentation for this struct was generated from the following file:

- kernel/sys/[acpi.h](#)



## 4.86 `rusage_t` Struct Reference

Collaboration diagram for `rusage_t`:



### Data Fields

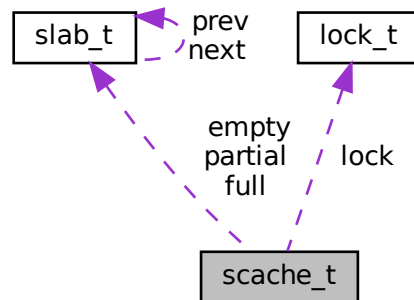
- [timeval\\_t](#) `ru_utime`
- [timeval\\_t](#) `ru_stime`
- `int64_t` `ru_maxrss`
- `int64_t` `ru_ixrss`
- `int64_t` `ru_idrss`
- `int64_t` `ru_isrss`
- `int64_t` `ru_minflt`
- `int64_t` `ru_majflt`
- `int64_t` `ru_nswap`
- `int64_t` `ru_inblock`
- `int64_t` `ru_oublock`
- `int64_t` `ru_msgsnd`
- `int64_t` `ru_msgrcv`
- `int64_t` `ru_nsignals`
- `int64_t` `ru_nvcsw`
- `int64_t` `ru_nivcsw`

The documentation for this struct was generated from the following file:

- [kernel/proc/syscall.h](#)

## 4.87 `scache_t` Struct Reference

Collaboration diagram for `scache_t`:



### Data Fields

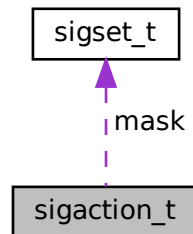
- `lock_t lock`
- `void(* ctor)(struct scache_t *cache, void *obj)`
- `void(* dtor)(struct scache_t *cache, void *obj)`
- `slab_t * full`
- `slab_t * partial`
- `slab_t * empty`
- `uint64_t size`
- `uint64_t truesize`
- `uint64_t alignment`
- `uint64_t slabobjcount`

The documentation for this struct was generated from the following file:

- `kernel/mm/slab.h`

## 4.88 sigaction\_t Struct Reference

Collaboration diagram for sigaction\_t:



### Data Fields

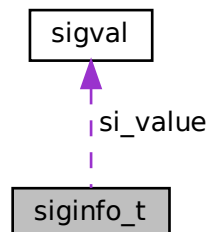
- void \* **address**
- [sigset\\_t](#) **mask**
- int **flags**
- void(\* **restorer** )(void)

The documentation for this struct was generated from the following file:

- kernel/proc/[signal.h](#)

## 4.89 siginfo\_t Struct Reference

Collaboration diagram for siginfo\_t:



## Data Fields

- int **si\_signo**
- int **si\_errno**
- int **si\_code**
- 

```

union {
 char __pad [128 - 2 *sizeof(int) - sizeof(long)]
 struct {
 union {
 struct {
 pid_t si_pid
 uid_t si_uid
 } __piduid
 struct {
 int si_timerid
 int si_overrun
 } __timer
 } __first
 union {
 union sigval si_value
 struct {
 int si_status
 clock_t si_utime
 clock_t si_stime
 } __sigchld
 } __second
 } __si_common
 struct {
 void * si_addr
 short si_addr_lsb
 union {
 struct {
 void * si_lower
 void * si_upper
 } __addr_bnd
 unsigned si_pkey
 } __first
 } __sigfault
 struct {
 long si_band
 int si_fd
 } __sigpoll
 struct {
 void * si_call_addr
 int si_syscall
 unsigned si_arch
 } __sigsys
} __si_fields

```

The documentation for this struct was generated from the following file:

- kernel/proc/[signal.h](#)

## 4.90 sigset\_t Struct Reference

### Data Fields

- uint64\_t **sig**

The documentation for this struct was generated from the following file:

- kernel/proc/[signal.h](#)

## 4.91 signal Union Reference

### Data Fields

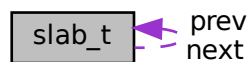
- int **sival\_int**
- void \* **sival\_ptr**

The documentation for this union was generated from the following file:

- kernel/proc/[signal.h](#)

## 4.92 slab\_t Struct Reference

Collaboration diagram for slab\_t:



### Data Fields

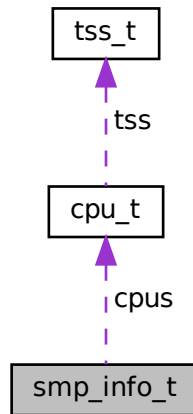
- struct [slab\\_t](#) \* **next**
- struct [slab\\_t](#) \* **prev**
- uint64\_t **used**
- void \*\* **free**
- void \* **base**

The documentation for this struct was generated from the following file:

- kernel/mm/[slab.h](#)

## 4.93 smp\_info\_t Struct Reference

Collaboration diagram for smp\_info\_t:



### Data Fields

- [cpu\\_t](#) **cpus** [CPU\_MAX]
- `uint16_t` **num\_cpus**

The documentation for this struct was generated from the following file:

- `kernel/sys/smp.h`

## 4.94 stack\_t Struct Reference

### Data Fields

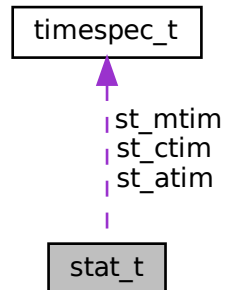
- `void *` **base**
- `int64_t` **flags**
- `uint64_t` **size**

The documentation for this struct was generated from the following file:

- `kernel/proc/signal.h`

## 4.95 stat\_t Struct Reference

Collaboration diagram for stat\_t:



### Data Fields

- `int64_t st_dev`
- `uint64_t st_ino`
- `int32_t st_mode`
- `int32_t st_nlink`
- `int32_t st_uid`
- `int32_t st_gid`
- `int64_t st_rdev`
- `int64_t st_size`
- `timespec_t st_atim`
- `timespec_t st_mtim`
- `timespec_t st_ctim`
- `int64_t st_blksize`
- `int64_t st_blocks`

The documentation for this struct was generated from the following file:

- `libc/stdio.h`

## 4.96 symbol\_t Struct Reference

### Data Fields

- `uint64_t addr`
- `char * name`

The documentation for this struct was generated from the following file:

- `kernel/symbols.h`

## 4.97 sys\_seg\_desc\_t Struct Reference

### Data Fields

- uint32\_t **segment\_limit\_low**: 16
- uint32\_t **segment\_base\_low**: 16
- uint32\_t **segment\_base\_mid**: 8
- uint32\_t **segment\_type**: 4
- uint32\_t **zero\_1**: 1
- uint32\_t **segment\_dpl**: 2
- uint32\_t **segment\_present**: 1
- uint32\_t **segment\_limit\_high**: 4
- uint32\_t **segment\_avail**: 1
- uint32\_t **zero\_2**: 2
- uint32\_t **segment\_gran**: 1
- uint32\_t **segment\_base\_mid2**: 8
- uint32\_t **segment\_base\_high**: 32
- uint32\_t **reserved\_1**: 8
- uint32\_t **zero\_3**: 5
- uint32\_t **reserved\_2**: 19

The documentation for this struct was generated from the following file:

- [kernel/sys/gdt.h](#)

## 4.98 task\_regs\_t Struct Reference

### Data Fields

- uint64\_t **r15**
- uint64\_t **r14**
- uint64\_t **r13**
- uint64\_t **r12**
- uint64\_t **r11**
- uint64\_t **r10**
- uint64\_t **r9**
- uint64\_t **r8**
- uint64\_t **rbp**
- uint64\_t **rdi**
- uint64\_t **rsi**
- uint64\_t **rdx**
- uint64\_t **rcx**
- uint64\_t **rbx**
- uint64\_t **rax**
- uint64\_t **rip**
- uint64\_t **cs**
- uint64\_t **rflags**
- uint64\_t **rsp**
- uint64\_t **ss**

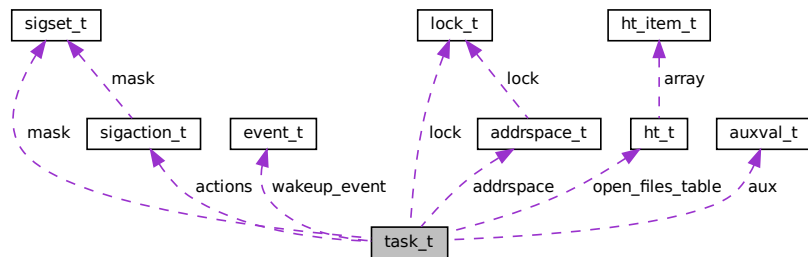
The documentation for this struct was generated from the following file:

- [kernel/proc/task.h](#)



## 4.99 task\_t Struct Reference

Collaboration diagram for task\_t:



### Public Member Functions

- **vec\_struct** (task\_id\_t) child\_list
- **vec\_struct** (file\_dup\_t) dup\_list
- **vec\_struct** (mem\_map\_t) mmap\_list

### Data Fields

- void \* **tstack\_top**
- void \* **tstack\_limit**
- void \* **kstack\_top**
- void \* **kstack\_limit**
- void \* **ustack\_top**
- void \* **ustack\_limit**
- task\_id\_t **tid**
- task\_id\_t **ptid**
- task\_priority\_t **priority**
- uint64\_t **last\_tick**
- uint64\_t **wakeup\_time**
- **event\_t** **wakeup\_event**
- task\_status\_t **status**
- task\_mode\_t **mode**
- bool **isforked**
- **auxval\_t** **aux**
- **ht\_t** **open\_files\_table**
- int64\_t **errno**
- **addrspace\_t** \* **addrspace**
- uint64\_t **fs\_base**
- char **cwd** [VFS\_MAX\_PATH\_LEN]
- char **name** [64]
-

```

struct {
 lock_t lock
 sigaction_t actions [NSIG]
 sigset_t mask
} signals

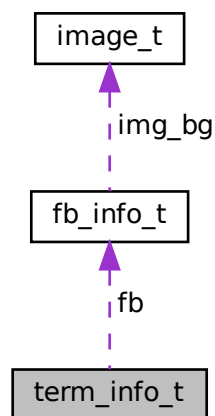
```

The documentation for this struct was generated from the following file:

- [kernel/proc/task.h](#)

## 4.100 term\_info\_t Struct Reference

Collaboration diagram for term\_info\_t:



### Public Types

- enum {  
**STATE\_UNKNOWN** = 0, **STATE\_IDLE**, **STATE\_CMD**, **STATE\_PARAM**,  
**STATE\_HYPERLINK\_HEADER**, **STATE\_HYPERLINK\_URL**, **STATE\_HYPERLINK\_TEXT**, **STATE\_HYPERLINK\_TAIL** }

### Data Fields

- [fb\\_info\\_t](#) **fb**
- uint32\_t **fgcolor**
- uint32\_t **bgcolor**
- bool **bold**
- uint32\_t **width**
- uint32\_t **height**

- `uint32_t cursor_x`
- `uint32_t cursor_y`
- `enum term_info_t:: { ... } state`
- `int cparams [16]`
- `int cparamcount`
- `uint8_t lastch`
- `bool last_qu_char`
- `int skip_left`

The documentation for this struct was generated from the following file:

- `kernel/device/display/term.h`

## 4.101 `termios_t` Struct Reference

### Data Fields

- `tcflag_t c_iflag`
- `tcflag_t c_oflag`
- `tcflag_t c_cflag`
- `tcflag_t c_lflag`
- `cc_t c_cc [NCCS]`
- `speed_t ibaud`
- `speed_t obaud`

The documentation for this struct was generated from the following file:

- `kernel/fs/ttyfs.c`

## 4.102 `timespec_t` Struct Reference

### Data Fields

- `int64_t tv_sec`
- `int64_t tv_nsec`

The documentation for this struct was generated from the following file:

- `libc/stdio.h`

## 4.103 `timeval_t` Struct Reference

### Data Fields

- `uint64_t tv_sec`
- `uint64_t tv_usec`

The documentation for this struct was generated from the following file:

- `kernel/proc/syscall.h`

## 4.104 `timezone_t` Struct Reference

### Data Fields

- int **minuteswest**
- int **dsttime**

The documentation for this struct was generated from the following file:

- kernel/base/[time.h](#)

## 4.105 `tm_t` Struct Reference

### Data Fields

- int **sec**
- int **min**
- int **hour**
- int **mday**
- int **mon**
- int **year**
- int **wday**
- int **yday**
- int **isdst**

The documentation for this struct was generated from the following file:

- kernel/base/[time.h](#)

## 4.106 `tss_t` Struct Reference

### Data Fields

- uint32\_t **reserved**
- uint64\_t **rsp0**
- uint64\_t **rsp1**
- uint64\_t **rsp2**
- uint32\_t **reserved\_1**
- uint32\_t **reserved\_2**
- uint64\_t **ist1**
- uint64\_t **ist2**
- uint64\_t **ist3**
- uint64\_t **ist4**
- uint64\_t **ist5**
- uint64\_t **ist6**
- uint64\_t **ist7**
- uint64\_t **reserved\_3**
- uint16\_t **reserved\_4**
- uint16\_t **io\_bitmap\_offset**

The documentation for this struct was generated from the following file:

- kernel/sys/[smp.h](#)

## 4.107 ttyfs\_file\_t Struct Reference

### Data Fields

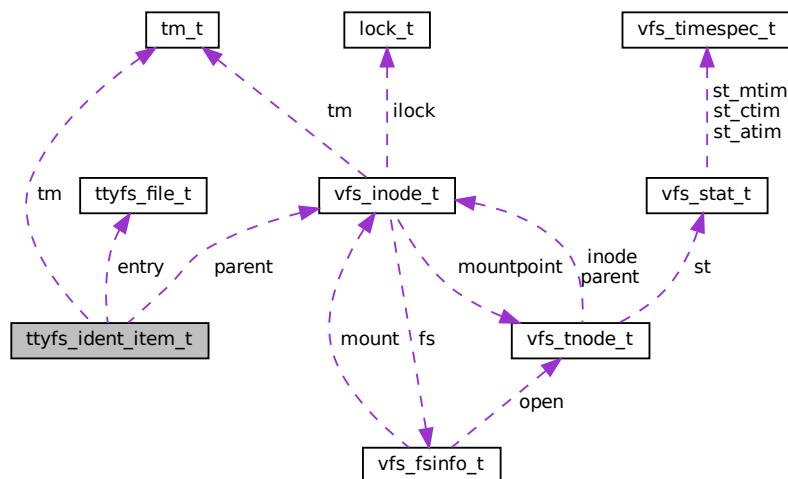
- char **name** [VFS\_MAX\_NAME\_LEN]

The documentation for this struct was generated from the following file:

- [kernel/fs/ttyfs.h](#)

## 4.108 ttyfs\_ident\_item\_t Struct Reference

Collaboration diagram for ttyfs\_ident\_item\_t:



### Data Fields

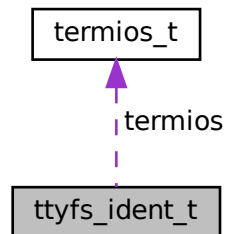
- [ttyfs\\_file\\_t](#) **entry**
- [tm\\_t](#) **tm**
- char **name** [VFS\_MAX\_NAME\_LEN]
- [vfs\\_inode\\_t](#) \* **parent**

The documentation for this struct was generated from the following file:

- [kernel/fs/ttyfs.h](#)

## 4.109 ttyfs\_ident\_t Struct Reference

Collaboration diagram for `ttyfs_ident_t`:



### Data Fields

- `char ibuff` [TTY\_BUFFER\_SIZE]
- `int64_t ibegin`
- `int64_t icursor`
- `int64_t isize`
- `termios_t termios`

The documentation for this struct was generated from the following file:

- `kernel/fs/ttyfs.c`

## 4.110 ustar\_file\_t Struct Reference

### Data Fields

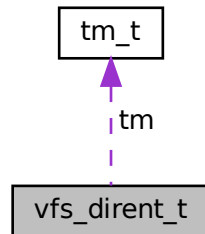
- `char name` [100]
- `uint64_t mode`
- `uint64_t owner_id`
- `uint64_t group_id`
- `uint8_t size` [12]
- `uint8_t last_modified` [12]
- `uint64_t checksum`
- `uint8_t type`
- `uint8_t linked_file_name` [100]
- `uint8_t indicator`
- `uint8_t version` [2]
- `uint8_t owner_user_name` [32]
- `uint8_t owner_group_name` [32]
- `uint64_t dev_major_number`
- `uint64_t dev_minor_number`
- `uint8_t filename_prefix` [155]

The documentation for this struct was generated from the following file:

- `kernel/fs/ramfs.h`

## 4.111 vfs\_dirent\_t Struct Reference

Collaboration diagram for vfs\_dirent\_t:



### Data Fields

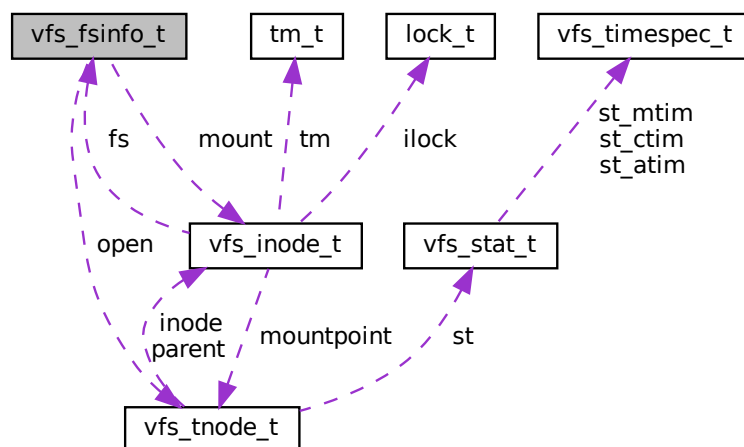
- `vfs_node_type_t` **type**
- `tm_t` **tm**
- `char` **name** [VFS\_MAX\_NAME\_LEN]
- `uint64_t` **size**

The documentation for this struct was generated from the following file:

- `kernel/fs/vfs.h`

## 4.112 vfs\_fsinfo\_t Struct Reference

Collaboration diagram for vfs\_fsinfo\_t:



## Public Member Functions

- **vec\_struct** (void \*) filelist

## Data Fields

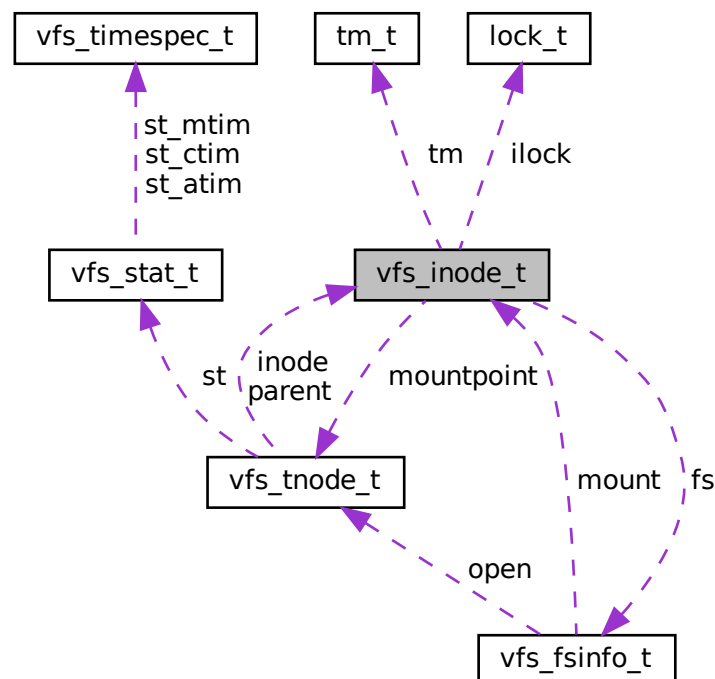
- char **name** [16]
- bool **istemp**
- [vfs\\_inode\\_t](#) \*(\* **mount** )(vfs\_inode\_t \*device)
- [vfs\\_tnode\\_t](#) \*(\* **open** )(vfs\_inode\_t \*this, const char \*path)
- int64\_t(\* **mknode** )(vfs\_tnode\_t \*this)
- int64\_t(\* **rmnode** )(vfs\_tnode\_t \*this)
- int64\_t(\* **read** )(vfs\_inode\_t \*this, uint64\_t offset, uint64\_t len, void \*buff)
- int64\_t(\* **write** )(vfs\_inode\_t \*this, uint64\_t offset, uint64\_t len, const void \*buff)
- int64\_t(\* **sync** )(vfs\_inode\_t \*this)
- int64\_t(\* **refresh** )(vfs\_inode\_t \*this)
- int64\_t(\* **getdent** )(vfs\_inode\_t \*this, uint64\_t pos, [vfs\\_dirent\\_t](#) \*dirent)
- int64\_t(\* **ioctl** )(vfs\_inode\_t \*this, int64\_t request, int64\_t arg)

The documentation for this struct was generated from the following file:

- kernel/fs/[vfs.h](#)

## 4.113 vfs\_inode\_t Struct Reference

Collaboration diagram for `vfs_inode_t`:





## Public Member Functions

- `vec_struct` (`vfs_tnode_t *`) `child`

## Data Fields

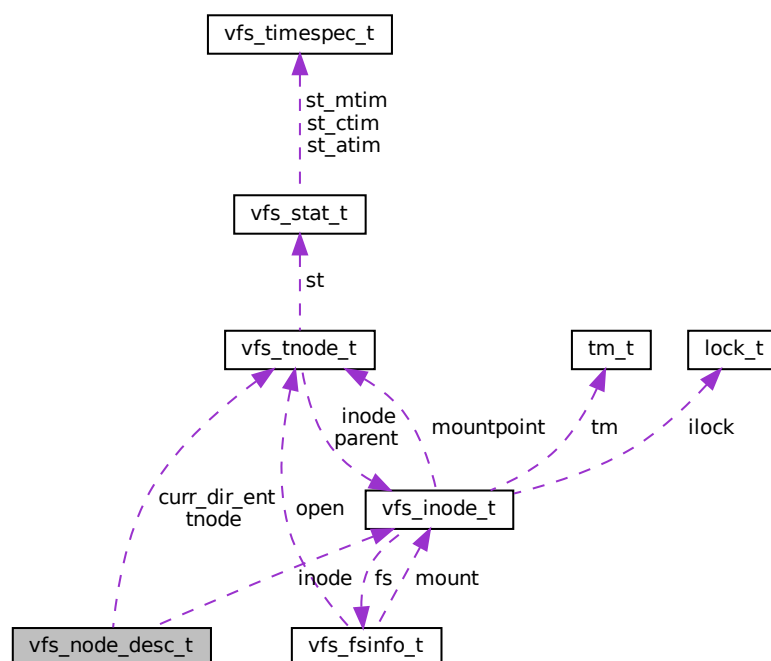
- `vfs_node_type_t` `type`
- `char` `link` [`VFS_MAX_NAME_LEN`]
- `uint64_t` `size`
- `uint32_t` `perms`
- `uint32_t` `uid`
- `uint32_t` `refcount`
- `tm_t` `tm`
- `vfs_fsinfo_t *` `fs`
- `void *` `ident`
- `lock_t` `ilock`
- `vfs_tnode_t *` `mountpoint`

The documentation for this struct was generated from the following file:

- `kernel/fs/vfs.h`

## 4.114 vfs\_node\_desc\_t Struct Reference

Collaboration diagram for `vfs_node_desc_t`:



## Data Fields

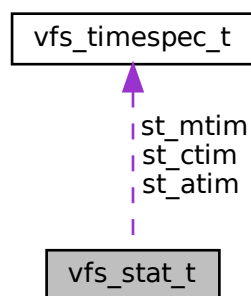
- char **path** [VFS\_MAX\_PATH\_LEN]
- [vfs\\_tnode\\_t](#) \* **tnode**
- [vfs\\_inode\\_t](#) \* **inode**
- [vfs\\_openmode\\_t](#) **mode**
- [uint64\\_t](#) **seek\_pos**
- [vfs\\_tnode\\_t](#) \* **curr\_dir\_ent**
- [uint64\\_t](#) **curr\_dir\_idx**

The documentation for this struct was generated from the following file:

- [kernel/fs/vfs.h](#)

## 4.115 [vfs\\_stat\\_t](#) Struct Reference

Collaboration diagram for [vfs\\_stat\\_t](#):



## Data Fields

- [dev\\_t](#) **st\_dev**
- [ino\\_t](#) **st\_ino**
- [mode\\_t](#) **st\_mode**
- [nlink\\_t](#) **st\_nlink**
- [uid\\_t](#) **st\_uid**
- [gid\\_t](#) **st\_gid**
- [dev\\_t](#) **st\_rdev**
- [off\\_t](#) **st\_size**
- [vfs\\_timespec\\_t](#) **st\_atim**
- [vfs\\_timespec\\_t](#) **st\_mtim**
- [vfs\\_timespec\\_t](#) **st\_ctim**
- [blksize\\_t](#) **st\_blksize**
- [blkcnt\\_t](#) **st\_blocks**

The documentation for this struct was generated from the following file:

- [kernel/fs/vfs.h](#)

## 4.116 vfs\_timespec\_t Struct Reference

### Data Fields

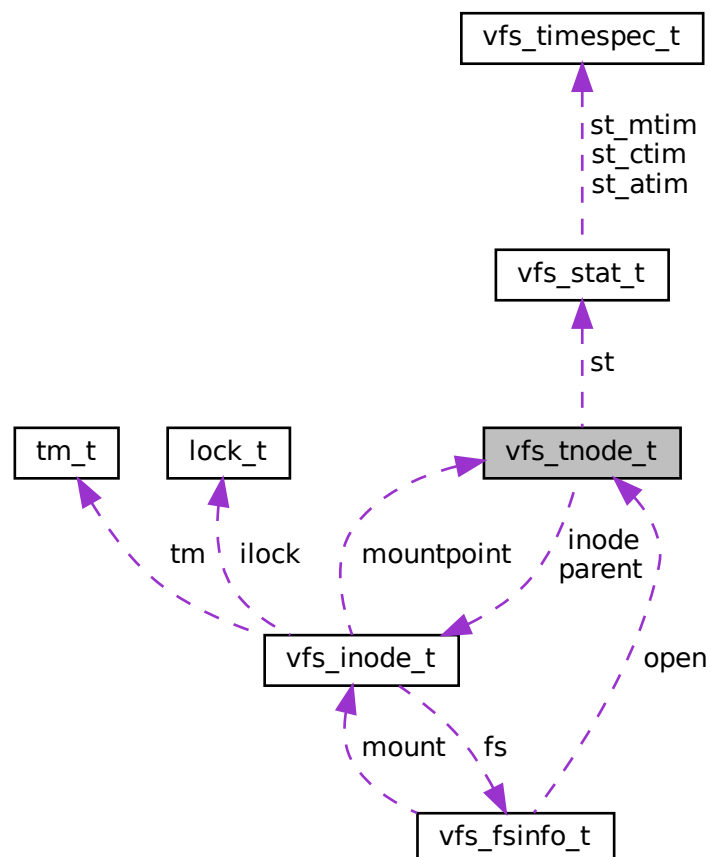
- int64\_t **tv\_sec**
- int64\_t **tv\_nsec**

The documentation for this struct was generated from the following file:

- [kernel/fs/vfs.h](#)

## 4.117 vfs\_tnode\_t Struct Reference

Collaboration diagram for vfs\_tnode\_t:



### Data Fields

- char **name** [VFS\_MAX\_NAME\_LEN]
- [vfs\\_stat\\_t](#) **st**
- [vfs\\_inode\\_t](#) \* **inode**
- [vfs\\_inode\\_t](#) \* **parent**

The documentation for this struct was generated from the following file:

- kernel/fs/[vfs.h](#)

## 4.118 winsize\_t Struct Reference

### Data Fields

- uint16\_t **row**
- uint16\_t **col**
- uint16\_t **xpixel**
- uint16\_t **ypixel**

The documentation for this struct was generated from the following file:

- kernel/device/display/[term.h](#)

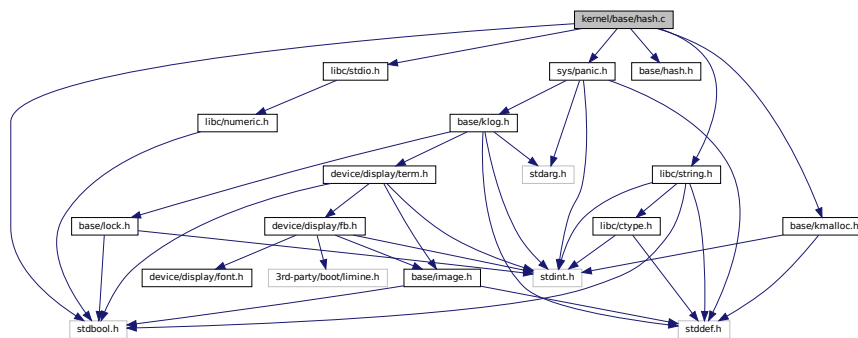
## Chapter 5

# File Documentation

### 5.1 kernel/base/hash.c File Reference

Implementation of a simple hash table.

```
#include <stdbool.h>
#include <libc/stdio.h>
#include <libc/string.h>
#include <base/hash.h>
#include <base/kmalloc.h>
#include <sys/panic.h>
Include dependency graph for hash.c:
```

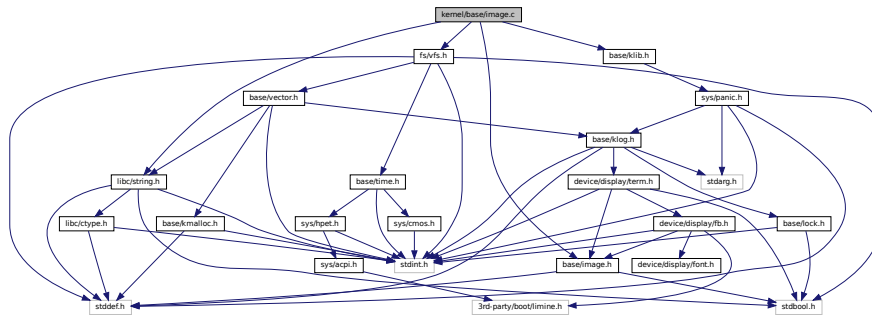


### Functions

- void **ht\_init** (`ht_t` \*ht, uint64\_t size)
- void \* **ht\_search** (`ht_t` \*ht, int64\_t key)
- bool **ht\_insert\_core** (`ht_t` \*ht, int64\_t key, void \*data, bool allow\_realloc)
- bool **ht\_insert** (`ht_t` \*ht, int64\_t key, void \*data)
- void \* **ht\_delete** (`ht_t` \*ht, int64\_t key)



```
#include <base/image.h>
Include dependency graph for image.c:
```



## Data Structures

- struct [bmp\\_header\\_t](#)

## Functions

- bool [bmp\\_load\\_from\\_file](#) ([image\\_t](#) \*image, char \*fn)

### 5.3.1 Detailed Description

Implementation of image handling functions.

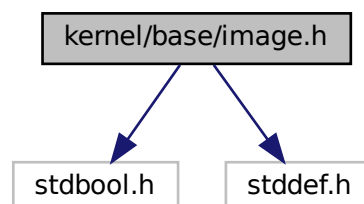
This file contains the implementation of functions for handling images within the HanOS kernel. It includes the function for loading BMP images from a file into an image structure. The BMP loading function reads the file, parses the BMP header, and extracts the image data for use within the kernel.

## 5.4 kernel/base/image.h File Reference

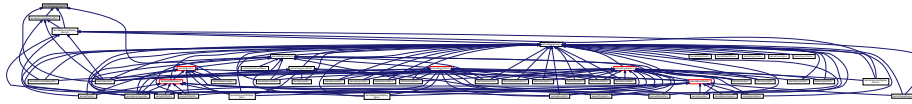
Image handling definitions.  

```
#include <stdbool.h>
#include <stddef.h>
```

 Include dependency graph for image.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [image\\_t](#)

## Functions

- bool [bmp\\_load\\_from\\_file](#) ([image\\_t](#) \*image, char \*fn)

### 5.4.1 Detailed Description

Image handling definitions.

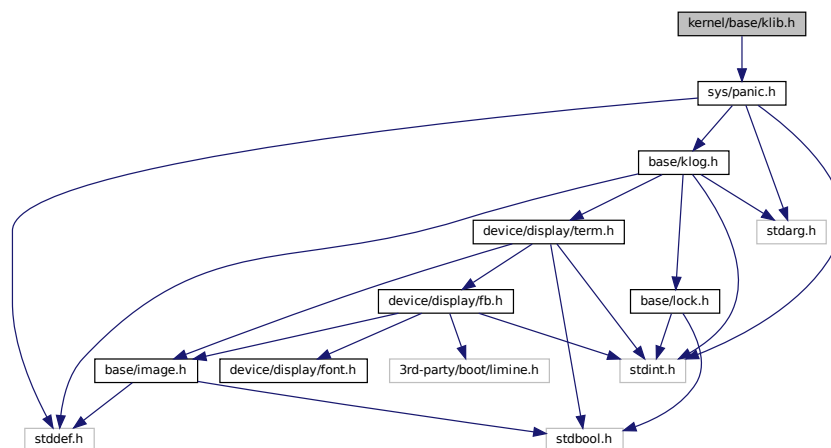
This file contains the type definitions and function declarations for image handling within the HanOS kernel. It includes the structure definition for representing an image and a function prototype for loading a BMP image from a file.

## 5.5 kernel/base/klib.h File Reference

Definition of fundamental data structures, macros and functions.

```
#include <sys/panic.h>
```

Include dependency graph for klib.h:



This graph shows which files directly or indirectly include this file:





## Macros

- #define **MIN**(x, y) ((x) < (y) ? (x) : (y))
- #define **MAX**(x, y) ((x) > (y) ? (x) : (y))
- #define **ROUND\_DOWN**(v, n) ((v) - ((v) % (n)))
- #define **ROUND\_UP**(v, n) ROUND\_DOWN((v) + (n) - 1, n)
- #define **DIV\_ROUNDUP**(a, b) (((a) + ((b) - 1)) / (b))
- #define **ALIGNUP**(x, a) (DIV\_ROUNDUP(x, a) \* a)
- #define **ASSERT**(x)

## Functions

- void **klog\_lock** (void)
- void **klog\_unlock** (void)
- void **klog\_debug** (void)
- void **klog\_refresh** (int mode)
- void **klog\_init** ()
- void **klog\_vprintf\_core** ([klog\\_info\\_t](#) \*k, const char \*s, va\_list args)
- void **klog\_vprintf\_wrapper** ([klog\\_info\\_t](#) \*k, const char \*s,...)
- void **klog\_vprintf** ([klog\\_level\\_t](#) level, const char \*s,...)
- void **kprintf** (const char \*s,...)

### 5.6.1 Detailed Description

Implementation of kernel log related functions.

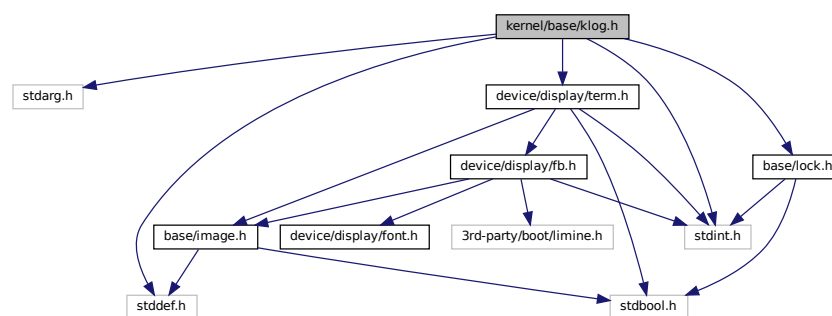
This file provides the implementation of the kernel-level log system for the HanOS kernel. It supports logging messages at various levels (verbose, debug, info, warn, error), handling log buffers, and displaying log output on the terminal.

## 5.7 kernel/base/klog.h File Reference

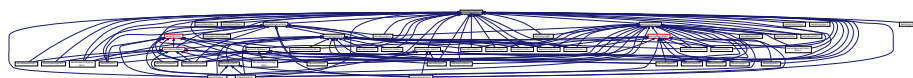
Definition of kernel log related functions.

```
#include <stdarg.h>
#include <stddef.h>
#include <stdint.h>
#include <device/display/term.h>
#include <base/lock.h>
```

Include dependency graph for klog.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [klog\\_info\\_t](#)

## Macros

- `#define KLOG_BUFFER_SIZE (4096 * 4)`
- `#define klogv(s, ...) klog_vprintf(KLOG_LEVEL_VERBOSE, s, ##__VA_ARGS__)`
- `#define klogd(s, ...) klog_vprintf(KLOG_LEVEL_DEBUG, s, ##__VA_ARGS__)`
- `#define klogi(s, ...) klog_vprintf(KLOG_LEVEL_INFO, s, ##__VA_ARGS__)`
- `#define klogw(s, ...) klog_vprintf(KLOG_LEVEL_WARN, s, ##__VA_ARGS__)`
- `#define kloge(s, ...) klog_vprintf(KLOG_LEVEL_ERROR, s, ##__VA_ARGS__)`
- `#define klogu(s, ...) klog_vprintf(KLOG_LEVEL_UNK, s, ##__VA_ARGS__)`
- `#define klog_printf(s, ...) klog_vprintf(KLOG_LEVEL_INFO, s, ##__VA_ARGS__)`

## Enumerations

- `enum klog_level_t {  
KLOG_LEVEL_VERBOSE, KLOG_LEVEL_DEBUG, KLOG_LEVEL_INFO, KLOG_LEVEL_WARN,  
KLOG_LEVEL_ERROR, KLOG_LEVEL_UNK }`

## Functions

- `void klog_init (void)`
- `void klog_vprintf (klog_level_t level, const char *,...)`
- `void klog_refresh (int mode)`
- `void klog_debug (void)`
- `void klog_lock (void)`
- `void klog_unlock (void)`
- `void kprintf (const char *,...)`

### 5.7.1 Detailed Description

Definition of kernel log related functions.

---

This header file provides the definitions for kernel log functions used in the HanOS kernel. It includes declarations for initializing the log system, printing log messages at various levels, and managing log buffers.

---

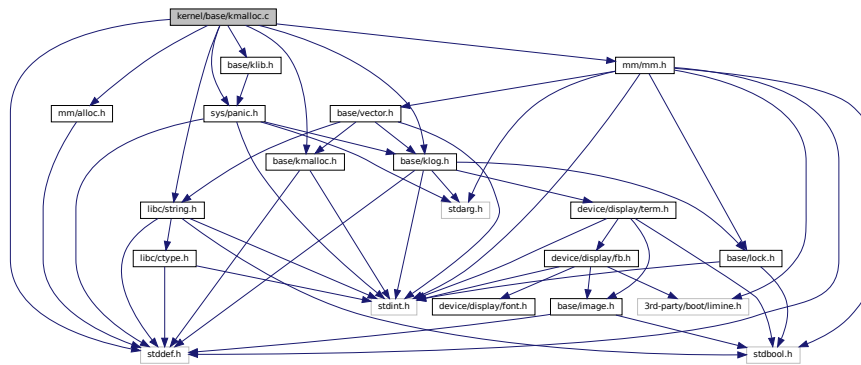
## 5.8 kernel/base/kmalloc.c File Reference

Implementation of kernel memory allocation functions.

```
#include <stddef.h>
#include <libc/string.h>
#include <base/kmalloc.h>
#include <base/klog.h>
#include <base/klib.h>
#include <sys/panic.h>
#include <mm/mm.h>
#include <mm/alloc.h>
```

---

Include dependency graph for `kmalloc.c`:



## Macros

- `#define SLAB_ALLOCATOR_USED true`

## Functions

- `void * kmalloc_core (uint64_t size, const char *func, uint64_t line)`
- `void * kmalloc_chunk (uint64_t size, const char *func, uint64_t line)`
- `void kmfree_core (void *addr, const char *func, uint64_t line)`
- `void kmfree_chunk (void *addr, const char *func, uint64_t line)`
- `void * kmrealloc_core (void *addr, uint64_t newsize, const char *func, uint64_t line)`
- `void * kmrealloc_chunk (void *addr, uint64_t newsize, const char *func, uint64_t line)`

## Variables

- `uint64_t kmalloc_checkno = 0`

### 5.8.1 Detailed Description

Implementation of kernel memory allocation functions.

Kernel memory allocation function includes `malloc`, `free` and `realloc`.

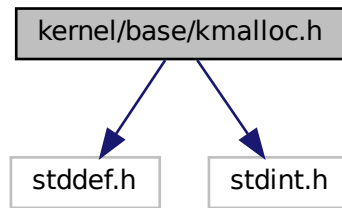
**Todo** Memory allocation should be improved for better efficiency.

## 5.9 kernel/base/kmalloc.h File Reference

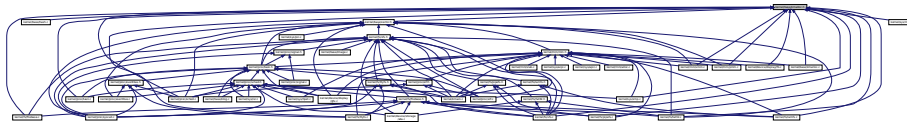
Definition of memory allocation related functions.

```
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for kmalloc.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [memory\\_metadata\\_t](#)

## Macros

- #define **MEM\_MAGIC\_NUM** 0xABEEABEE
- #define **kmalloc**(x) kmalloc\_core(x, \_\_func\_\_, \_\_LINE\_\_)
- #define **kmfree**(x) kmfree\_core(x, \_\_func\_\_, \_\_LINE\_\_)
- #define **kmrealloc**(x, y) kmrealloc\_core(x, y, \_\_func\_\_, \_\_LINE\_\_)

## Functions

- void \* **kmalloc\_core** (uint64\_t size, const char \*func, uint64\_t line)
- void **kmfree\_core** (void \*addr, const char \*func, uint64\_t line)
- void \* **kmrealloc\_core** (void \*addr, uint64\_t newsize, const char \*func, uint64\_t line)
- void \* **kmalloc\_chunk** (uint64\_t size, const char \*func, uint64\_t line)
- void **kmfree\_chunk** (void \*addr, const char \*func, uint64\_t line)
- void \* **kmrealloc\_chunk** (void \*addr, uint64\_t newsize, const char \*func, uint64\_t line)

## Variables

- uint64\_t **kmalloc\_checkno**

### 5.9.1 Detailed Description

Definition of memory allocation related functions.

e.g., malloc, free and realloc.

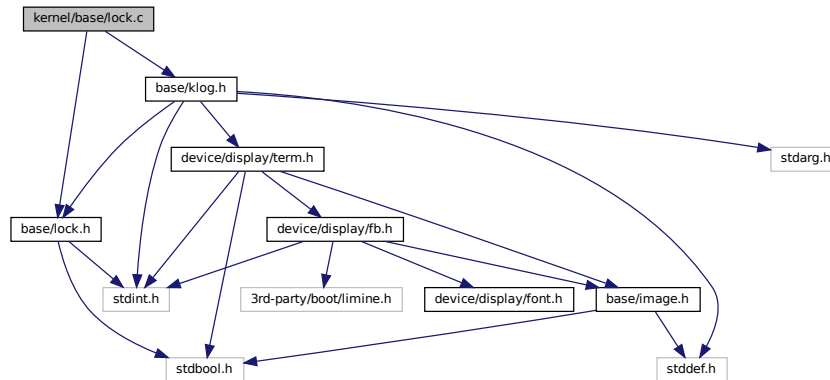
## 5.10 kernel/base/lock.c File Reference

Implementation of spin lock related functions.

```
#include <base/lock.h>
```

```
#include <base/klog.h>
```

Include dependency graph for lock.c:



### Functions

- void **lock\_lock\_impl** ([lock\\_t](#) \*s, const char \*fn, const int ln)
- void **lock\_release\_impl** ([lock\\_t](#) \*s, const char \*fn, const int ln)

### 5.10.1 Detailed Description

Implementation of spin lock related functions.

There are a few atomic operations on the x86 processor that set and compare memory or registers, that can be used as the basis of a spin lock. See below implementation for details.

Note that when any task acquires spin lock, the system interrupts will be disabled. When this task releases the corresponding spin lock, interrupts will be enabled. So if some tasks acquire spin lock twice, it will block task scheduler and system will fall into DEAD busy loop.

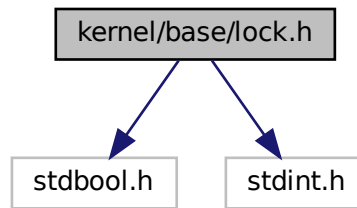
## 5.11 kernel/base/lock.h File Reference

Definition of spin lock related data structures and functions.

```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Include dependency graph for lock.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [lock\\_t](#)

## Macros

- `#define lock_new() (lock\_t){0, 0}`
- `#define lock_lock(x) lock_lock_impl(x, __FILE__, __LINE__)`
- `#define lock_release(x) lock_release_impl(x, __FILE__, __LINE__)`

## Functions

- void **lock\_lock\_impl** ([lock\\_t](#) \*s, const char \*fn, const int ln)
- void **lock\_release\_impl** ([lock\\_t](#) \*s, const char \*fn, const int ln)

### 5.11.1 Detailed Description

Definition of spin lock related data structures and functions.

Busy waiting is a technique in which a process repeatedly checks to see if a condition is true (from Wikipedia).

Spinlock uses the above technique for the purpose of checking if a lock is available.

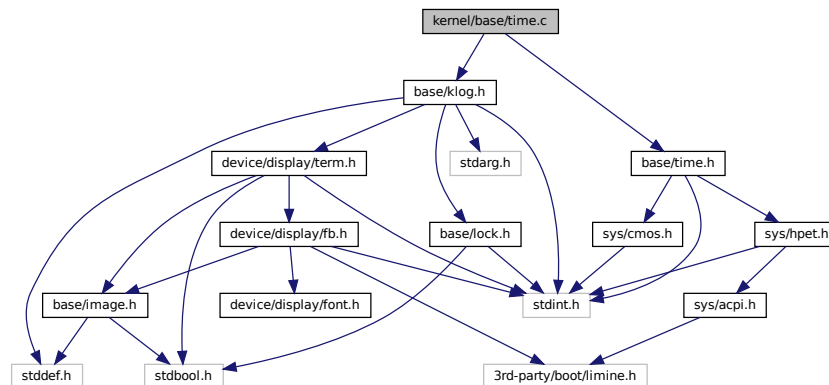
Three functions are implemented here: `init (new)`, `acquire (lock)` and `release`.

## 5.12 kernel/base/time.c File Reference

Implementation of time related functions.

```
#include <base/klog.h>
#include <base/time.h>
```

Include dependency graph for time.c:



## Functions

- void **localtime** (const time\_t \*timep, [tm\\_t](#) \*\_timevalue)
- time\_t **mktime** ([tm\\_t](#) \*tm)

### 5.12.1 Detailed Description

Implementation of time related functions.

---

This file provides the implementation of various time-related functions used in the HanOS kernel. It includes functions to determine if a year is a leap year, calculate the day of the week, determine the number of days in a month, convert time values to a structured format, and convert structured time back to a time value.

---

## 5.13 kernel/base/time.h File Reference

Definition of time related data structures and functions.

```

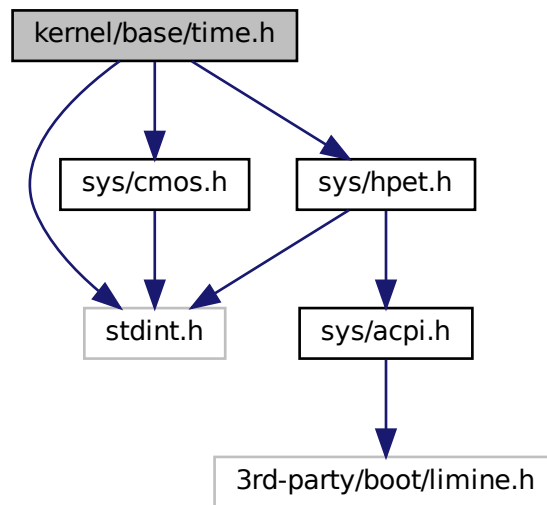
#include <stdint.h>
#include <sys/hpet.h>
#include <sys/cmos.h>

```

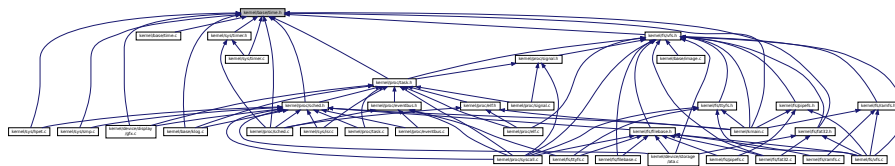
---



Include dependency graph for time.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [timezone\\_t](#)
- struct [tm\\_t](#)

## Macros

- `#define hpet_sleep(x) hpet_nanosleep(MILLIS_TO_NANOS(x))`
- `#define SECONDS_TO_NANOS(x) ((x)*1000000000ULL)`
- `#define MILLIS_TO_NANOS(x) ((x)*1000000ULL)`
- `#define MICROS_TO_NANOS(x) ((x)*1000ULL)`
- `#define NANOS_TO_SECONDS(x) ((x) / 1000000000ULL)`
- `#define NANOS_TO_MILLIS(x) ((x) / 1000000ULL)`
- `#define NANOS_TO_MICROS(x) ((x) / 1000ULL)`

## Typedefs

- typedef uint64\_t **time\_t**

## Functions

- void **localtime** (const time\_t \*timep, [tm\\_t](#) \*tm)

### 5.13.1 Detailed Description

#### Definition of time related data structures and functions.

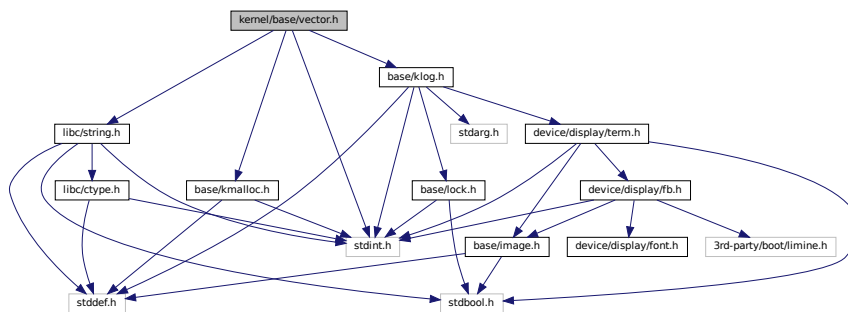
This header file provides the definitions for time-related data structures and functions used in the HanOS kernel. It includes macros for time conversions, type definitions for representing time and time zones, and a function prototype for converting time values to a structured format.

## 5.14 kernel/base/vector.h File Reference

vector - fundamental data structure, variable length array

```
#include <stdint.h>
#include <libc/string.h>
#include <base/klog.h>
#include <base/kmalloc.h>
```

Include dependency graph for vector.h:



This graph shows which files directly or indirectly include this file:



### Macros

- #define **VECTOR\_RESIZE\_FACTOR** 4
- #define **vec\_struct**(type)
- #define **vec\_extern**(type, name) extern vec\_struct(type) name
- #define **vec\_new**(type, name) vec\_struct(type) name = {0}
- #define **vec\_new\_static**(type, name) static vec\_new(type, name)
- #define **vec\_push\_back**(vec, elem)
- #define **vec\_length**(vec) (vec)->len
- #define **vec\_at**(vec, index) (vec)->data[index]
- #define **vec\_erase**(vec, index)
- #define **vec\_erase\_all**(vec)
- #define **vec\_erase\_val**(vec, val)

### 5.14.1 Detailed Description

vector - fundamental data structure, variable length array

Implementation of variable length array. All the functions are defined in macros.

**Attention**

If memories are randomly initialized, random "data" will cause #PF.  
Pls make sure memories are initially set to zero.

---

**5.14.2 Macro Definition Documentation****5.14.2.1 vec\_erase**

```
#define vec_erase(
 vec,
 index)
```

**Value:**

```
{
 memcpy(&((vec)->data[index]), &((vec)->data[index + 1]),
 sizeof((vec)->data[0]) * (vec)->len - index - 1);
 memset(&((vec)->data[(vec)->len - 1]), 0,
 sizeof((vec)->data[0]));
 (vec)->len--;
 if ((vec)->len == 0) {
 (vec)->capacity = 0;
 if ((vec)->data != NULL) kmfree((vec)->data);
 (vec)->data = NULL;
 }
}
```

**5.14.2.2 vec\_erase\_all**

```
#define vec_erase_all(
 vec)
```

**Value:**

```
{
 if ((vec)->len > 0)
 memset((vec)->data, 0,
 sizeof((vec)->data[0]) * (vec)->len);
 (vec)->len = 0;
 (vec)->capacity = 0;
 if ((vec)->data != NULL) kmfree((vec)->data);
 (vec)->data = NULL;
}
```

**5.14.2.3 vec\_erase\_val**

```
#define vec_erase_val(
 vec,
 val)
```

**Value:**

```
{
 for(int64_t __i = 0; __i < (vec)->len; __i++) {
 if (vec_at(vec, __i) == (val)) {
 vec_erase(vec, __i);
 break;
 }
 }
}
```

**5.14.2.4 vec\_push\_back**

```
#define vec_push_back(
 vec,
 elem)
```

**Value:**

```
{
 (vec)->len++;
 if ((vec)->capacity < (vec)->len * sizeof(elem)) {
```

```

 (vec)->capacity = (vec)->len * sizeof(elem)
 * VECTOR_RESIZE_FACTOR;
 (vec)->data = kmrealloc((vec)->data, (vec)->capacity);
 }
 (vec)->data[(vec)->len - 1] = elem;
}

```

#### 5.14.2.5 vec\_struct

```

#define vec_struct(
 type)

```

Value:

```

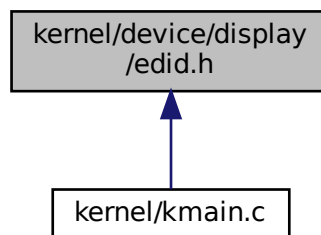
 struct {
 uint64_t len;
 uint64_t capacity;
 type* data;
 }

```

## 5.15 kernel/device/display/edid.h File Reference

EDID (Extended Display Identification Data) definitions.

This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [edid\\_info\\_t](#)

#### 5.15.1 Detailed Description

EDID (Extended Display Identification Data) definitions.

This file contains the type definitions related to EDID (Extended Display Identification Data) used within the HanOS kernel. The EDID data structure provides detailed information about a display device's capabilities. This includes the manufacturer ID, serial number, week and year of manufacture, EDID version and revision, video input type, display size, gamma factor, power management features, chromaticity coordinates, established timings, standard timings, and detailed timings.

## 5.16 kernel/device/display/fb.c File Reference

Implementation of framebuffer related functions.

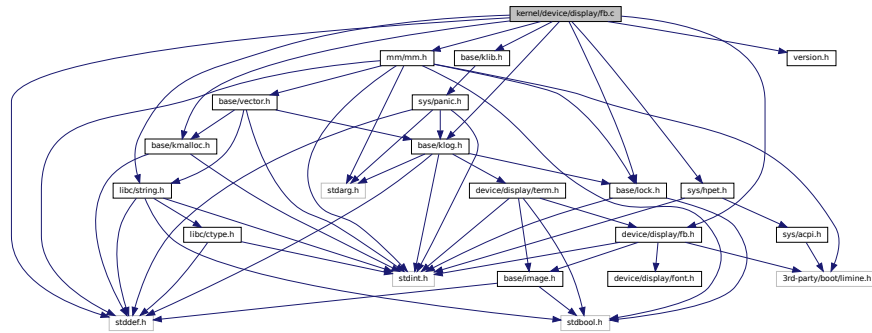
```

#include <stddef.h>
#include <version.h>
#include <libc/string.h>

```

```
#include <mm/mm.h>
#include <sys/hpet.h>
#include <device/display/fb.h>
#include <base/kmalloc.h>
#include <base/lock.h>
#include <base/klog.h>
#include <base/klib.h>
```

Include dependency graph for fb.c:



## Macros

- `#define LOGO_SCALE 6`

## Functions

- `bool fb_set_bg_image (fb_info_t *fb, image_t *img)`
- `void fb_putch (fb_info_t *fb, uint32_t x, uint32_t y, uint32_t fgcolor, uint32_t bgcolor, uint8_t ch, bool bold)`
- `void fb_putlogo (fb_info_t *fb, uint32_t fgcolor, uint32_t bgcolor)`
- `void fb_putpixel (fb_info_t *fb, uint32_t x, uint32_t y, uint32_t color)`
- `uint32_t fb_getpixel (fb_info_t *fb, uint32_t x, uint32_t y)`
- `void fb_init (fb_info_t *fb, struct lime_framebuffer *s)`
- `void fb_refresh (fb_info_t *fb)`

## Variables

- `font_psf1_t` `term_font_norm`
- `font_psf1_t` `term_font_bold`

### 5.16.1 Detailed Description

### Implementation of framebuffer related functions.

Graphics can be displayed in a linear framebuffer - a simple array mapped in memory that represents the screen. The address of framebuffer was got from Limine bootloader.

History:

Mar 27, 2022 - Rewrite fb\_refresh() by memcpy() significantly improve the frame rate. In the future, we should rewrite memcpy() by SSE enhancements.

Nov 25, 2022 - Background picture could be set by a bitmap file.

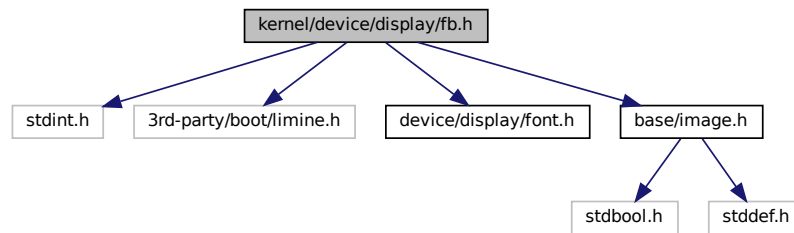
**Todo** Improve memcpy() by SSE enhancements.

## 5.17 kernel/device/display/fb.h File Reference

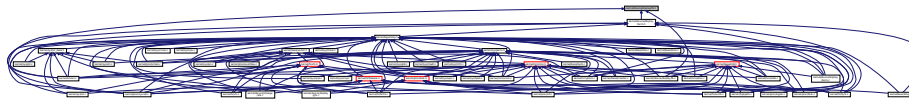
Definition of framebuffer related functions.

```
#include <stdint.h>
#include <3rd-party/boot/limine.h>
#include <device/display/font.h>
#include <base/image.h>
```

Include dependency graph for fb.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [fb\\_info\\_t](#)

### Macros

- `#define FB_WIDTH 2048`
- `#define FB_HEIGHT 1536`
- `#define FB_PITCH (FB_WIDTH * 4)`
- `#define COLOR_BLACK 0x000000`
- `#define COLOR_RED 0xAA0000`
- `#define COLOR_GREEN 0x00AA00`
- `#define COLOR_YELLOW 0xAAAA00`
- `#define COLOR_BROWN 0xAA5500`
- `#define COLOR_BLUE 0x0000AA`
- `#define COLOR_MAGENTA 0xAA00AA`
- `#define COLOR_CYAN 0x00AAAA`
- `#define COLOR_GREY 0xAAAAAA`
- `#define DEFAULT_FGCOLOR COLOR_GREY`
- `#define DEFAULT_BGCOLOR COLOR_BLACK`

### Functions

- void `fb_init` ([fb\\_info\\_t](#) \*fb, struct `limine_framebuffer` \*s)
- void `fb_putpixel` ([fb\\_info\\_t](#) \*fb, uint32\_t x, uint32\_t y, uint32\_t color)
- void `fb_putch` ([fb\\_info\\_t](#) \*fb, uint32\_t x, uint32\_t y, uint32\_t fgcolor, uint32\_t bgcolor, uint8\_t ch, bool bold)
- void `fb_putlogo` ([fb\\_info\\_t](#) \*fb, uint32\_t fgcolor, uint32\_t bgcolor)

- uint32\_t **fb\_getpixel** ([fb\\_info\\_t](#) \*fb, uint32\_t x, uint32\_t y)
- void **fb\_refresh** ([fb\\_info\\_t](#) \*fb)
- bool **fb\_set\_bg\_image** ([fb\\_info\\_t](#) \*fb, [image\\_t](#) \*img)

### 5.17.1 Detailed Description

Definition of framebuffer related functions.

---

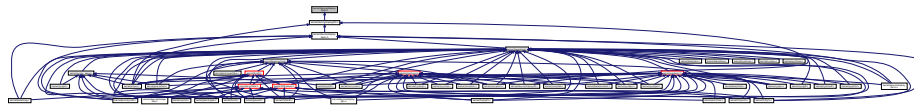
Graphics can be displayed in a linear framebuffer - a simple array mapped in memory that represents the screen. The address of framebuffer was got from Limine bootloader.

---

## 5.18 kernel/device/display/font.h File Reference

Definition of font data matrix.

This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [font\\_psf1\\_t](#)
- struct [font\\_psf2\\_t](#)

### Macros

- #define **PSF1\_MAGIC0** 0x36
- #define **PSF1\_MAGIC1** 0x04
- #define **PSF1\_MODE512** 0x01
- #define **PSF1\_MODEHASTAB** 0x02
- #define **PSF1\_MODEHASSEQ** 0x04
- #define **PSF1\_MAXMODE** 0x05
- #define **PSF1\_SEPARATOR** 0xFFFF
- #define **PSF1\_STARTSEQ** 0xFFFE
- #define **PSF2\_MAGIC0** 0x72
- #define **PSF2\_MAGIC1** 0xb5
- #define **PSF2\_MAGIC2** 0x4a
- #define **PSF2\_MAGIC3** 0x86
- #define **PSF2\_HAS\_UNICODE\_TABLE** 0x01
- #define **PSF2\_MAXVERSION** 0
- #define **PSF2\_SEPARATOR** 0xFF
- #define **PSF2\_STARTSEQ** 0xFE

### 5.18.1 Detailed Description

Definition of font data matrix.

---

It contains font data by static defined matrix. The ASC16 and HZK16 were included.

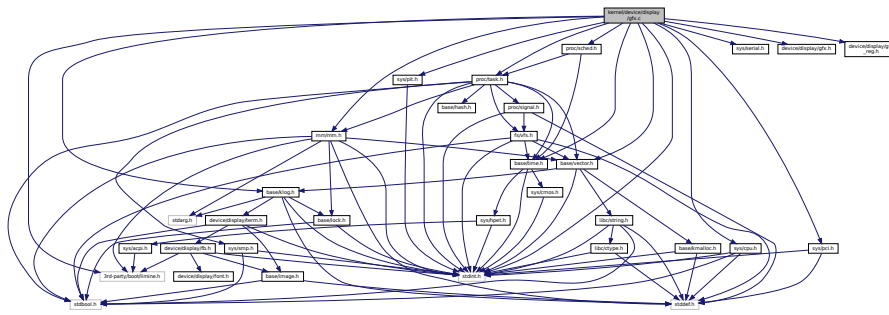
---

## 5.19 kernel/device/display/gfx.c File Reference

Implementation of graphics device handling.

```
#include <3rd-party/boot/limine.h>
#include <stdint.h>
#include <sys/cpu.h>
#include <sys/pci.h>
#include <sys/pit.h>
#include <sys/serial.h>
#include <mm/mm.h>
#include <base/vector.h>
#include <base/klog.h>
#include <base/time.h>
#include <proc/task.h>
#include <proc/sched.h>
#include <device/display/gfx.h>
#include <device/display/gfx_reg.h>
```

Include dependency graph for gfx.c:



### Macros

- `#define DEVICE_HD5500 0x1616`
- `#define DEVICE_HD520 0x1916`
- `#define DEVICE_SUNRISE_PANTHERPOINT 0x9D00`

### Functions

- `vec_extern (pci_device_t, pci_devices)`
- `bool gfx_validate_chipset (void)`
- `void gfx_init_pci (gfx_pci_t *pci, pci_device_t dev)`
- `void gfx_init_gtt (gfx_pci_t *pci, gfx_gtt_t *gtt, pci_device_t dev)`
- `void gfx_init_mem_manager (gfx_pci_t *pci, gfx_gtt_t *gtt, gfx_mem_manager_t *mgr)`
- `void gfx_enter_force_wake (gfx_pci_t *pci)`
- `void gfx_exit_force_wake (gfx_pci_t *pci)`
- `bool pci_get_gfx_device (pci_device_t *gfx_dev)`
- `bool gfx_init (void)`
- `void gfx_mem_enable_swizzle (gfx_pci_t *pci)`
- `uint64_t gfx_addr (gfx_mem_manager_t *mgr, void *phy_addr)`
- `bool gfx_alloc (gfx_mem_manager_t *mgr, gfx_object_t *obj, uint64_t size, uint64_t align)`
- `void gfx_start (void)`

#### 5.19.1 Detailed Description

Implementation of graphics device handling.

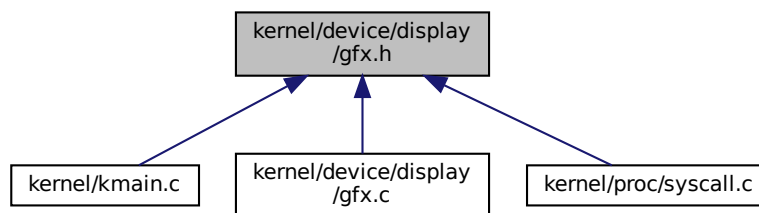


This file contains the implementation of functions for handling graphics devices within the HanOS kernel. It includes functions for initializing PCI graphics devices, configuring graphics translation tables (GTT), managing graphics memory, and enabling specific graphics features. It also includes functions for entering and exiting force wake states and for allocating graphics memory objects.

## 5.20 kernel/device/display/gfx.h File Reference

Graphics device handling definitions.

This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [gfx\\_pci\\_t](#)
- struct [gfx\\_gtt\\_t](#)
- struct [gfx\\_object\\_t](#)
- struct [gfx\\_mem\\_range\\_t](#)
- struct [gfx\\_mem\\_manager\\_t](#)

### Macros

- `#define FENCE_BASE 0x100000`
- `#define FENCE_COUNT 16`
- `#define gfx_ind(pci, reg) mmio_ind((uint8_t*)pci->mmio_bar + reg)`
- `#define gfx_outd(pci, reg, val) mmio_outd((uint8_t*)pci->mmio_bar + reg, val)`
- `#define gfx_inl(pci, reg) mmio_inl((uint8_t*)pci->mmio_bar + reg)`
- `#define gfx_outl(pci, reg, val) mmio_outl((uint8_t*)pci->mmio_bar + reg, val)`

### Functions

- bool `pci_get_gfx_device` ([pci\\_device\\_t](#) \*gfx\_dev)
- bool `gfx_init` (void)
- void `gfx_start` (void)

#### 5.20.1 Detailed Description

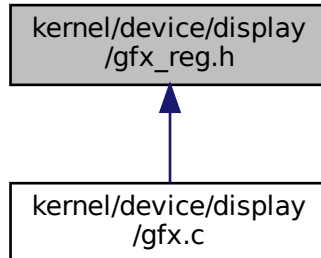
Graphics device handling definitions.

This file contains the type definitions and function declarations for handling graphics devices within the HanOS kernel. It includes structures for representing PCI graphics devices, graphics translation tables (GTT), graphics objects, and memory ranges. Additionally, it provides function prototypes for initializing and starting the graphics system, as well as macros for interacting with memory-mapped I/O registers of the graphics device.

## 5.21 kernel/device/display/gfx\_reg.h File Reference

Graphics device register definitions.

This graph shows which files directly or indirectly include this file:



### Macros

- #define **MASKED\_ENABLE**(x) (((x) << 16) | (x))
- #define **MASKED\_DISABLE**(x) ((x) << 16)
- #define **GTT\_PAGE\_SHIFT** 12
- #define **GTT\_PAGE\_SIZE** (1 << GTT\_PAGE\_SHIFT)
- #define **GTT\_ENTRY\_VALID** (1 << 0)
- #define **GTT\_ENTRY\_L3\_CACHE\_CONTROL** (1 << 1)
- #define **GTT\_ENTRY\_LLC\_CACHE\_CONTROL** (1 << 2)
- #define **GTT\_ENTRY\_GFX\_DATA\_TYPE** (1 << 3)
- #define **GTT\_ENTRY\_ADDR**(x) ((x) | ((x >> 28) & 0xff0))
- #define **GFX\_MCHBAR** 0x140000
- #define **ARB\_MODE** 0x04030 /\* R/W \*/
- #define **ARB\_MODE\_GGTAGDR** (1 << 0) /\* GTT Accesses GDR \*/
- #define **ARB\_MODE\_CCGDREN** (1 << 1) /\* Color Cache GDR Enable Bit \*/
- #define **ARB\_MODE\_DCGDREN** (1 << 2) /\* Depth Cache GDR Enable Bit \*/
- #define **ARB\_MODE\_TCGDREN** (1 << 3) /\* Texture Cache GDR Enable Bit \*/
- #define **ARB\_MODE\_VMC\_GDR\_EN** (1 << 4) /\* VMC GDR Enable \*/
- #define **ARB\_MODE\_AS4TS** (1 << 5) /\* Address Swizzling for Tiled Surfaces \*/
- #define **ARB\_MODE\_CDPs** (1 << 8) /\* Color/Depth Port Share Bit \*/
- #define **ARB\_MODE\_GAMPD\_GDR** (1 << 9) /\* GAM PD GDR \*/
- #define **ARB\_MODE\_BLB\_GDR** (1 << 10) /\* BLB GDR \*/
- #define **ARB\_MODE\_STC\_GDR** (1 << 11) /\* STC GDR \*/
- #define **ARB\_MODE\_HIZ\_GDR** (1 << 12) /\* HIZ GDR \*/
- #define **ARB\_MODE\_DC\_GDR** (1 << 13) /\* DC GDR \*/
- #define **ARB\_MODE\_GAM2BGTTT** (1 << 14) /\* GAM to Bypass GTT Translation \*/
- #define **SR\_INDEX** 0x3c4
- #define **SR\_DATA** 0x3c5
- #define **SEQ\_CLOCKING** 0x01
- #define **SCREEN\_OFF** (1 << 5)
- #define **MGGC0** 0x50 /\* In PCI Config Space \*/
- #define **GGC\_LOCK** (1 << 0)
- #define **GGC\_IVD** (1 << 1) /\* IGD VGA Disable \*/
- #define **GGC\_GMS\_SHIFT** 3 /\* Graphics Mode Select \*/

- `#define GGC_GMS_MASK 0x1f`
- `#define GGC_GGMS_SHIFT 8 /* GTT Graphics Memory Size */`
- `#define GGC_GGMS_MASK 0x3`
- `#define GGC_VAMEN (1 << 14) /* Versatile Acceleration Mode Enable */`
- `#define GMS_32MB 0x05`
- `#define GMS_48MB 0x06`
- `#define GMS_64MB 0x07`
- `#define GMS_128MB 0x08`
- `#define GMS_256MB 0x09`
- `#define GMS_96MB 0x0A`
- `#define GMS_160MB 0x0B`
- `#define GMS_224MB 0x0C`
- `#define GMS_352MB 0x0D`
- `#define GMS_0MB 0x00`
- `#define GMS_32MB_1 0x01`
- `#define GMS_64MB_1 0x02`
- `#define GMS_96MB_1 0x03`
- `#define GMS_128MB_1 0x04`
- `#define GMS_448MB 0x0E`
- `#define GMS_480MB 0x0F`
- `#define GMS_512MB 0x10`
- `#define GGMS_None 0x00`
- `#define GGMS_1MB 0x01`
- `#define GGMS_2MB 0x02`
- `#define BDSM 0x5C /* In PCI Config Space */`
- `#define BDSM_LOCK (1 << 0)`
- `#define BDSM_ADDR_MASK (0xfff << 20)`
- `#define VGA_CONTROL 0x41000 /* R/W */`
- `#define VGA_DISABLE (1 << 31)`
- `#define ARB_CTL 0x45000 /* R/W */`
- `#define ARB_CTL_HP_DATA_REQUEST_LIMIT_MASK 0x7f`
- `#define ARB_CTL_HP_PAGE_BREAK_LIMIT_SHIFT 8`
- `#define ARB_CTL_HP_PAGE_BREAK_LIMIT_MASK 0x1f`
- `#define ARB_CTL_TILED_ADDRESS_SWIZZLING (1 << 13)`
- `#define ARB_CTL_TLB_REQUEST_IN_FLIGHT_LIMIT_SHIFT 16`
- `#define ARB_CTL_TLB_REQUEST_IN_FLIGHT_LIMIT_MASK 0x7`
- `#define ARB_CTL_TLB_REQUEST_LIMIT_SHIFT 20`
- `#define ARB_CTL_TLB_REQUEST_LIMIT_MASK 0x7`
- `#define ARB_CTL_LP_WRITE_REQUEST_LIMIT_SHIFT 24`
- `#define ARB_CTL_LP_WRITE_REQUEST_LIMIT_MASK 0x3`
- `#define ARB_CTL_HP_QUEUE_WATERMARK_SHIFT 26`
- `#define ARB_CTL_HP_QUEUE_WATERMARK_MASK 0x7`
- `#define MAD_DIMM_CH0 0x5004`
- `#define MAD_DIMM_CH1 0x5008`
- `#define MAD_DIMM_A_SIZE_SHIFT 0`
- `#define MAD_DIMM_A_SIZE_MASK 0xff`
- `#define MAD_DIMM_B_SIZE_SHIFT 8`
- `#define MAD_DIMM_B_SIZE_MASK 0xff`
- `#define MAD_DIMM_AB_SIZE_MASK 0xffff`
- `#define MAD_DIMM_A_SELECT (1 << 16)`
- `#define MAD_DIMM_A_DUAL_RANK (1 << 17)`
- `#define MAD_DIMM_B_DUAL_RANK (1 << 18)`
- `#define MAD_DIMM_A_X16 (1 << 19)`
- `#define MAD_DIMM_B_X16 (1 << 20)`
- `#define MAD_DIMM_RANK_INTERLEAVE (1 << 21)`

- #define **MAD\_DIMM\_ENH\_INTERLEAVE** (1 << 22)
- #define **MAD\_DIMM\_ECC\_MODE** (3 << 24)
- #define **ECOBUS** 0xA180
- #define **FORCE\_WAKE\_MT** 0xA188
- #define **FORCE\_WAKE** 0xA18C
- #define **FORCE\_WAKE\_MT\_ACK** 0x130044
- #define **FORCE\_WAKE\_ACK** 0x130090
- #define **TILE\_CTL** 0x101000 /\* R/W \*/
- #define **TILE\_CTL\_SWIZZLE** (1 << 0)
- #define **TILE\_CTL\_TLB\_PREFETCH\_DISABLE** (1 << 2)
- #define **TILE\_CTL\_BACKSNOOP\_DISABLE** (1 << 3)

### 5.21.1 Detailed Description

Graphics device register definitions.

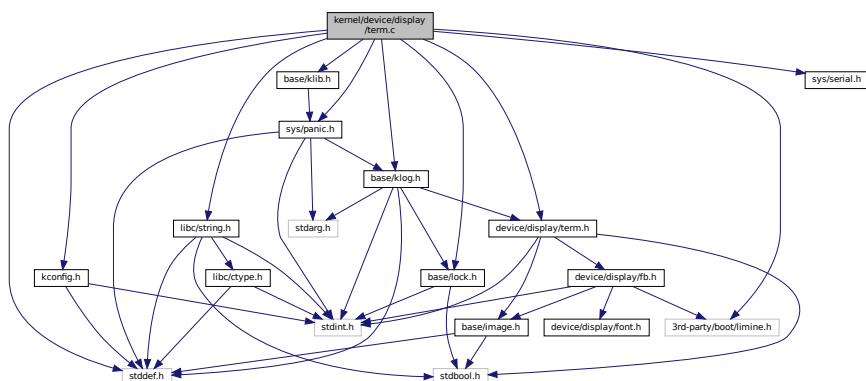
This file contains the definitions of various registers and constants used for handling graphics devices within the HanOS kernel. It includes bit masks and shifts for manipulating register values, as well as addresses for memory mapped I/O registers related to graphics memory management, GTT (Graphics Translation Table), and display control.

## 5.22 kernel/device/display/term.c File Reference

Implementation of framebuffer terminal related functions.

```
#include <stddef.h>
#include <kconfig.h>
#include <libc/string.h>
#include <device/display/term.h>
#include <base/klib.h>
#include <base/klog.h>
#include <base/lock.h>
#include <sys/panic.h>
#include <sys/serial.h>
#include <3rd-party/boot/limine.h>
```

Include dependency graph for term.c:



### Macros

- #define **CHECK\_ACTIVE\_TERM()** { if (term\_act == NULL) kpanic("Active terminal does not exist"); }

## Functions

- void **term\_get\_winsize** ([winsize\\_t](#) \*ws)
- bool **term\_set\_winsize** ([winsize\\_t](#) \*ws)
- bool **term\_set\_bg\_image** ([image\\_t](#) \*img)
- bool **term\_parse\_cmd** ([term\\_info\\_t](#) \*term\_act, uint8\_t byte)
- void **term\_scroll** ([term\\_info\\_t](#) \*term\_act)
- void **term\_set\_cursor** (uint8\_t c)
- int **term\_get\_mode** (void)
- void **term\_refresh** (int mode)
- void **term\_clear** (int mode)
- void **term\_print** (int mode, uint8\_t c)
- void **term\_putch** (int mode, uint8\_t c)
- void **term\_init** (struct [limine\\_framebuffer](#) \*s)
- void **term\_start** ()
- bool **term\_get\_redraw** ()
- void **term\_set\_redraw** (bool val)
- void **term\_switch** (int mode)

## Variables

- [term\\_cursor\\_visible\\_t](#) **cursor\_visible** = [CURSOR\\_INVISIBLE](#)

### 5.22.1 Detailed Description

Implementation of framebuffer terminal related functions.

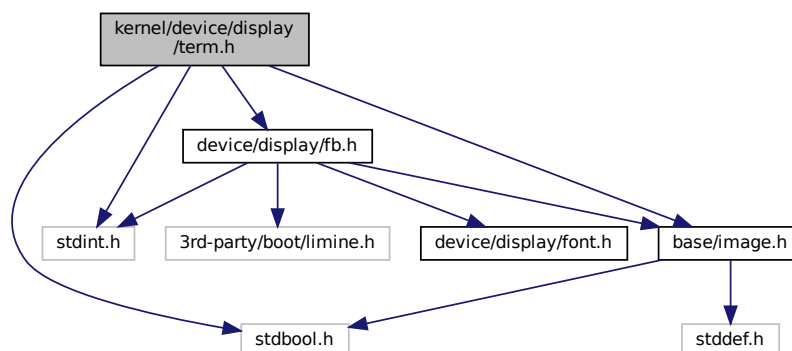
This file provides the implementation of a framebuffer-based terminal for the HanOS kernel. The terminal primarily supports information display and includes functions for handling terminal commands, managing the display, and rendering text and graphics on the screen.

## 5.23 kernel/device/display/term.h File Reference

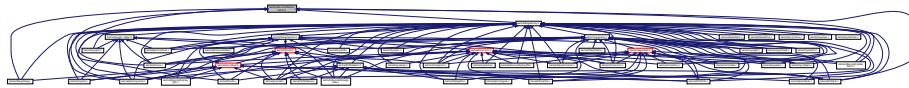
Definition of framebuffer terminal related functions.

```
#include <stdbool.h>
#include <stdint.h>
#include <device/display/fb.h>
#include <base/image.h>
```

Include dependency graph for term.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [term\\_info\\_t](#)
- struct [winsize\\_t](#)

## Macros

- `#define TERM_MODE_INFO 0`
- `#define TERM_MODE_CLI 1`
- `#define TERM_MODE_GUI 2`
- `#define TERM_MODE_UNKNOWN 3`
- `#define FONT_WIDTH 8`
- `#define FONT_HEIGHT 15`

## Enumerations

- enum `term_cursor_visible_t` { `CURSOR_INVISIBLE` = 0, `CURSOR_VISIBLE`, `CURSOR_HIDE` }

## Functions

- void `kdisplay` (char \*s)
- void `term_init` (struct `limine_framebuffer` \*s)
- void `term_putch` (int mode, uint8\_t c)
- void `term_clear` (int mode)
- void `term_refresh` (int mode)
- void `term_start` ()
- void `term_switch` (int mode)
- int `term_get_mode` (void)
- bool `term_get_redraw` (void)
- void `term_set_redraw` (bool val)
- void `term_set_cursor` (uint8\_t c)
- bool `term_set_bg_image` ([image\\_t](#) \*img)
- void `term_get_winsize` ([winsize\\_t](#) \*ws)
- bool `term_set_winsize` ([winsize\\_t](#) \*ws)

## Variables

- `term_cursor_visible_t` `cursor_visible`

### 5.23.1 Detailed Description

Definition of framebuffer terminal related functions.

A framebuffer based terminal was implemented. As the first step, it mainly supports information display.

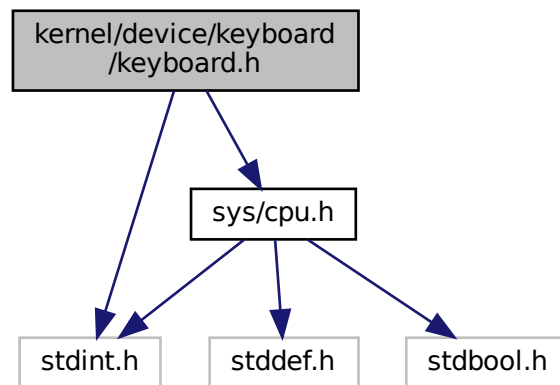
## 5.24 kernel/device/keyboard/keyboard.h File Reference

Definition of keyboard related macros and data structures.

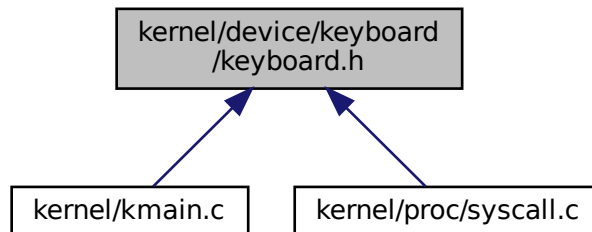
```
#include <stdint.h>
```

```
#include <sys/cpu.h>
```

Include dependency graph for keyboard.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [keyboard\\_t](#)

### Macros

- #define **KEY\_COUNT** 128
- #define **KEYBOARD\_PORT\_DATA** 0x60
- #define **KEYBOARD\_PORT\_STATUS** 0x64
- #define **KEYBOARD\_PORT\_CMD** 0x64
- #define **KEYBOARD\_CMD\_WRITE** 0x60
- #define **KEYBOARD\_CMD\_READ** 0x20

- `#define KEYBOARD_CMD_OPEN_MOUSE_INTERFACE 0xa8`
- `#define KEYBOARD_CMD_SENDTO_MOUSE 0xd4`
- `#define KEYBOARD_INIT_MODE 0x47`
- `#define KEYBOARD_MOUSE_ENABLE 0xf4`
- `#define KEYBOARD_STATUS_INBUF_FULL 0x02`
- `#define KEYBOARD_STATUS_OUTBUF_FULL 0x01`
- `#define KEYBOARD_STATUS_WHICHBUF 0x20`
- `#define WAIT_KB_WRITE() while(port_inb(KEYBOARD_PORT_STATUS) & KEYBOARD_STATUS_INBUF_FULL)`
- `#define WAIT_KB_READ() while(port_inb(KEYBOARD_PORT_STATUS) & KEYBOARD_STATUS_OUTBUF_FULL)`
- `#define KEYBOARD_DISABLE_FIRST_PORT 0xAD`
- `#define KEYBOARD_DISABLE_SECOND_PORT 0xA7`
- `#define KEYBOARD_ENABLE_FIRST_PORT 0xAE`
- `#define KEYBOARD_ENABLE_SECOND_PORT 0xA8`

## Functions

- `void keyboard_init ()`
- `uint8_t keyboard_get_key ()`

### 5.24.1 Detailed Description

Definition of keyboard related macros and data structures.

---

Keyboard interrupt is triggered by IRQ1 of 8259A (master) and mouse interrupt is triggered by IRQ4 of 8259A (slave). Please note that slave 8259A is connected to master 8259A by IRQ2 on master chip.

Mouse driver comes from SANiK.

Ref: [https://wiki.osdev.org/PS2\\_Keyboard](https://wiki.osdev.org/PS2_Keyboard)

---

In this module, both keyboard and mouse driver are implemented.

---

## 5.25 kernel/device/keyboard/keycode.h File Reference

Keycode definitions for keyboard driver.

### Macros

- `#define KB_ARROW_UP 72`
- `#define KB_BACKSPACE 14`
- `#define KB_CAPS_LOCK 58`
- `#define KB_ENTER 28`
- `#define KB_LCTRL 29 /* LCTRL same with RCTRL */`
- `#define KB_LSHIFT 42`
- `#define KB_RSHIFT 54`
- `#define KB_TAB 15`

### Functions

- `char keyboard_get_ascii (uint8_t scancode, bool shift, bool caps_lock)`



### 5.25.1 Detailed Description

Keycode definitions for keyboard driver.

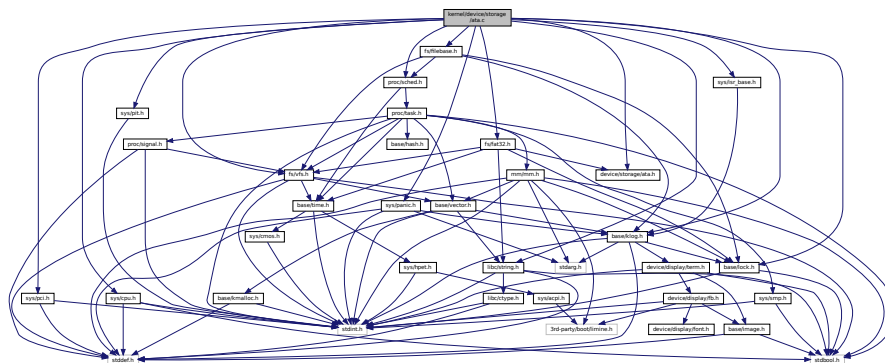
This file contains the keycode definitions and function declarations used in the HanOS keyboard driver. It defines constants for common keyboard keys such as arrow keys, backspace, caps lock, enter, control, shift, and tab. It also declares a function for converting scancode inputs into ASCII characters considering the shift and caps lock states.

## 5.26 kernel/device/storage/ata.c File Reference

Implementation of ATA (Advanced Technology Attachment) device handling.

```
#include <libc/string.h>
#include <sys/pci.h>
#include <sys/cpu.h>
#include <sys/isr_base.h>
#include <sys/panic.h>
#include <sys/pit.h>
#include <device/storage/ata.h>
#include <base/lock.h>
#include <base/klog.h>
#include <proc/sched.h>
#include <fs/filebase.h>
#include <fs/vfs.h>
#include <fs/fat32.h>
```

Include dependency graph for ata.c:



## Data Structures

- union `atapi_command_t`

## Macros

- `#define ATA_SECTOR_SIZE 512`
- `#define htonl(l) ( (((l) & 0xFF) << 24) | (((l) & 0xFF00) << 8) | (((l) & 0xFF0000) >> 8) | (((l) & 0xFF000000) >> 24) )`

## Functions

- int **ata\_init** (void)
- void **ata\_pio\_read28** ([ata\\_device\\_t](#) \*dev, uint32\_t lba, uint8\_t sector\_count, uint8\_t \*target)
- void **ata\_pio\_write28** ([ata\\_device\\_t](#) \*dev, uint32\_t lba, uint8\_t sector\_count, uint8\_t \*source)

### 5.26.1 Detailed Description

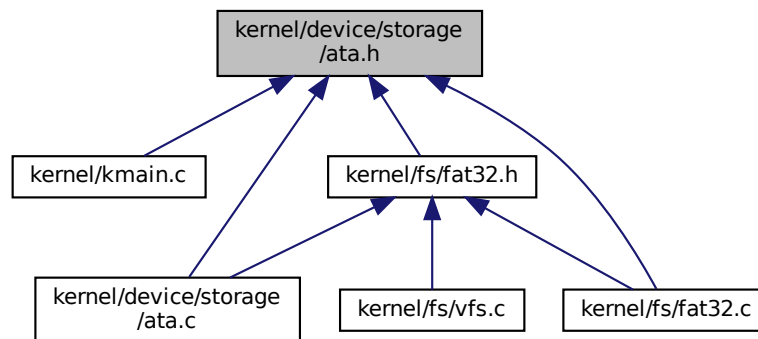
Implementation of ATA (Advanced Technology Attachment) device handling.

This file contains the implementation of functions for handling ATA devices within the HanOS kernel. It includes functions for initializing and detecting ATA and ATAPI devices, performing PIO read and write operations, and reading partition maps. Additionally, it provides utility functions for interacting with ATA registers and handling device-specific operations such as soft resets and polling for device status.

## 5.27 kernel/device/storage/ata.h File Reference

ATA (Advanced Technology Attachment) definitions.

This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [ide\\_channel\\_regs\\_t](#)
- struct [ide\\_device\\_t](#)
- struct [partition\\_t](#)
- struct [ata\\_identify\\_t](#)
- struct [prdt\\_t](#)
- struct [ata\\_device\\_t](#)
- struct [mbr\\_t](#)

### Macros

- #define **ATA\_SR\_BSY** 0x80
- #define **ATA\_SR\_DRDY** 0x40
- #define **ATA\_SR\_DF** 0x20
- #define **ATA\_SR\_DSC** 0x10
- #define **ATA\_SR\_DRQ** 0x08
- #define **ATA\_SR\_CORR** 0x04
- #define **ATA\_SR\_IDX** 0x02
- #define **ATA\_SR\_ERR** 0x01
- #define **ATA\_ER\_BBK** 0x80
- #define **ATA\_ER\_UNC** 0x40
- #define **ATA\_ER\_MC** 0x20
- #define **ATA\_ER\_IDNF** 0x10

- `#define ATA_ER_MCR 0x08`
- `#define ATA_ER_ABRT 0x04`
- `#define ATA_ER_TK0NF 0x02`
- `#define ATA_ER_AMNF 0x01`
- `#define ATA_CMD_READ_PIO 0x20`
- `#define ATA_CMD_READ_PIO_EXT 0x24`
- `#define ATA_CMD_READ_DMA 0xC8`
- `#define ATA_CMD_READ_DMA_EXT 0x25`
- `#define ATA_CMD_WRITE_PIO 0x30`
- `#define ATA_CMD_WRITE_PIO_EXT 0x34`
- `#define ATA_CMD_WRITE_DMA 0xCA`
- `#define ATA_CMD_WRITE_DMA_EXT 0x35`
- `#define ATA_CMD_CACHE_FLUSH 0xE7`
- `#define ATA_CMD_CACHE_FLUSH_EXT 0xEA`
- `#define ATA_CMD_PACKET 0xA0`
- `#define ATA_CMD_IDENTIFY_PACKET 0xA1`
- `#define ATA_CMD_IDENTIFY 0xEC`
- `#define ATAPI_CMD_READ 0xA8`
- `#define ATAPI_CMD_EJECT 0x1B`
- `#define ATA_IDENT_DEVICETYPE 0`
- `#define ATA_IDENT_CYLINDERS 2`
- `#define ATA_IDENT_HEADS 6`
- `#define ATA_IDENT_SECTORS 12`
- `#define ATA_IDENT_SERIAL 20`
- `#define ATA_IDENT_MODEL 54`
- `#define ATA_IDENT_CAPABILITIES 98`
- `#define ATA_IDENT_FIELDVALID 106`
- `#define ATA_IDENT_MAX_LBA 120`
- `#define ATA_IDENT_COMMANDSETS 164`
- `#define ATA_IDENT_MAX_LBA_EXT 200`
- `#define IDE_ATA 0x00`
- `#define IDE_ATAPI 0x01`
- `#define ATA_MASTER 0x00`
- `#define ATA_SLAVE 0x01`
- `#define ATA_REG_DATA 0x00`
- `#define ATA_REG_ERROR 0x01`
- `#define ATA_REG_FEATURES 0x01`
- `#define ATA_REG_SECCOUNT0 0x02`
- `#define ATA_REG_LBA0 0x03`
- `#define ATA_REG_LBA1 0x04`
- `#define ATA_REG_LBA2 0x05`
- `#define ATA_REG_HDDEVSEL 0x06`
- `#define ATA_REG_COMMAND 0x07`
- `#define ATA_REG_STATUS 0x07`
- `#define ATA_REG_SECCOUNT1 0x08`
- `#define ATA_REG_LBA3 0x09`
- `#define ATA_REG_LBA4 0x0A`
- `#define ATA_REG_LBA5 0x0B`
- `#define ATA_REG_CONTROL 0x0C`
- `#define ATA_REG_ALTSTATUS 0x0C`
- `#define ATA_REG_DEVADDRESS 0x0D`
- `#define ATA_PRIMARY 0x00`
- `#define ATA_SECONDARY 0x01`
- `#define ATA_READ 0x00`
- `#define ATA_WRITE 0x01`



- `int64_t fat32_read (vfs_inode_t *this, uint64_t offset, uint64_t len, void *buff)`
- `int64_t fat32_write (vfs_inode_t *this, uint64_t offset, uint64_t len, const void *buff)`
- `int64_t fat32_sync (vfs_inode_t *this)`
- `int64_t fat32_getdent (vfs_inode_t *this, uint64_t pos, vfs_dirent_t *dirent)`
- `int64_t fat32_refresh (vfs_inode_t *this)`
- `int64_t fat32_mknode (vfs_tnode_t *this)`
- `int fat32_compare_entry_and_path (fat32_entry_t *ent, const char *path)`
- `void fat32_dump_entry (fat32_entry_t fe)`
- `fat32_entry_t fat32_parse_path (vfs_inode_t *this, const char *path)`
- `vfs_tnode_t * fat32_open (vfs_inode_t *this, const char *path)`
- `vfs_inode_t * fat32_mount (vfs_inode_t *at)`

## Variables

- `vfs_fsinfo_t fat32`

### 5.28.1 Detailed Description

Implementation of FAT32 related functions.

---

The functions in this file are all needed by VFS requirements. The r/w of FAT32 are PIO 28 which need to be improved for higher speed.

---

### 5.28.2 Variable Documentation

#### 5.28.2.1 fat32

`vfs_fsinfo_t fat32`

**Initial value:**

```
= {
 .name = "fat32",
 .istemp = false,
 .filelist = {0},
 .open = fat32_open,
 .mount = fat32_mount,
 .mknode = fat32_mknode,
 .rmnode = NULL,
 .sync = fat32_sync,
 .refresh = fat32_refresh,
 .read = fat32_read,
 .getdent = fat32_getdent,
 .write = fat32_write,
 .ioctl = NULL
}
```

## 5.29 kernel/fs/fat32.h File Reference

Definition of FAT32 related data structures and functions.

```
#include <libc/string.h>
#include <fs/vfs.h>
#include <base/lock.h>
#include <base/time.h>
#include <device/storage/ata.h>
```

---



## Functions

- `vfs_inode_t * fat32_mount (vfs_inode_t *at)`
- `vfs_tnode_t * fat32_open (vfs_inode_t *this, const char *path)`
- `int64_t fat32_mknode (vfs_tnode_t *this)`
- `int64_t fat32_read (vfs_inode_t *this, uint64_t offset, uint64_t len, void *buff)`
- `int64_t fat32_write (vfs_inode_t *this, uint64_t offset, uint64_t len, const void *buff)`
- `int64_t fat32_sync (vfs_inode_t *this)`
- `int64_t fat32_refresh (vfs_inode_t *this)`
- `int64_t fat32_getdent (vfs_inode_t *this, uint64_t pos, vfs_dirent_t *dirent)`

## Variables

- `vfs_fsinfo_t` **fat32**

### 5.29.1 Detailed Description

### Definition of FAT32 related data structures and functions.

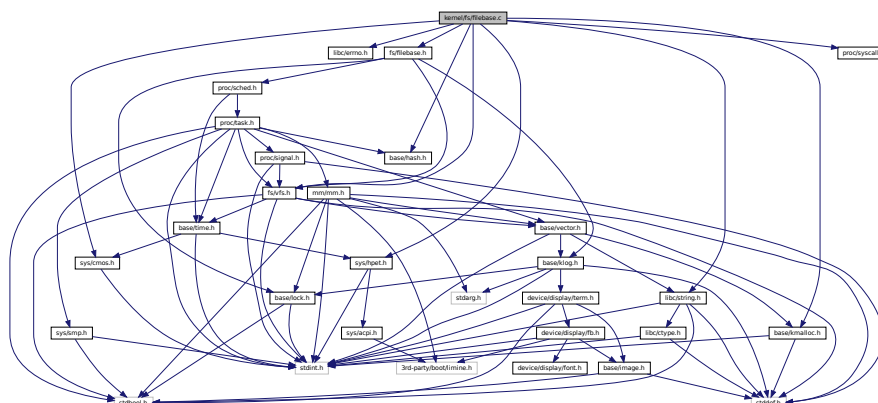
FAT 32 was introduced to us by Windows95-B and Windows98. FAT32 solved some of FAT's problems. No more 64K max clusters! Although FAT32 uses 32 bits per FAT entry, only the bottom 28 bits are actually used to address clusters on the disk (top 4 bits are reserved). With 28 bits per FAT entry, the filesystem can address a maximum of about 270 million clusters in a partition. This enables very large hard disks to still maintain reasonably small cluster sizes and thus reduce slack space between files.

### 5.30 kernel/fs/filebase.c File Reference

Implementation of fundamental VFS file node related functions.

```
#include <libc/string.h>
#include <libc/errno.h>
#include <fs/filebase.h>
#include <base/kmalloc.h>
#include <base/hash.h>
#include <sys/hpet.h>
#include <sys/cmos.h>
#include <fs/vfs.h>
#include <proc/syscall.h>
```

Include dependency graph for filebase.c:







## Functions

- `vfs_tnode_t * vfs_alloc_tnode` (const char \*name, `vfs_inode_t` \*inode, `vfs_inode_t` \*parent)
- `vfs_inode_t * vfs_alloc_inode` (`vfs_node_type_t` type, uint32\_t perms, uint32\_t uid, `vfs_fsinfo_t` \*fs, `vfs_tnode_t` \*mnt)
- void `vfs_free_nodes` (`vfs_tnode_t` \*tnode)
- `vfs_node_desc_t * vfs_handle_to_fd` (`vfs_handle_t` handle, const char \*func)
- `vfs_tnode_t * vfs_path_to_node` (const char \*path, uint8\_t mode, `vfs_node_type_t` create\_type)

## Variables

- `lock_t` `vfs_lock`
- `vfs_tnode_t` `vfs_root`

### 5.31.1 Detailed Description

Definition of fundamental VFS file node related functions.

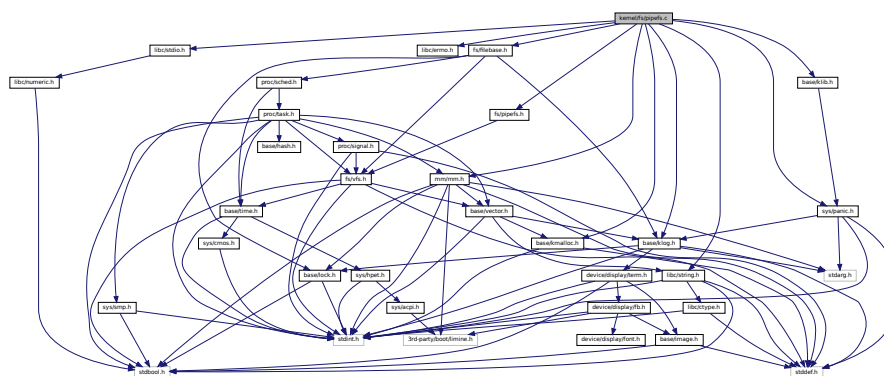
This file contains definition of inode (index node) and tnode (tree node) related functions, e.g., alloc, free and node to fd (file descriptor), path to node conversions.

## 5.32 kernel/fs/pipefs.c File Reference

Implementation of PipeFS functions.

```
#include <libc/string.h>
#include <libc/errno.h>
#include <libc/stdio.h>
#include <fs/pipefs.h>
#include <fs/filebase.h>
#include <base/kmalloc.h>
#include <base/klog.h>
#include <base/klib.h>
#include <sys/panic.h>
#include <mm/mm.h>
```

Include dependency graph for pipefs.c:



## Data Structures

- struct `pipefs_ident_t`

## Macros

- `#define PIPE_BUFFER_SIZE 4096`

## Functions

- void **pipefs\_init** (void)
- [vfs\\_tnode\\_t](#) \* **pipefs\_open** ([vfs\\_inode\\_t](#) \*this, const char \*path)
- int64\_t **pipefs\_read** ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, void \*buff)
- int64\_t **pipefs\_write** ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, const void \*buff)
- int64\_t **pipefs\_mknnode** ([vfs\\_tnode\\_t](#) \*this)
- int64\_t **pipefs\_rmnode** ([vfs\\_tnode\\_t](#) \*this)
- [vfs\\_inode\\_t](#) \* **pipefs\_mount** ([vfs\\_inode\\_t](#) \*at)

## Variables

- [vfs\\_fsinfo\\_t](#) **pipefs**
- [lock\\_t](#) **pipe\_lock** = {0}
- [lock\\_t](#) **vfs\_lock**
- uint8\_t **pipe\_eof\_magic\_word** [5] = {0xFF, 0x0E, 0x00, 0x0F, 0x00}

### 5.32.1 Detailed Description

Implementation of PipeFS functions.

---

This file contains the implementation of the PipeFS, a simple filesystem for handling pipe files within the HanOS Kernel. It provides functions for mounting, creating, removing, opening, reading, and writing pipe files. The PipeFS is designed to facilitate inter-process communication by using pipes as a means to transfer data between processes.

---

### 5.32.2 Variable Documentation

#### 5.32.2.1 pipefs

[vfs\\_fsinfo\\_t](#) pipefs

**Initial value:**

```
= {
 .name = "pipefs",
 .istemp = true,
 .filelist = {0},
 .open = pipefs_open,
 .mount = pipefs_mount,
 .mknnode = pipefs_mknnode,
 .rmnode = pipefs_rmnode,
 .sync = NULL,
 .refresh = NULL,
 .read = pipefs_read,
 .getdent = NULL,
 .write = pipefs_write,
 .ioctl = NULL
}
```

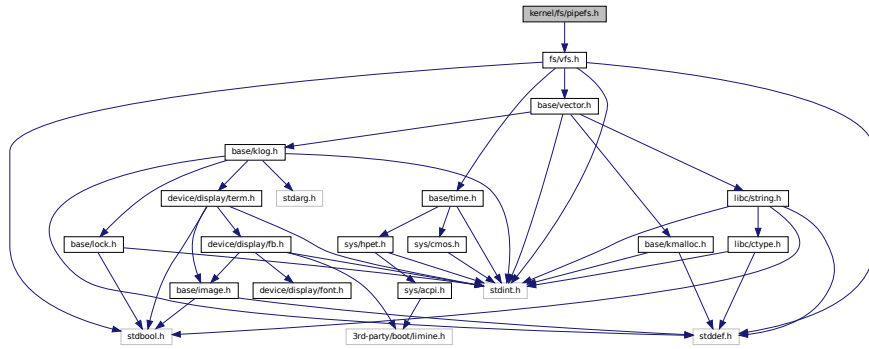
## 5.33 kernel/fs/pipefs.h File Reference

Definition of PipeFS related data structures and functions.

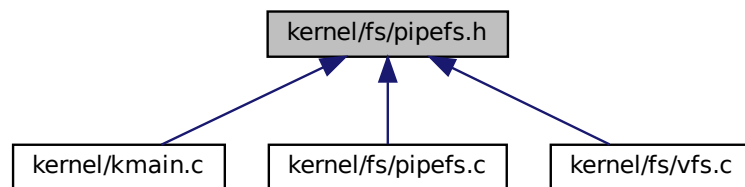
---

```
#include <fs/vfs.h>
```

Include dependency graph for pipefs.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [pipefs\\_file\\_t](#)
- struct [pipefs\\_ident\\_item\\_t](#)

## Functions

- [vfs\\_inode\\_t](#) \* **pipefs\_mount** ([vfs\\_inode\\_t](#) \*at)
- int64\_t **pipefs\_mknode** ([vfs\\_tnode\\_t](#) \*this)
- int64\_t **pipefs\_rmnode** ([vfs\\_tnode\\_t](#) \*this)
- [vfs\\_tnode\\_t](#) \* **pipefs\_open** ([vfs\\_inode\\_t](#) \*this, const char \*path)
- int64\_t **pipefs\_read** ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, void \*buff)
- int64\_t **pipefs\_write** ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, const void \*buff)
- void **pipefs\_init** (void)

## Variables

- [vfs\\_fsinfo\\_t](#) **pipefs**

### 5.33.1 Detailed Description

Definition of PipeFS related data structures and functions.

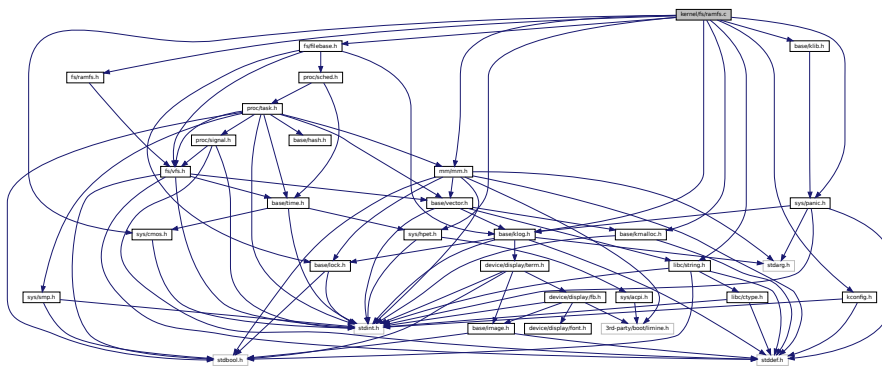
PipeFS is a simple file system designed to handle pipe files within the HanOS Kernel. It provides basic file system operations such as mounting, creating, removing, opening, reading, and writing pipe files. This file defines the necessary data structures and function prototypes used by PipeFS.

## 5.34 kernel/fs/ramfs.c File Reference

Implementation of RAMFS functions.

```
#include <libc/string.h>
#include <fs/ramfs.h>
#include <fs/filebase.h>
#include <base/kmalloc.h>
#include <base/klog.h>
#include <base/klib.h>
#include <sys/panic.h>
#include <sys/hpet.h>
#include <sys/cmos.h>
#include <mm/mm.h>
#include <kconfig.h>
```

Include dependency graph for ramfs.c:



## Data Structures

- struct [ramfs\\_ident\\_t](#)

## Functions

- void **ramfs\_init** (void \*address, uint64\_t size)
- [vfs\\_tnode\\_t](#) \* **ramfs\_open** ([vfs\\_inode\\_t](#) \*this, const char \*pathname)
- int64\_t **ramfs\_read** ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, void \*buff)
- int64\_t **ramfs\_rmnode** ([vfs\\_tnode\\_t](#) \*this)
- int64\_t **ramfs\_write** ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, const void \*buff)
- int64\_t **ramfs\_sync** ([vfs\\_inode\\_t](#) \*this)
- int64\_t **ramfs\_setlink** ([vfs\\_tnode\\_t](#) \*this, [vfs\\_inode\\_t](#) \*inode)
- int64\_t **ramfs\_refresh** ([vfs\\_inode\\_t](#) \*this)
- int64\_t **ramfs\_getdent** ([vfs\\_inode\\_t](#) \*this, uint64\_t pos, [vfs\\_dirent\\_t](#) \*dirent)
- int64\_t **ramfs\_mknnode** ([vfs\\_tnode\\_t](#) \*this)
- [vfs\\_inode\\_t](#) \* **ramfs\_mount** ([vfs\\_inode\\_t](#) \*at)

## Variables

- [vfs\\_fsinfo\\_t](#) **ramfs**

### 5.34.1 Detailed Description

Implementation of RAMFS functions.

This file contains the implementation of the RAMFS, a simple filesystem that resides entirely in memory. It provides functions for mounting, creating, removing, opening, reading, writing, and syncing files. The RAMFS is designed to facilitate fast in-memory file operations.

### 5.34.2 Variable Documentation

#### 5.34.2.1 ramfs

`vfs_fsinfo_t` ramfs

Initial value:

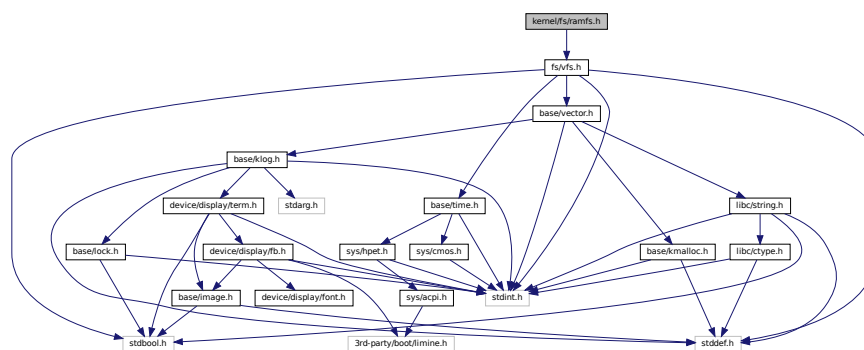
```
= {
 .name = "ramfs",
 .istemp = true,
 .filelist = {0},
 .open = ramfs_open,
 .mount = ramfs_mount,
 .mknod = ramfs_mknod,
 .rmnode = ramfs_rmnode,
 .sync = ramfs_sync,
 .refresh = ramfs_refresh,
 .read = ramfs_read,
 .getdent = ramfs_getdent,
 .write = ramfs_write,
 .ioctl = NULL
}
```

## 5.35 kernel/fs/ramfs.h File Reference

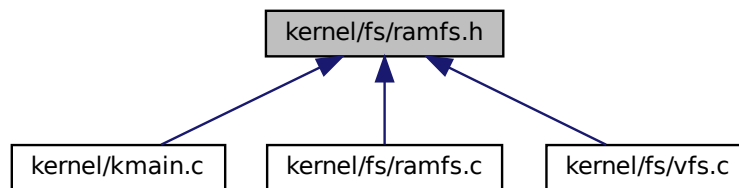
Definition of RAMFS related data structures and functions.

```
#include <fs/vfs.h>
```

Include dependency graph for ramfs.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [ramfs\\_file\\_t](#)
- struct [ustar\\_file\\_t](#)
- struct [ramfs\\_ident\\_item\\_t](#)

## Functions

- [vfs\\_inode\\_t](#) \* **ramfs\_mount** ([vfs\\_inode\\_t](#) \*at)
- int64\_t **ramfs\_mknode** ([vfs\\_tnode\\_t](#) \*this)
- int64\_t **ramfs\_rmnode** ([vfs\\_tnode\\_t](#) \*this)
- [vfs\\_tnode\\_t](#) \* **ramfs\_open** ([vfs\\_inode\\_t](#) \*this, const char \*path)
- int64\_t **ramfs\_read** ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, void \*buff)
- int64\_t **ramfs\_getdent** ([vfs\\_inode\\_t](#) \*this, uint64\_t pos, [vfs\\_dirent\\_t](#) \*dirent)
- int64\_t **ramfs\_write** ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, const void \*buff)
- int64\_t **ramfs\_sync** ([vfs\\_inode\\_t](#) \*this)
- int64\_t **ramfs\_refresh** ([vfs\\_inode\\_t](#) \*this)
- void **ramfs\_init** (void \*address, uint64\_t size)

## Variables

- [vfs\\_fsinfo\\_t](#) **ramfs**

### 5.35.1 Detailed Description

Definition of RAMFS related data structures and functions.

RAMFS is a simple filesystem that resides entirely in memory. It provides basic file system operations such as mounting, creating, removing, opening, reading, writing, and syncing files. This file defines the necessary data structures and function prototypes used by RAMFS.

## 5.36 kernel/fs/ttyfs.c File Reference

Implementation of TTYFS functions.

```

#include <libc/string.h>
#include <libc/errno.h>
#include <libc/stdio.h>
#include <fs/ttyfs.h>
#include <fs/filebase.h>
#include <base/kmalloc.h>
#include <base/klog.h>

```



- `#define VSTOP 8`
- `#define VSUSP 9`
- `#define VTIME 10`
- `#define NCCS 11`
- `#define TTY_BUFFER_SIZE 4096`

## Typedefs

- `typedef uint32_t cc_t`
- `typedef uint32_t speed_t`
- `typedef uint32_t tcflag_t`

## Functions

- `void ttyfs_init (void)`
- `int64_t ttyfs_ioctl (vfs_inode_t *this, int64_t request, int64_t arg)`
- `vfs_tnode_t * ttyfs_open (vfs_inode_t *this, const char *path)`
- `int64_t ttyfs_read (vfs_inode_t *this, uint64_t offset, uint64_t len, void *buff)`
- `int64_t ttyfs_write (vfs_inode_t *this, uint64_t offset, uint64_t len, const void *buff)`
- `int64_t ttyfs_sync (vfs_inode_t *this)`
- `int64_t ttyfs_refresh (vfs_inode_t *this)`
- `int64_t ttyfs_getdent (vfs_inode_t *this, uint64_t pos, vfs_dirent_t *dirent)`
- `int64_t ttyfs_mknode (vfs_tnode_t *this)`
- `vfs_inode_t * ttyfs_mount (vfs_inode_t *at)`

## Variables

- `vfs_fsinfo_t ttyfs`
- `lock_t tty_lock`

### 5.36.1 Detailed Description

Implementation of TTYFS functions.

---

This file contains the implementation of the TTYFS (Teletypewriter Filesystem), a simple filesystem designed to handle TTY files within the HanOS Kernel. It provides functions for mounting, creating, opening, reading, writing, and syncing TTY files, as well as handling IO control operations.

---

### 5.36.2 Variable Documentation

#### 5.36.2.1 ttyfs

`vfs_fsinfo_t ttyfs`

**Initial value:**

```
= {
 .name = "ttyfs",
 .istemp = true,
 .filelist = {0},
 .open = ttyfs_open,
 .mount = ttyfs_mount,
 .mknode = ttyfs_mknode,
 .rmnode = NULL,
 .sync = ttyfs_sync,
 .refresh = ttyfs_refresh,
 .read = ttyfs_read,
 .getdent = ttyfs_getdent,
 .write = ttyfs_write,
 .ioctl = ttyfs_ioctl
}
```

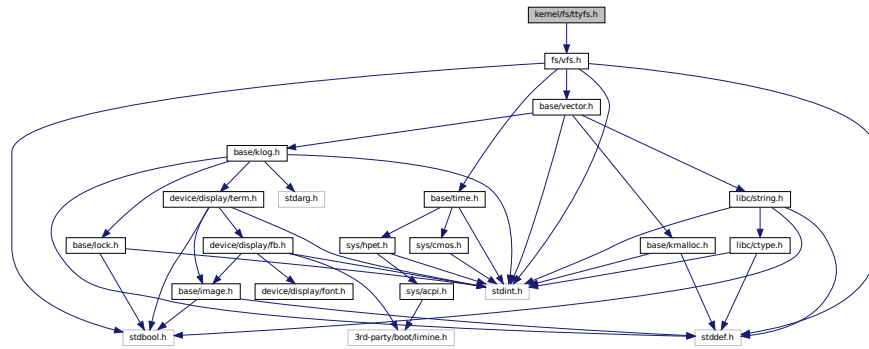


## 5.37 kernel/fs/ttyfs.h File Reference

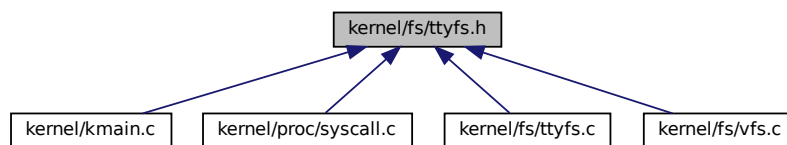
Definition of TTYFS related data structures and functions.

```
#include <fs/vfs.h>
```

Include dependency graph for ttyfs.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [ttyfs\\_file\\_t](#)
- struct [ttyfs\\_ident\\_item\\_t](#)

### Functions

- [vfs\\_inode\\_t](#) \* [ttyfs\\_mount](#) ([vfs\\_inode\\_t](#) \*at)
- int64\_t [ttyfs\\_mknode](#) ([vfs\\_tnode\\_t](#) \*this)
- [vfs\\_tnode\\_t](#) \* [ttyfs\\_open](#) ([vfs\\_inode\\_t](#) \*this, const char \*path)
- int64\_t [ttyfs\\_read](#) ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, void \*buff)
- int64\_t [ttyfs\\_getdent](#) ([vfs\\_inode\\_t](#) \*this, uint64\_t pos, [vfs\\_dirent\\_t](#) \*dirent)
- int64\_t [ttyfs\\_write](#) ([vfs\\_inode\\_t](#) \*this, uint64\_t offset, uint64\_t len, const void \*buff)
- int64\_t [ttyfs\\_sync](#) ([vfs\\_inode\\_t](#) \*this)
- int64\_t [ttyfs\\_refresh](#) ([vfs\\_inode\\_t](#) \*this)
- int64\_t [ttyfs\\_ioctl](#) ([vfs\\_inode\\_t](#) \*this, int64\_t request, int64\_t arg)
- void [ttyfs\\_init](#) (void)

### Variables

- [vfs\\_fsinfo\\_t](#) [ttyfs](#)

### 5.37.1 Detailed Description

#### Definition of TTYFS related data structures and functions.

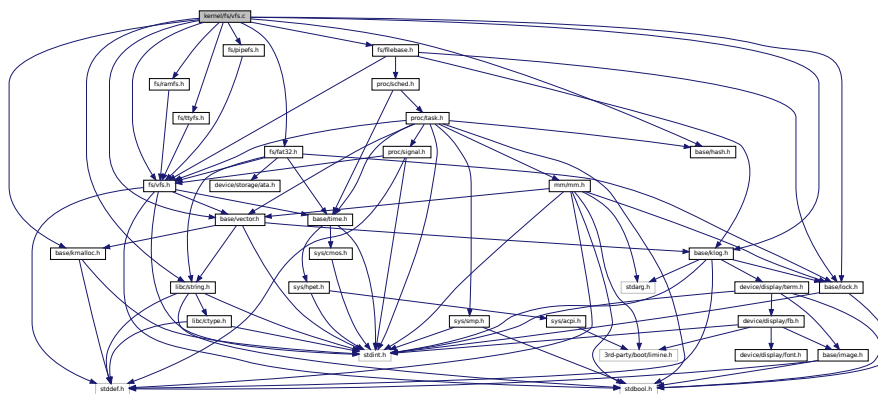
TTYFS is a simple filesystem designed to handle TTY (teletypewriter) files within the HanOS Kernel. It provides basic file system operations such as mounting, creating, opening, reading, writing, and syncing TTY files. This file defines the necessary data structures and function prototypes used by TTYFS.

## 5.38 kernel/fs/vfs.c File Reference

Implementation of VFS related functions.

```
#include <libc/string.h>
#include <fs/vfs.h>
#include <fs/filebase.h>
#include <fs/fat32.h>
#include <fs/ramfs.h>
#include <fs/ttyfs.h>
#include <fs/pipefs.h>
#include <base/klog.h>
#include <base/kmalloc.h>
#include <base/lock.h>
#include <base/vector.h>
#include <base/hash.h>
```

Include dependency graph for vfs.c:



## Functions

- `vec_new_static(vfs_fsinfo_t *, vfs_fslist)`
- `dev_t vfs_new_dev_id(void)`
- `ino_t vfs_new_ino_id(void)`
- `void vfs_debug()`
- `void vfs_register_fs(vfs_fsinfo_t *fs)`
- `vfs_fsinfo_t * vfs_get_fs(char *name)`
- `void vfs_init()`
- `int64_t vfs_create(char *path, vfs_node_type_t type)`
- `int64_t vfs_chmod(vfs_handle_t handle, int32_t newperms)`
- `int64_t vfs_ioctl(vfs_handle_t handle, int64_t request, int64_t arg)`
- `int64_t vfs_mount(char *device, char *path, char *fsname)`
- `uint64_t vfs_tell(vfs_handle_t handle)`
- `int64_t vfs_read(vfs_handle_t handle, uint64_t len, void *buff)`
- `int64_t vfs_unlink(char *path)`

- `int64_t` **vfs\_write** (`vfs_handle_t` handle, `uint64_t` len, `const void *`buff)
- `int64_t` **vfs\_seek** (`vfs_handle_t` handle, `uint64_t` pos, `int64_t` whence)
- `int64_t` **vfs\_get\_parent\_dir** (`const char *`path, `char *`parent, `char *`currdir)
- `vfs_handle_t` **vfs\_open** (`char *`path, `vfs_openmode_t` mode)
- `int64_t` **vfs\_close** (`vfs_handle_t` handle)
- `int64_t` **vfs\_refresh** (`vfs_handle_t` handle)
- `int64_t` **vfs\_getdent** (`vfs_handle_t` handle, `vfs_dirent_t *`dirent)

## Variables

- `lock_t` **vfs\_lock** = {0}
- `lock_t` **dev\_lock** = {0}
- `lock_t` **ino\_lock** = {0}
- `vfs_tnode_t` **vfs\_root** = {0}
- `uint8_t` **pipe\_eof\_magic\_word** []

## 5.38.1 Detailed Description

### Implementation of VFS related functions.

VFS is an abstraction layer that provides a unified interface for various physical file systems. This allows users to access the file system through standard file operation functions without knowing the details of the underlying physical file system.

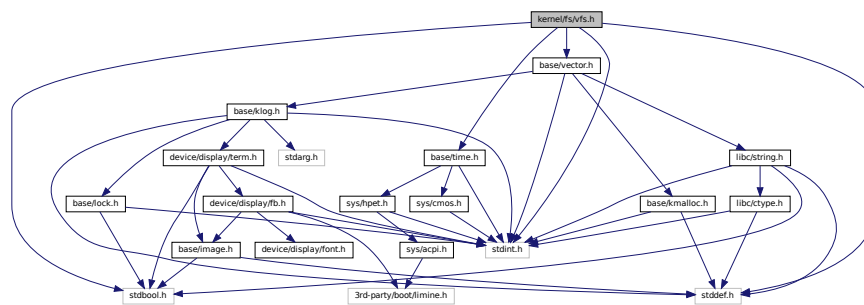
Like all Unix-like system, inode is the fundamental data structure of VFS which stores file index information. All children node pointers will be stored in inode. tnode is used to store tree information, e.g., parent node. node\_desc data structure is used for every file operation, from fopen, fread to fclose.

## 5.39 kernel/fs/vfs.h File Reference

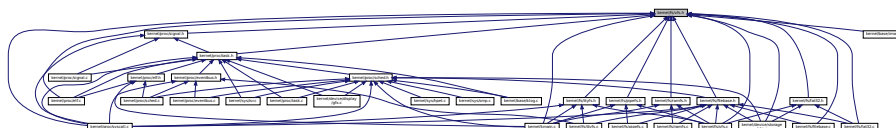
Definition of VFS related data structures and functions.

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <base/vector.h>
#include <base/time.h>
```

Include dependency graph for `vfs.h`:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [dirent\\_t](#)
- struct [vfs\\_timespec\\_t](#)
- struct [vfs\\_stat\\_t](#)
- struct [vfs\\_dirent\\_t](#)
- struct [vfs\\_fsinfo\\_t](#)
- struct [vfs\\_tnode\\_t](#)
- struct [vfs\\_inode\\_t](#)
- struct [vfs\\_node\\_desc\\_t](#)

## Macros

- `#define VFS_MAX_PATH_LEN 4096`
- `#define VFS_MAX_NAME_LEN 256`
- `#define VFS_FDCWD -100`
- `#define VFS_INVALID_HANDLE -1`
- `#define VFS_MIN_HANDLE 100`
- `#define SEEK_CUR 1`
- `#define SEEK_END 2`
- `#define SEEK_SET 3`
- `#define DT_UNKNOWN 0`
- `#define DT_FIFO 1`
- `#define DT_CHR 2`
- `#define DT_DIR 4`
- `#define DT_BLK 6`
- `#define DT_REG 8`
- `#define DT_LNK 10`
- `#define DT SOCK 12`
- `#define DT_WHT 14`
- `#define S_IFMT 0170000 /* bit mask for the file type bit field */`
- `#define S_IFSOCK 0140000 /* socket */`
- `#define S_IFLNK 0120000 /* symbolic link */`
- `#define S_IFREG 0100000 /* regular file */`
- `#define S_IFBLK 0060000 /* block device */`
- `#define S_IFDIR 0040000 /* directory */`
- `#define S_IFCHR 0020000 /* character device */`
- `#define S_IFIFO 0010000 /* FIFO */`
- `#define S_ISUID 04000 /* set-user-ID bit */`
- `#define S_ISGID 02000 /* set-group-ID bit */`
- `#define S_ISVTX 01000 /* sticky bit */`
- `#define S_IRWXU`
- `#define S_IRUSR 00400 /* owner has read permission */`
- `#define S_IWUSR 00200 /* owner has write permission */`
- `#define S_IXUSR 00100 /* owner has execute permission */`
- `#define S_IRWXG`
- `#define S_IRGRP 00040 /* group has read permission */`
- `#define S_IWGRP 00020 /* group has write permission */`
- `#define S_IXGRP 00010 /* group has execute permission */`
- `#define S_IRWXO`
- `#define S_IROTH 00004 /* others have read permission */`
- `#define S_IWOTH 00002 /* others have write permission */`
- `#define S_IXOTH 00001 /* others have execute permission */`

## Typedefs

- typedef int64\_t **dev\_t**
- typedef uint64\_t **ino\_t**
- typedef int64\_t **off\_t**
- typedef int32\_t **mode\_t**
- typedef int32\_t **nlink\_t**
- typedef int64\_t **blksize\_t**
- typedef int64\_t **blkcnt\_t**
- typedef int32\_t **pid\_t**
- typedef int32\_t **tid\_t**
- typedef int32\_t **uid\_t**
- typedef int32\_t **gid\_t**
- typedef int64\_t **vfs\_handle\_t**
- typedef struct [vfs\\_inode\\_t](#) **vfs\_inode\_t**
- typedef struct [vfs\\_tnode\\_t](#) **vfs\_tnode\_t**
- typedef struct [vfs\\_fsinfo\\_t](#) **vfs\_fsinfo\_t**

## Enumerations

- enum **vfs\_node\_type\_t** {  
**VFS\_NODE\_FILE**, **VFS\_NODE\_SYMLINK**, **VFS\_NODE\_FOLDER**, **VFS\_NODE\_BLOCK\_DEVICE**,  
**VFS\_NODE\_CHAR\_DEVICE**, **VFS\_NODE\_MOUNTPOINT**, **VFS\_NODE\_INVALID** }
- enum **vfs\_openmode\_t** { **VFS\_MODE\_READ**, **VFS\_MODE\_WRITE**, **VFS\_MODE\_READWRITE** }

## Functions

- int64\_t **vfs\_get\_parent\_dir** (const char \*path, char \*parent, char \*currdir)
- void **vfs\_init** ()
- void **vfs\_register\_fs** ([vfs\\_fsinfo\\_t](#) \*fs)
- [vfs\\_fsinfo\\_t](#) \* **vfs\_get\_fs** (char \*name)
- void **vfs\_debug** ()
- [vfs\\_handle\\_t](#) **vfs\_open** (char \*path, [vfs\\_openmode\\_t](#) mode)
- int64\_t **vfs\_create** (char \*path, [vfs\\_node\\_type\\_t](#) type)
- int64\_t **vfs\_close** ([vfs\\_handle\\_t](#) handle)
- uint64\_t **vfs\_tell** ([vfs\\_handle\\_t](#) handle)
- int64\_t **vfs\_seek** ([vfs\\_handle\\_t](#) handle, uint64\_t pos, int64\_t whence)
- int64\_t **vfs\_read** ([vfs\\_handle\\_t](#) handle, uint64\_t len, void \*buff)
- int64\_t **vfs\_write** ([vfs\\_handle\\_t](#) handle, uint64\_t len, const void \*buff)
- int64\_t **vfs\_unlink** (char \*path)
- int64\_t **vfs\_chmod** ([vfs\\_handle\\_t](#) handle, int32\_t newperms)
- int64\_t **vfs\_refresh** ([vfs\\_handle\\_t](#) handle)
- int64\_t **vfs\_getdent** ([vfs\\_handle\\_t](#) handle, [vfs\\_dirent\\_t](#) \*dirent)
- int64\_t **vfs\_mount** (char \*device, char \*path, char \*fsname)
- int64\_t **vfs\_ioctl** ([vfs\\_handle\\_t](#) handle, int64\_t request, int64\_t arg)
- [dev\\_t](#) **vfs\_new\_dev\_id** (void)
- [ino\\_t](#) **vfs\_new\_ino\_id** (void)

### 5.39.1 Detailed Description

#### Definition of VFS related data structures and functions.

---

VFS is an abstraction layer that provides a unified interface for various physical file systems. This allows users to access the file system through standard file operation functions without knowing the details of the underlying physical file system.

Like all Unix-like system, inode is the fundamental data structure of VFS which stores file index information. All children node pointers will be stored in inode. tnode is used to store tree information, e.g., parent node. node\_desc data structure is used for every file operation, from fopen, fread to fclose.

---

## 5.39.2 Macro Definition Documentation

### 5.39.2.1 S\_IRWXG

```
#define S_IRWXG
```

**Value:**

```
00070 /* group has read, write, and execute
 * permission */
```

### 5.39.2.2 S\_IRWXO

```
#define S_IRWXO
```

**Value:**

```
00007 /* others (not in group) have read, write,
 * and execute permission */
```

### 5.39.2.3 S\_IRWXU

```
#define S_IRWXU
```

**Value:**

```
00700 /* owner has read, write, and execute
 * permission */
```

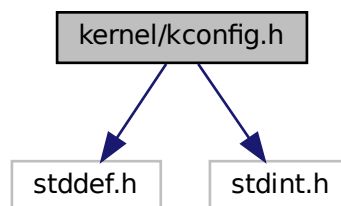
## 5.40 kernel/kconfig.h File Reference

Kernel configuration header.

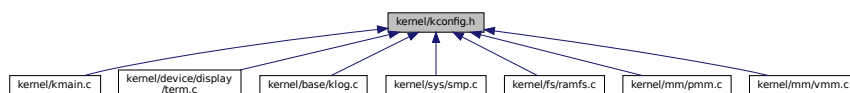
```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for kconfig.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [computer\\_info\\_t](#)

## Macros

- `#define LAUNCHER_CLI true /* default: true */`
- `#define BSP_CORE_ONLY true /* default: true */`
- `#define ENABLE_KLOG_DEBUG false /* default: false */`
- `#define ENABLE_MEM_DEBUG false /* default: false */`
- `#define ENABLE_BASH false /* default: false */`
- `#define DEFAULT_SHELL_APP "/bin/init"`
- `#define DEFAULT_INPUT_SVR "/server/input"`
- `#define DEFAULT_TZ_SEC_SHIFT (8 * 60 * 60)`

### 5.40.1 Detailed Description

Kernel configuration header.

---

This header file contains configuration macros and type definitions used in the HanOS kernel. It includes standard headers and defines various configuration options and default values for the kernel.

---

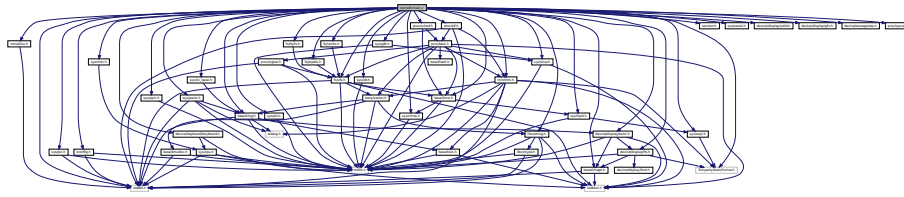
## 5.41 kernel/kmain.c File Reference

Entry function of HanOS kernel.

```
#include <stddef.h>
#include <libc/string.h>
#include <kconfig.h>
#include <version.h>
#include <3rd-party/boot/limine.h>
#include <base/time.h>
#include <base/image.h>
#include <base/klog.h>
#include <mm/mm.h>
#include <mm/alloc.h>
#include <sys/gdt.h>
#include <sys/idt.h>
#include <sys/isr_base.h>
#include <sys/smp.h>
#include <sys/mtrr.h>
#include <sys/cmos.h>
#include <sys/serial.h>
#include <sys/acpi.h>
#include <sys/apic.h>
#include <sys/hpet.h>
#include <sys/panic.h>
#include <sys/pci.h>
#include <sys/pit.h>
#include <device/display/fb.h>
#include <device/display/term.h>
#include <device/display/edid.h>
#include <device/display/gfx.h>
#include <device/keyboard/keyboard.h>
#include <device/storage/ata.h>
#include <proc/sched.h>
#include <proc/syscall.h>
#include <fs/vfs.h>
#include <fs/ramfs.h>
#include <fs/ttyfs.h>
#include <fs/pipefs.h>
```

---

```
#include <proc/elf.h>
Include dependency graph for kmain.c:
```



## Functions

- void **done** (void)
- \_Noreturn void **kcursor** (task\_id\_t tid)
- \_Noreturn void **kshell** (task\_id\_t tid)
- void **kmain** (void)

## Variables

- [addrspace\\_t](#) **kaddrspace**

### 5.41.1 Detailed Description

Entry function of HanOS kernel.

---

This function initializes various components of the operating system, such as the CPU, serial communication, logging, memory management, interrupt handling, ACPI, HPET, CMOS, APIC, PIT, keyboard, VFS, SMP, syscall, INITRD, and terminal.

It also sets up the background image, prints system information, and starts the kcursor task.

Finally, it executes the default shell application.

History:

- Feb 19, 2022 Added CLI task which supports some simple commands.
  - May 21, 2022 Changed boot protocol to limine with corresponding modifications.
  - Jul 13, 2024 Added SLAB-based memory allocator.
- 

## 5.42 kernel/mm/alloc.c File Reference

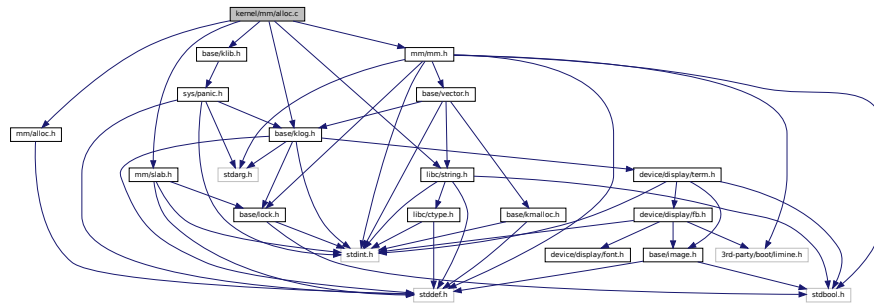
Implementation of memory allocation functions.

```
#include <mm/mm.h>
#include <mm/alloc.h>
#include <mm/slab.h>
#include <base/klib.h>
#include <base/klog.h>
#include <libc/string.h>
```

---



Include dependency graph for alloc.c:



## Macros

- `#define USE_POISON 0`
- `#define POISON_VALUE 0xdeadbeefbadc0ffell`
- `#define CACHE_COUNT 12`
- `#define CAPACITY_SIZE(cache)`

## Functions

- `void * alloc (uint64_t size)`
- `void free (void *ptr)`
- `void * realloc (void *ptr, uint64_t size)`
- `void alloc_init ()`

### 5.42.1 Detailed Description

Implementation of memory allocation functions.

This file contains the implementation of memory allocation functions for the HanOS kernel. The allocator uses a slab allocation mechanism to manage memory with different cache sizes. Functions for initializing the allocator, as well as allocating, reallocating, and freeing memory are provided. Each allocation includes metadata for capacity and size, and optionally a poison value for debugging purposes.

### 5.42.2 Macro Definition Documentation

#### 5.42.2.1 CAPACITY\_SIZE

```
#define CAPACITY_SIZE(
 cache)
```

**Value:**

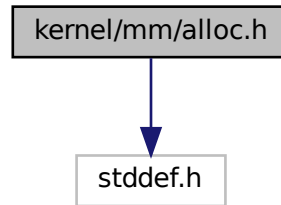
```
(cache->size - sizeof(uint64_t) * 2 \
 - USE_POISON * sizeof(uint64_t))
```

## 5.43 kernel/mm/alloc.h File Reference

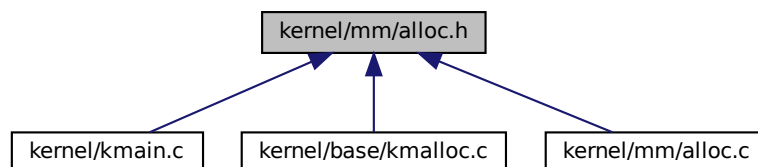
Memory allocation definitions.

```
#include <stddef.h>
```

Include dependency graph for alloc.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define ALLOC_MAX_SIZE 65536`

## Functions

- `void alloc_init ()`
- `void * alloc (uint64_t s)`
- `void * realloc (void *addr, uint64_t s)`
- `void free (void *addr)`

### 5.43.1 Detailed Description

Memory allocation definitions.

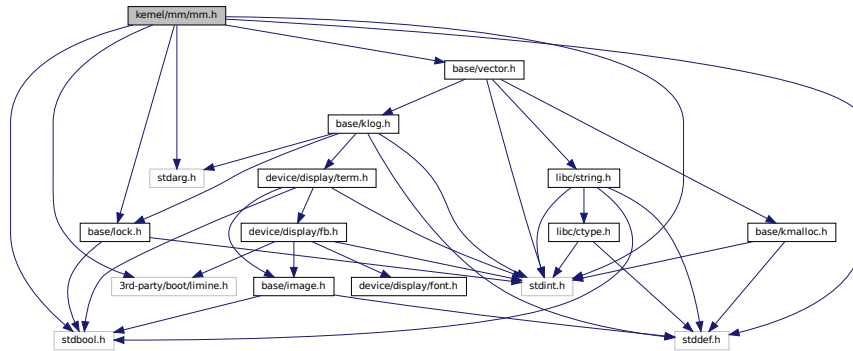
This file contains the function declarations and constants used for memory allocation within the HanOS kernel. It includes the initialization function for the allocator, and functions for allocating, reallocating, and freeing memory. It also defines the maximum size of memory that can be allocated.

## 5.44 kernel/mm/mm.h File Reference

Definition of memory management related data structures.

```
#include <stdarg.h>
#include <stddef.h>
#include <stdint.h>
```

```
#include <stdbool.h>
#include <3rd-party/boot/limine.h>
#include <base/lock.h>
#include <base/vector.h>
Include dependency graph for mm.h:
```



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [mem\\_info\\_t](#)
- struct [mem\\_map\\_t](#)
- struct [addrspace\\_t](#)

## Macros

- `#define PAGE_SIZE 4096`
- `#define BMP_PAGES_PER_BYTE 8`
- `#define MEM_VIRT_OFFSET 0xffff800000000000`
- `#define VIRT_TO_PHYS(a) (((uint64_t)(a)) - MEM_VIRT_OFFSET)`
- `#define PHYS_TO_VIRT(a) (((uint64_t)(a)) + MEM_VIRT_OFFSET)`
- `#define KERNEL_CODE_OFFSET 0xffffffff80200000`
- `#define NUM_PAGES(num) (((num) + PAGE_SIZE - 1) / PAGE_SIZE)`
- `#define PAGE_ALIGN_UP(num) (NUM_PAGES(num) * PAGE_SIZE)`
- `#define MAX_MEM_TOTAL_SIZE (16 * GB)`
- `#define MAX_MEM_BMP_SIZE (NUM_PAGES(MAX_MEM_TOTAL_SIZE) / BMP_PAGES_PER_BYTE)`
- `#define MAX_MEM_USABLE_SIZE (1 * GB)`
- `#define VMM_FLAG_PRESENT (1 << 0) /* P */`
- `#define VMM_FLAG_READWRITE (1 << 1) /* R/W */`
- `#define VMM_FLAG_USER (1 << 2) /* U/S */`
- `#define VMM_FLAG_WRITETHROUGH (1 << 3) /* PWT */`
- `#define VMM_FLAG_CACHE_DISABLE (1 << 4) /* PCD */`
- `#define VMM_FLAG_PAT (1 << 7) /* PAT */`
- `#define VMM_FLAGS_DEFAULT (VMM_FLAG_PRESENT | VMM_FLAG_READWRITE)`
- `#define VMM_FLAGS_MMIO`
- `#define VMM_FLAGS_USERMODE (VMM_FLAGS_DEFAULT | VMM_FLAG_USER)`
- `#define PAGE_TABLE_ENTRIES 512`

## Functions

- void **pmm\_init** (struct limine\_memmap\_response \*map, uint64\_t higher\_half)
- uint64\_t **pmm\_get** (uint64\_t numpages, uint64\_t baseaddr, const char \*func, int64\_t line)
- void **pmm\_free** (uint64\_t addr, uint64\_t numpages, const char \*func, int64\_t line)
- bool **pmm\_alloc** (uint64\_t addr, uint64\_t numpages)
- void **pmm\_dump\_usage** (void)
- uint64\_t **pmm\_get\_total\_memory** (void)
- void **vmm\_init** (struct limine\_memmap\_response \*map, struct limine\_kernel\_address\_response \*kernel)
- void **vmm\_map** (addrspace\_t \*addrspace, uint64\_t vaddr, uint64\_t paddr, uint64\_t np, uint64\_t flags)
- void **vmm\_unmap** (addrspace\_t \*addrspace, uint64\_t vaddr, uint64\_t np)
- uint64\_t **vmm\_get\_paddr** (addrspace\_t \*addrspace, uint64\_t vaddr)
- [addrspace\\_t](#) \* **create\_addrspace** (void)

## Variables

- [addrspace\\_t](#) kaddrspace

### 5.44.1 Detailed Description

Definition of memory management related data structures.

---

Memory management is a critical part of any operating system kernel. Providing a quick way for programs to allocate and free memory on a regular basis is a major responsibility of the kernel.

---

### 5.44.2 Macro Definition Documentation

#### 5.44.2.1 VMM\_FLAGS\_MMIO

```
#define VMM_FLAGS_MMIO
```

**Value:**

```
(VMM_FLAGS_DEFAULT | VMM_FLAG_CACHE_DISABLE \
 | VMM_FLAG_WRITETHROUGH)
```

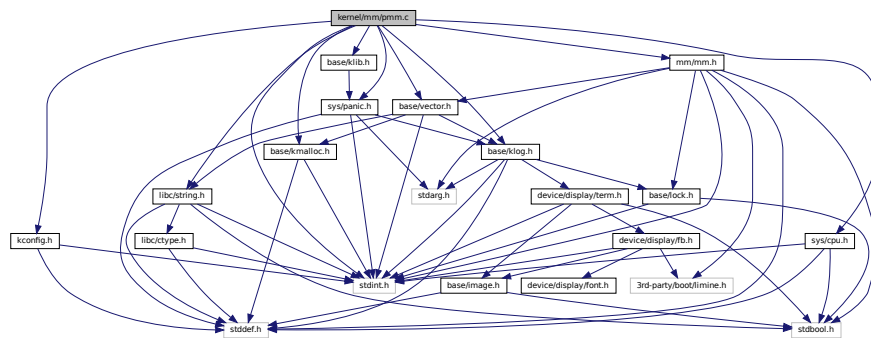
## 5.45 kernel/mm/pmm.c File Reference

Implementation of physical memory management functions.

```
#include <stdint.h>
#include <kconfig.h>
#include <libc/string.h>
#include <sys/cpu.h>
#include <sys/panic.h>
#include <mm/mm.h>
#include <base/klog.h>
#include <base/kmalloc.h>
#include <base/klib.h>
#include <base/vector.h>
```

---

Include dependency graph for pmm.c:



## Functions

- void **pmm\_free** (uint64\_t addr, uint64\_t numpages, const char \*func, int64\_t line)
- bool **pmm\_alloc** (uint64\_t addr, uint64\_t numpages)
- uint64\_t **pmm\_get** (uint64\_t numpages, uint64\_t baseaddr, const char \*func, int64\_t line)
- void **pmm\_init** (struct limine\_memmap\_response \*map, uint64\_t higher\_half)
- uint64\_t **pmm\_get\_total\_memory** (void)
- void **pmm\_dump\_usage** (void)

## Variables

- `mem_info_t kmem_info = {0}`

### 5.45.1 Detailed Description

Implementation of physical memory management functions.

Memory management is a critical part of any operating system kernel. Providing a quick way for programs to allocate and free memory on a regular basis is a major responsibility of the kernel.

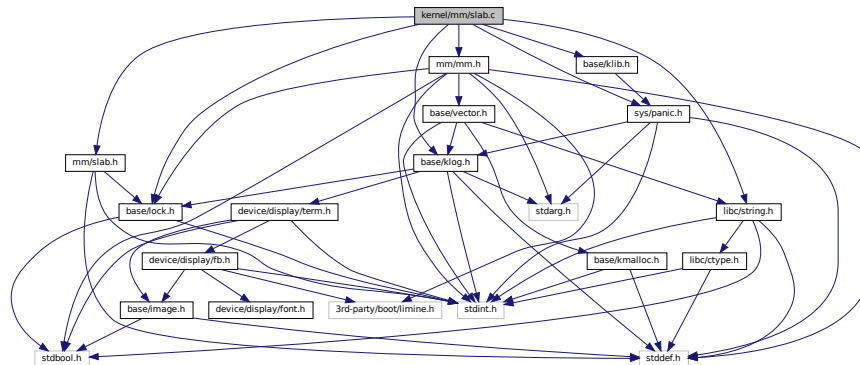
PMM: The method behind PMM is very simple. The memories with type - STIVALE2\_MMAP\_USABLE are divided into 4K-size pages. A bitmap array is used for indicated whether it is free or not. One bit for one page in bitmap array.

## 5.46 kernel/mm/slab.c File Reference

Implementation of slab allocator.

```
#include <base/klib.h>
#include <base/lock.h>
#include <base/klog.h>
#include <mm/slab.h>
#include <mm/mm.h>
#include <sys/panic.h>
#include <libc/string.h>
```

Include dependency graph for slab.c:



## Macros

- `#define SLAB_INDIRECT_CUTOFF 256`
- `#define SLAB_INDIRECT_COUNT 8`
- `#define SLAB_PAGE_OFFSET (PAGE_SIZE - sizeof(slab_t))`
- `#define SLAB_DATA_SIZE SLAB_PAGE_OFFSET`
- `#define SLAB_INDIRECT_PTR_COUNT (SLAB_DATA_SIZE / sizeof(void **))`
- `#define GET_SLAB(x)`
- `#define SLAB_DEBUG 0`

## Functions

- `void * slab_allocate (scache_t *cache)`
- `void slab_free (scache_t *cache, void *addr)`
- `scache_t * slab_newcache (uint64_t size, uint64_t alignment, void(*ctor)(scache_t *, void *), void(*dtor)(scache_t *, void *))`
- `void slab_freecache (scache_t *cache)`

### 5.46.1 Detailed Description

Implementation of slab allocator.

This file contains the implementation of a slab allocator for managing memory allocation in the HanOS kernel. It defines functions for creating and managing slab caches, allocating and freeing memory from these caches, and initializing and destructing objects within slabs. The slab allocator optimizes memory usage by dividing memory into small, fixed-size chunks and reusing them efficiently.

### 5.46.2 Macro Definition Documentation

#### 5.46.2.1 GET\_SLAB

```
#define GET_SLAB(
 x)
```

Value:

```
(slab_t *) (ROUND_DOWN((uintptr_t)x, PAGE_SIZE) \
+ SLAB_PAGE_OFFSET)
```

## 5.47 kernel/mm/slab.h File Reference

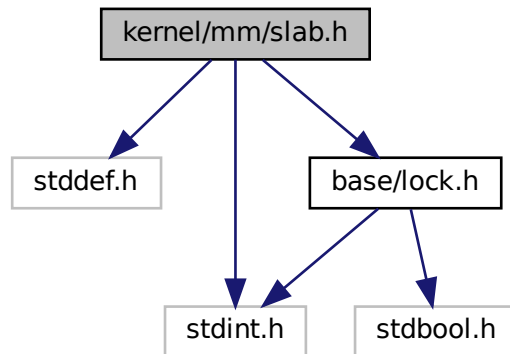
Slab allocator definitions.

```
#include <stddef.h>
```

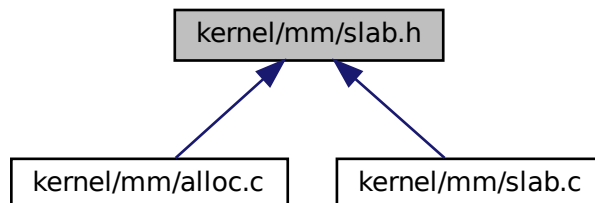
```
#include <stdint.h>
```

```
#include <base/lock.h>
```

Include dependency graph for slab.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- struct `slab_t`
- struct `scache_t`

### Typedefs

- typedef struct `slab_t` `slab_t`
- typedef struct `scache_t` `scache_t`

### Functions

- void \* `slab_allocate` (`scache_t` \*cache)





## Variables

- `mem_info_t` `kmem_info`
- `addrspace_t` `kaddrspace` = {0}

### 5.48.1 Detailed Description

### Implementation of virtual memory management functions.

High Half Kernel: To setup a higher half kernel, you have to map your kernel to the appropriate virtual address. Without a boot loader help, you'll need a small trampoline code which runs in lower half, sets up higher half paging and jumps.

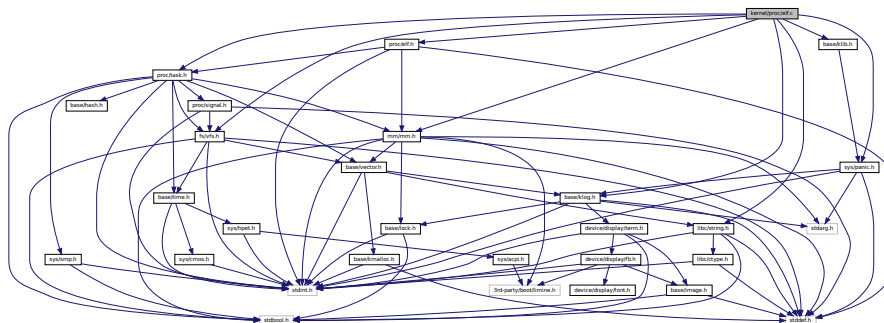
If page protection is not enabled, virtual address is equal with physical address. The highest bit of CR0 indicates whether paging is enabled or not: `mov cr0,80000000` can enable paging.

## 5.49 kernel/proc/elf.c File Reference

### Implementation of ELF file loading and handling functions.

```
#include <libc/string.h>
#include <proc/elf.h>
#include <proc/task.h>
#include <base/klib.h>
#include <base/klog.h>
#include <fs/vfs.h>
#include <mm/mm.h>
#include <sys/panic.h>
```

Include dependency graph for elf.c:



## Macros

- `#define RTDL_ADDR 0x40000000`
- `#define IS_TEXT(p) (p.flags & PF_X)`
- `#define IS_DATA(p) (p.flags & PF_W)`
- `#define IS_BSS(p) (p.filesz < p.memsz)`

## Functions

- int **elf\_find\_symbol\_table** (elf\_hdr\_t \*hdr, elf\_shdr\_t \*shdr)
- void \* **elf\_find\_sym** (const char \*name, elf\_shdr\_t \*shdr, elf\_shdr\_t \*shdr\_sym, const char \*src, char \*dst)
- int64\_t **elf\_load** (task\_t \*task, const char \*path\_name, uint64\_t \*entry, auxval\_t \*aux)

## Variables

- lock t sched lock



- #define **ET\_EXEC** 2
- #define **ET\_SHARED** 3
- #define **ET\_CORE** 4
- #define **PT\_LOAD** 0x00000001 /\* Loadable program segment \*/
- #define **PT\_INTERP** 0x00000003 /\* Program interpreter \*/
- #define **PT\_PHDR** 0x00000006 /\* Entry for header table itself \*/
- #define **ABI\_SYSV** 0x00
- #define **ARCH\_X86\_64** 0x3e
- #define **BITS\_LE** 0x01
- #define **EI\_CLASS** 0
- #define **EI\_DATA** 1
- #define **EI\_VERSION** 2
- #define **EI\_OSABI** 3
- #define **EI\_ABIVERSION** 4
- #define **PF\_X** 1
- #define **PF\_W** 2
- #define **PF\_R** 4
- #define **SHT\_NULL** 0 /\* sh\_type \*/
- #define **SHT\_PROGBITS** 1
- #define **SHT\_SYMTAB** 2
- #define **SHT\_STRTAB** 3
- #define **SHT\_RELA** 4
- #define **SHT\_HASH** 5
- #define **SHT\_DYNAMIC** 6
- #define **SHT\_NOTE** 7
- #define **SHT\_NOBITS** 8
- #define **SHT\_REL** 9
- #define **SHT\_SHLIB** 10
- #define **SHT\_DYNSYM** 11
- #define **SHT\_UNKNOWN12** 12
- #define **SHT\_UNKNOWN13** 13
- #define **SHT\_INIT\_ARRAY** 14
- #define **SHT\_FINI\_ARRAY** 15
- #define **SHT\_PREINIT\_ARRAY** 16
- #define **SHT\_GROUP** 17
- #define **SHT\_SYMTAB\_SHNDX** 18
- #define **SHT\_NUM** 19

## Typedefs

- typedef struct [elf\\_shdr\\_t](#) **elf\_shdr\_t**

## Functions

- [int64\\_t elf\\_load](#) ([task\\_t](#) \*task, const char \*path\_name, [uint64\\_t](#) \*entry, [auxval\\_t](#) \*aux)

### 5.50.1 Detailed Description

Definitions for ELF file format structures and constants.

---

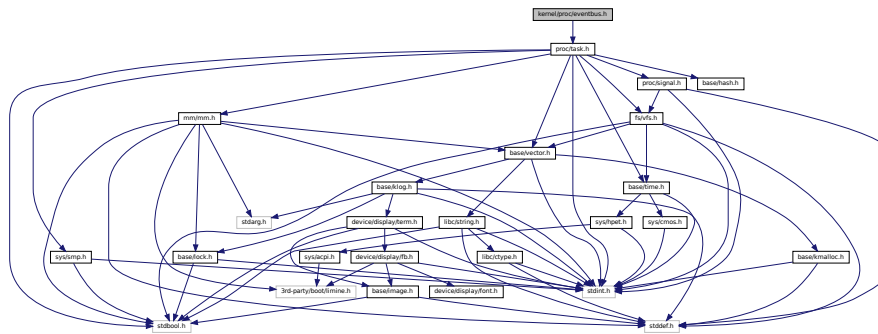
This file contains definitions for the structures and constants used to interpret and manipulate ELF (Executable and Linkable Format) files within the HanOS Kernel. It includes definitions for ELF headers, program headers, section headers, and other related data structures.

---

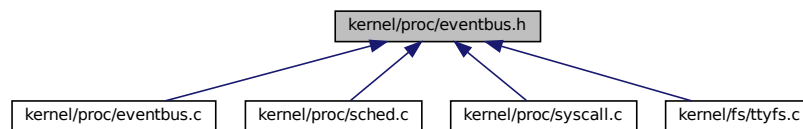


```
#include <proc/task.h>
```

Include dependency graph for eventbus.h:



This graph shows which files directly or indirectly include this file:



## Functions

- bool **eb\_publish** (task\_id\_t tid, event\_type\_t type, event\_para\_t para)
- bool **eb\_subscribe** (task\_id\_t tid, event\_type\_t type, event\_para\_t \*para)
- bool **eb\_dispatch** (void)

### 5.52.1 Detailed Description

### Definitions for the event bus system.

This file contains the definitions for the event bus system used in the HanOS kernel. The event bus allows tasks to publish and subscribe to events, enabling inter-task communication and event-driven execution. The functions provided facilitate publishing events, subscribing to events, and dispatching events to their respective handlers.

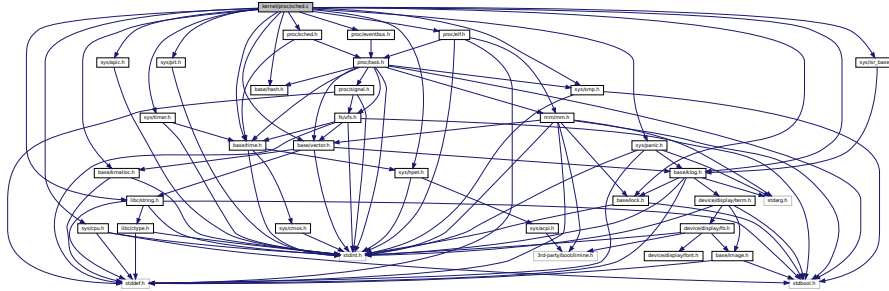
### 5.53 kernel/proc/sched.c File Reference

Maintain task list and schedule tasks according to system clock ticks.

```
#include <libc/string.h>
#include <base/klog.h>
#include <base/lock.h>
#include <base/time.h>
#include <base/kmalloc.h>
#include <base/vector.h>
#include <base/hash.h>
#include <proc/sched.h>
#include <proc/elf.h>
#include <proc/eventbus.h>
#include <sys/smp.h>
```

```
#include <sys/timer.h>
#include <sys/apic.h>
#include <sys/hpet.h>
#include <sys/pit.h>
#include <sys/isr_base.h>
#include <sys/panic.h>
#include <sys/cpu.h>
```

Include dependency graph for sched.c:



## Macros

- `#define TIMESLICE_DEFAULT MILLIS_TO_NANOS(1)`

## Functions

- `vec_new_static(task_t *, tasks_active)`
- `void enter_context_switch(void *v)`
- `void exit_context_switch(task_t *next, uint64_t cr3val)`
- `void force_context_switch(void)`
- `void fork_context_switch(void)`
- `void sched_debug(bool showlog)`
- `_Noreturn void task_idle_proc(task_id_t tid)`
- `void do_context_switch(void *stack, int64_t mode)`
- `task_id_t sched_get_tid()`
- `task_id_t sched_fork(void)`
- `void sched_sleep(time_t millis)`
- `task_status_t sched_get_task_status(task_id_t tid)`
- `void sched_exit(int64_t status)`
- `bool sched_resume_event(event_t event)`
- `event_t sched_wait_event(event_t event)`
- `task_t * sched_get_current_task()`
- `uint64_t sched_get_ticks()`
- `void sched_init(const char *name, uint16_t cpu_id)`
- `uint16_t sched_get_cpu_num()`
- `task_t * sched_new(const char *name, void(*entry)(task_id_t), bool usermode)`
- `void sched_add(task_t *t)`
- `task_t * sched_execve(const char *path, const char *argv[], const char *envp[], const char *cwd)`

## Variables

- `lock_t sched_lock = lock_new()`



- void **sched\_exit** (int64\_t status)
- [event\\_t](#) **sched\_wait\_event** ([event\\_t](#) event)
- bool **sched\_resume\_event** ([event\\_t](#) event)
- [task\\_t](#) \* **sched\_get\_current\_task** (void)
- uint16\_t **sched\_get\_cpu\_num** (void)
- uint64\_t **sched\_get\_ticks** (void)
- task\_id\_t **sched\_get\_tid** (void)
- task\_status\_t **sched\_get\_task\_status** (task\_id\_t tid)
- [task\\_t](#) \* **sched\_execve** (const char \*path, const char \*argv[], const char \*envp[], const char \*cwd)

### 5.54.1 Detailed Description

Definition of scheduling related functions.

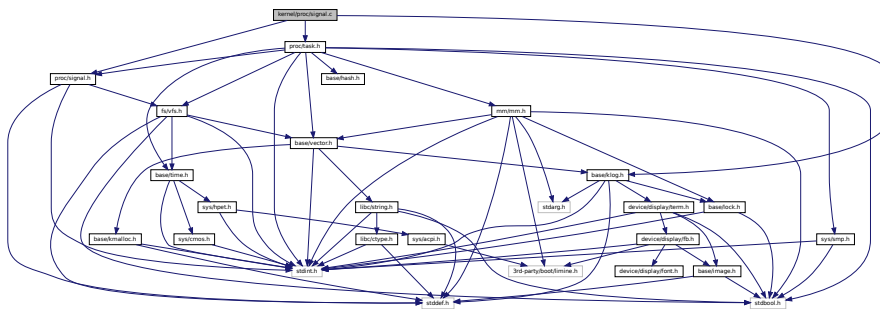
This header file provides the definitions for scheduling-related functions used in the HanOS kernel. It includes declarations for context switching, scheduling algorithms, task management, and various scheduler utilities.

## 5.55 kernel/proc/signal.c File Reference

Implementation of signal handling functions.

```
#include <base/klog.h>
#include <proc/signal.h>
#include <proc/task.h>
```

Include dependency graph for signal.c:



### Functions

- void **signal\_action** ([task\\_t](#) \*t, int64\_t signal, [sigaction\\_t](#) \*new, [sigaction\\_t](#) \*old)
- void **signal\_changemask** ([task\\_t](#) \*t, int64\_t how, [sigset\\_t](#) \*new, [sigset\\_t](#) \*old)

### Variables

- int32\_t **signal\_defaultactions** [NSIG]

### 5.55.1 Detailed Description

Implementation of signal handling functions.

This file contains the implementation of functions required to handle signals within the HanOS kernel. It includes functions to set signal actions, change signal masks, and manage the default actions for various signals. The signal handling mechanism allows tasks to handle asynchronous events such as interrupts, exceptions, and inter-process communication.





- #define **POLL\_OUT** 2
- #define **POLL\_MSG** 3
- #define **POLL\_ERR** 4
- #define **POLL\_PRI** 5
- #define **POLL\_HUP** 6
- #define **SIGHUP** 1
- #define **SIGQUIT** 3
- #define **SIGTRAP** 5
- #define **SIGABRT** 6
- #define **SIGIOT** SIGABRT
- #define **SIGBUS** 7
- #define **SIGKILL** 9
- #define **SIGUSR1** 10
- #define **SIGUSR2** 12
- #define **SIGPIPE** 13
- #define **SIGALRM** 14
- #define **SIGSTKFLT** 16
- #define **SIGCHLD** 17
- #define **SIGCONT** 18
- #define **SIGSTOP** 19
- #define **SIGTSTP** 20
- #define **SIGTTIN** 21
- #define **SIGTTOU** 22
- #define **SIGURG** 23
- #define **SIGXCPU** 24
- #define **SIGXFSZ** 25
- #define **SIGVTALRM** 26
- #define **SIGWINCH** 28
- #define **SIGPOLL** 29
- #define **SIGSYS** 31
- #define **SIGUNUSED** SIGSYS
- #define **SIGCANCEL** 32
- #define **SIGABRT** 6
- #define **SIGFPE** 8
- #define **SIGILL** 4
- #define **SIGINT** 2
- #define **SIGSEGV** 11
- #define **SIGTERM** 15
- #define **SIGPROF** 27
- #define **SIGIO** 29 /\* Same with SIGPOLL? \*/
- #define **SIGPWR** 30
- #define **SIGRTMIN** 35
- #define **SIGRTMAX** 64
- #define **SIG\_BLOCK** 0
- #define **SIG\_UNBLOCK** 1
- #define **SIG\_SETMASK** 2
- #define **SIG\_ACTION\_TERM** 0
- #define **SIG\_ACTION\_IGN** 1
- #define **SIG\_ACTION\_CORE** 2
- #define **SIG\_ACTION\_STOP** 3
- #define **SIG\_ACTION\_CONT** 4

## Typedefs

- typedef long **clock\_t**
- typedef struct [task\\_t](#) **task\_t**

## Functions

- void **signal\_action** ([task\\_t](#) \*t, int64\_t signal, [sigaction\\_t](#) \*new, [sigaction\\_t](#) \*old)
- void **signal\_changemask** ([task\\_t](#) \*t, int64\_t how, [sigset\\_t](#) \*new, [sigset\\_t](#) \*old)

### 5.56.1 Detailed Description

Definitions for signal handling.

---

This file contains the definitions and structures used for signal handling within the HanOS kernel. It includes definitions for signal sets, signal actions, and various signal-related constants. The functions provided allow for setting signal actions, changing signal masks, and managing signals within tasks.

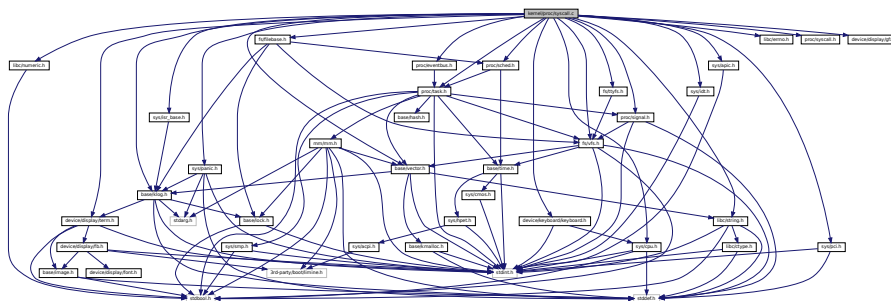
---

## 5.57 kernel/proc/syscall.c File Reference

Implementation of system calls.

```
#include <libc/string.h>
#include <libc/errno.h>
#include <libc/numeric.h>
#include <sys/cpu.h>
#include <sys/idt.h>
#include <sys/apic.h>
#include <sys/panic.h>
#include <sys/pci.h>
#include <sys/isr_base.h>
#include <base/klog.h>
#include <base/vector.h>
#include <proc/task.h>
#include <proc/sched.h>
#include <proc/syscall.h>
#include <proc/eventbus.h>
#include <proc/signal.h>
#include <fs/filebase.h>
#include <fs/vfs.h>
#include <fs/ttyfs.h>
#include <device/keyboard/keyboard.h>
#include <device/display/term.h>
#include <device/display/gfx.h>
```

Include dependency graph for syscall.c:



## Macros

- #define **MMAP\_ANON\_BASE** 0x800000000000
-

## Typedefs

- typedef int64\_t(\* **syscall\_ptr\_t**) (void)

## Functions

- int64\_t **syscall\_handler** ()
- int64\_t **k\_print\_log** ()
- int64\_t **k\_not\_implemented** ()
- int64\_t **k\_debug\_log** (char \*message)
- int64\_t **k\_sigprocmask** (int64\_t how, sigset\_t \*set, sigset\_t \*oldset)
- int64\_t **k\_sigaction** (int64\_t s, sigaction\_t \*new, sigaction\_t \*old)
- int64\_t **k\_runcmd** (char \*cmd)
- int64\_t **k\_getentropy** (void \*buffer, uint64\_t length)
- uint64\_t **k\_vm\_map** (uint64\_t \*hint, uint64\_t length, uint64\_t prot, uint64\_t flags, uint64\_t fd, uint64\_t offset)
- int64\_t **k\_vm\_unmap** (void \*ptr, uint64\_t size)
- int **get\_full\_path** (int64\_t dirfh, const char \*path, char \*full\_path, uint64\_t full\_path\_size)
- int64\_t **k\_openat** (int64\_t dirfh, char \*path, int64\_t flags, int64\_t mode)
- int64\_t **k\_chmod** (char \*path, int64\_t flags)
- int64\_t **k\_unlink** (char \*path)
- int64\_t **k\_seek** (int64\_t fh, int64\_t offset, int64\_t whence)
- int64\_t **k\_close** (int64\_t fh)
- int64\_t **k\_read** (int64\_t fh, void \*buf, uint64\_t count)
- int64\_t **k\_write** (int64\_t fh, const void \*buf, uint64\_t count)
- void **k\_set\_fs\_base** (uint64\_t val)
- int64\_t **k\_ioctl** (int64\_t fd, int64\_t request, int64\_t arg)
- int64\_t **k\_fstatat** (int64\_t dirfh, const char \*path, int64\_t statbuf, int64\_t flags)
- int64\_t **k\_fstat** (int64\_t handle, int64\_t statbuf)
- int64\_t **k\_faccessat** (int64\_t dirfh, const char \*path, uint64\_t mode, uint64\_t flags)
- int64\_t **k\_getpid** ()
- int64\_t **k\_chdir** (char \*dir)
- int64\_t **k\_readdir** (int64\_t handle, uint64\_t buff)
- int64\_t **k\_meminfo** ()
- int64\_t **k\_pipe** (int32\_t \*fh, uint32\_t flags)
- int64\_t **k\_fork** ()
- int64\_t **k\_getppid** ()
- int64\_t **k\_fcntl** (int64\_t fd, int64\_t request, int64\_t arg)
- int64\_t **k\_waitpid** (int64\_t pid, int32\_t \*status, int32\_t flags)
- void **k\_exit** (int64\_t status)
- int **k\_getcwd** (char \*buffer, uint64\_t size)
- int **k\_getrusage** (int64\_t who, uint64\_t usage)
- int64\_t **k\_execve** (const char \*path, const char \*argv[], const char \*envp[])
- int **k\_getclock** (void \*, int64\_t which, vfs\_timespec\_t \*out)
- int64\_t **k\_readlink** (int64\_t dirfh, const char \*path, void \*buffer, uint64\_t max\_size)
- void **k\_uname** (void)
- int64\_t **k\_dup3** (int64\_t fh, int64\_t newfh, int64\_t flags)
- int64\_t **k\_futex\_wait** (int64\_t \*ptr, vfs\_timespec\_t \*tv, int64\_t expected)
- int64\_t **k\_futex\_wake** (int64\_t \*ptr)
- void **syscall\_init** (void)

## Variables

- lock\_t **vfs\_lock**
- lock\_t **sched\_lock**
- syscall\_ptr\_t **syscall\_funcs** []

### 5.57.1 Detailed Description

#### Implementation of system calls.

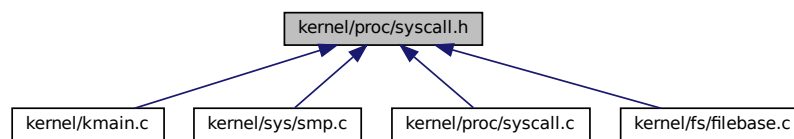
This file contains the implementation of system calls within the HanOS kernel. System calls provide an interface for user space applications to request services from the kernel. This file defines various system call handlers, including those for file operations, process management, memory management, and signal handling.

Ref: Page 2994 in Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4

## 5.58 kernel/proc/syscall.h File Reference

Definitions and declarations for system calls.

This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [timeval\\_t](#)
- struct [rusage\\_t](#)

### Macros

- #define **SYSCALL\_DEBUGLOG** 0
- #define **SYSCALL\_MMAP** 1
- #define **SYSCALL\_OPENAT** 2
- #define **SYSCALL\_READ** 3
- #define **SYSCALL\_WRITE** 4
- #define **SYSCALL\_SEEK** 5
- #define **SYSCALL\_CLOSE** 6
- #define **SYSCALL\_SET\_FS\_BASE** 7
- #define **SYSCALL\_IOCTL** 8
- #define **SYSCALL\_GETPID** 9
- #define **SYSCALL\_CHDIR** 10
- #define **SYSCALL\_MKDIRAT** 11
- #define **SYSCALL\_SOCKET** 12
- #define **SYSCALL\_BIND** 13
- #define **SYSCALL\_FORK** 14
- #define **SYSCALL\_EXECVE** 15
- #define **SYSCALL\_FACCESSAT** 16
- #define **SYSCALL\_FSTATAT** 17
- #define **SYSCALL\_FSTAT** 18
- #define **SYSCALL\_GETPPID** 19
- #define **SYSCALL\_FCNTL** 20
- #define **SYSCALL\_DUP3** 21
- #define **SYSCALL\_WAITPID** 22
- #define **SYSCALL\_EXIT** 23

- #define **SYSCALL\_READDIR** 24
- #define **SYSCALL\_MUNMAP** 25
- #define **SYSCALL\_GETCWD** 26
- #define **SYSCALL\_GETCLOCK** 27
- #define **SYSCALL\_READLINK** 28
- #define **SYSCALL\_GETRUSAGE** 29
- #define **SYSCALL\_GETRLIMIT** 30
- #define **SYSCALL\_UNAME** 31
- #define **SYSCALL\_FUTEX\_WAIT** 32
- #define **SYSCALL\_FUTEX\_WAKE** 33
- #define **SYSCALL\_MEMINFO** 34
- #define **SYSCALL\_PIPE** 35
- #define **SYSCALL\_UNLINK** 36
- #define **SYSCALL\_CHMOD** 39
- #define **SYSCALL\_RUNCMD** 40
- #define **SYSCALL\_GETENTROPY** 41
- #define **SYSCALL\_SIGPROCMASK** 42
- #define **SYSCALL\_SIGACTION** 43
- #define **STDIN** 0
- #define **STDOUT** 1
- #define **STDERR** 2
- #define **MAP\_PRIVATE** 0x01
- #define **MAP\_SHARED** 0x02
- #define **MAP\_FIXED** 0x04
- #define **MAP\_ANONYMOUS** 0x08
- #define **O\_ACCMODE** 0x0007
- #define **O\_EXEC** 1
- #define **O\_RDONLY** 2
- #define **O\_RDWR** 3
- #define **O\_SEARCH** 4
- #define **O\_WRONLY** 5
- #define **O\_APPEND** 0x0008
- #define **O\_CREAT** 0x0010
- #define **O\_DIRECTORY** 0x0020
- #define **O\_EXCL** 0x0040
- #define **O\_NOCTTY** 0x0080
- #define **O\_NOFOLLOW** 0x0100
- #define **O\_TRUNC** 0x0200
- #define **O\_NONBLOCK** 0x0400
- #define **O\_DSYNC** 0x0800
- #define **O\_RSYNC** 0x1000
- #define **O\_SYNC** 0x2000
- #define **O\_CLOEXEC** 0x4000
- #define **O\_PATH** 0x8000
- #define **X86\_EFLAGS\_CF** 0x00000001 /\* Carry Flag \*/
- #define **X86\_EFLAGS\_PF** 0x00000004 /\* Parity Flag \*/
- #define **X86\_EFLAGS\_AF** 0x00000010 /\* Auxillary carry Flag \*/
- #define **X86\_EFLAGS\_ZF** 0x00000040 /\* Zero Flag \*/
- #define **X86\_EFLAGS\_SF** 0x00000080 /\* Sign Flag \*/
- #define **X86\_EFLAGS\_TF** 0x00000100 /\* Trap Flag \*/
- #define **X86\_EFLAGS\_IF** 0x00000200 /\* Interrupt Flag \*/
- #define **X86\_EFLAGS\_DF** 0x00000400 /\* Direction Flag \*/
- #define **X86\_EFLAGS\_OF** 0x00000800 /\* Overflow Flag \*/
- #define **X86\_EFLAGS\_IOPL** 0x00003000 /\* IOPL mask \*/
- #define **X86\_EFLAGS\_NT** 0x00004000 /\* Nested Task \*/

- `#define X86_EFLAGS_RF 0x00010000 /* Resume Flag */`
- `#define X86_EFLAGS_VM 0x00020000 /* Virtual Mode */`
- `#define X86_EFLAGS_AC 0x00040000 /* Alignment Check */`
- `#define X86_EFLAGS_VIF 0x00080000 /* Virtual Interrupt Flag */`
- `#define X86_EFLAGS_VIP 0x00100000 /* Virtual Interrupt Pending */`
- `#define X86_EFLAGS_ID 0x00200000 /* CPUID detection flag */`
- `#define CLOCK_REALTIME 0`
- `#define CLOCK_MONOTONIC 1`
- `#define CLOCK_PROCESS_CPUTIME_ID 2`
- `#define CLOCK_THREAD_CPUTIME_ID 3`
- `#define CLOCK_MONOTONIC_RAW 4`
- `#define CLOCK_REALTIME_COARSE 5`
- `#define CLOCK_MONOTONIC_COARSE 6`
- `#define CLOCK_BOOTTIME 7`
- `#define R_OK 4 /* Test mode for read permission */`
- `#define W_OK 2 /* Test mode for write permission */`
- `#define X_OK 1 /* Test mode for execute permission */`
- `#define F_OK 0 /* Test mode for existence */`
- `#define AT_SYMLINK_FOLLOW 2`
- `#define AT_EACCESS 4`
- `#define PROT_NONE 0x00`
- `#define PROT_READ 0x01`
- `#define PROT_WRITE 0x02`
- `#define PROT_EXEC 0x04`

## Functions

- `int get_full_path (int64_t dirfh, const char *path, char *full_path, uint64_t full_path_size)`
- `void syscall_init (void)`
- `int64_t syscall_entry (uint64_t id,...)`

### 5.58.1 Detailed Description

Definitions and declarations for system calls.

---

This file contains the definitions and declarations for system calls within the HanOS kernel. System calls provide an interface for user space applications to request services from the operating system kernel. The file includes syscall numbers, macros, and function prototypes necessary for implementing and handling system calls.

---

## 5.59 kernel/proc/task.c File Reference

Implementation of task related functions.

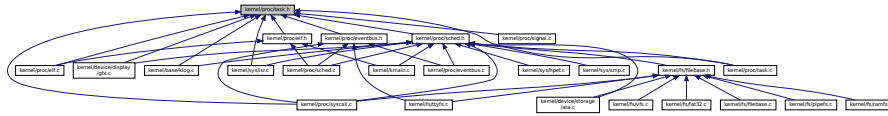
```
#include <stddef.h>
#include <libc/string.h>
#include <proc/task.h>
#include <proc/sched.h>
#include <base/kmalloc.h>
#include <base/klog.h>
#include <base/lock.h>
#include <sys/cpu.h>
#include <sys/hpet.h>
#include <sys/apic.h>
```

---





This graph shows which files directly or indirectly include this file:



## Data Structures

- struct `auxval_t`
- struct `task_regs_t`
- struct `event_t`
- struct `file_dup_t`
- struct `task_t`

## Macros

- `#define DEFAULT_KMODE_CODE 0b00101000 /* 0x28 */`
- `#define DEFAULT_KMODE_DATA 0b00110000 /* 0x30 */`
- `#define DEFAULT_UMODE_DATA 0b00111011 /* 0x3b */`
- `#define DEFAULT_UMODE_CODE 0b01000011 /* 0x43 */`
- `#define DEFAULT_RFLAGS 0b0000001000000010 /* 0x0202 */`
- `#define TID_MAX UINT64_MAX`
- `#define TID_NONE 0`

## Typedefs

- typedef uint64\_t **task\_id\_t**
- typedef uint8\_t **task\_priority\_t**
- typedef uint64\_t **event\_para\_t**
- typedef struct **task** **task\_t**

## Enumerations

- enum **task\_mode\_t** { TASK\_KERNEL\_MODE, TASK\_USER\_MODE }
- enum **task\_status\_t** { TASK\_READY, TASK\_RUNNING, TASK\_SLEEPING, TASK\_DYING, TASK\_DEAD, TASK\_UNKNOWN }
- enum **event\_type\_t** { EVENT\_UNDEFINED = 1, EVENT\_KEY\_PRESSED }

## Functions

- `task_t * task_make` (const char \*name, void(\*entry)(task\_id\_t), task\_priority\_t priority, task\_mode\_t mode, addrspace\_t \*pas)
- `task_t * task_fork` (task\_t \*tp)
- void `task_debug` (task\_t \*t, bool force)
- void `task_free` (task\_t \*t)

### 5.60.1 Detailed Description

### Definition of task related functions.

Create and return task data structure which contains registers and other task related information.

Below is interrupt related information.

When the CPU calls the interrupt handlers, it changes the value in the RSP register to the value specified in the IST, and if there is none, the stack stays the same. Onto the new stack, the CPU pushes these values in this order:

```
SS:RSP (original RSP) -> RFLAGS -> CS -> RIP
```

CS is padded to form a quadword. RIP stores code address.

If the interrupt is called from a different ring, SS is set to 0, indicating a null selector. The CPU will modify the RFLAGS register, setting the TF, NT, and RF bits to 0. If the gate type is a trap gate, the CPU will clear the interrupt flag.

The CPU will load the segment-selector value from the associated IDT descriptor into CS, and check to ensure that CS is a valid code segment selector.

## 5.61 kernel/proc/wait.h File Reference

Definitions for process wait options and macros.

### Macros

- `#define WCONTINUED 1`
- `#define WNOHANG 2`
- `#define WUNTRACED 4`
- `#define WEXITED 8`
- `#define WNOWAIT 16`
- `#define WSTOPPED 32`
- `#define WCOREFLAG 0x80`
- `#define WEXITSTATUS(x) ((x) & 0x000000FF)`
- `#define WIFCONTINUED(x) ((x) & 0x00000100)`
- `#define WIFEXITED(x) ((x) & 0x00000200)`
- `#define WIFSIGNALED(x) ((x) & 0x00000400)`
- `#define WIFSTOPPED(x) ((x) & 0x00000800)`
- `#define WSTOPSIG(x) (((x) & 0x00FF0000) >> 16)`
- `#define WTERMSIG(x) (((x) & 0xFF000000) >> 24)`
- `#define WCOREDUMP(x) ((x) & WCOREFLAG)`

### 5.61.1 Detailed Description

Definitions for process wait options and macros.

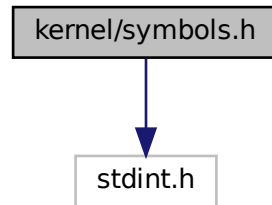
This file contains definitions and macros used for process waiting options within the HanOS kernel. The macros provided are used to determine the status of a process that has been waited on, including whether it has exited, was stopped, continued, or terminated by a signal. These definitions are essential for handling process synchronization and management in the kernel.

## 5.62 kernel/symbols.h File Reference

Definition of symbol related data structures.

```
#include <stdint.h>
```

Include dependency graph for symbols.h:



## Data Structures

- struct [symbol\\_t](#)

## Variables

- const [symbol\\_t](#) \_kernel\_syntab []

### 5.62.1 Detailed Description

Definition of symbol related data structures.

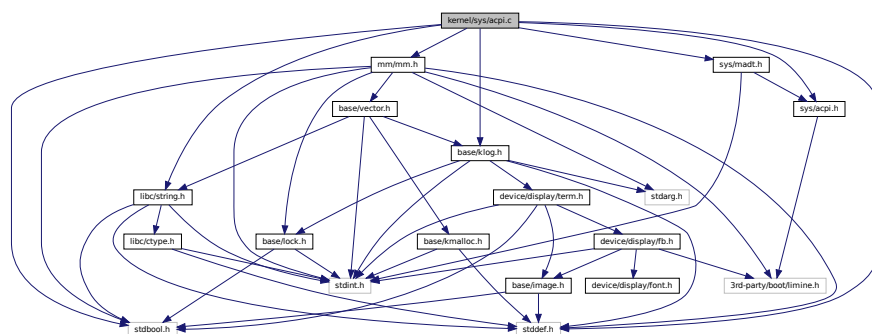
Symbols are used for backtrace when kernel is crashed. It can help to provide context information for debugging.

## 5.63 kernel/sys/acpi.c File Reference

Implementation of ACPI (Advanced Configuration and Power Management Interface) functions.

```
#include <stdbool.h>
#include <stddef.h>
#include <libc/string.h>
#include <sys/acpi.h>
#include <sys/madt.h>
#include <mm/mm.h>
#include <base/klog.h>
```

Include dependency graph for acpi.c:



## Functions

- [acpi\\_sdt\\_t](#) \* **acpi\_get\_sdt** (const char \*sign)
- void **acpi\_init** (struct limine\_rsdp\_response \*rsdp\_info)

### 5.63.1 Detailed Description

Implementation of ACPI (Advanced Configuration and Power Management Interface) functions.

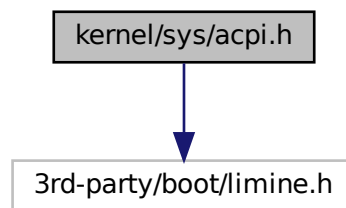
This module includes implementation of RSDT/XSDT initialization and "MADT/HPET" parsing.

## 5.64 kernel/sys/acpi.h File Reference

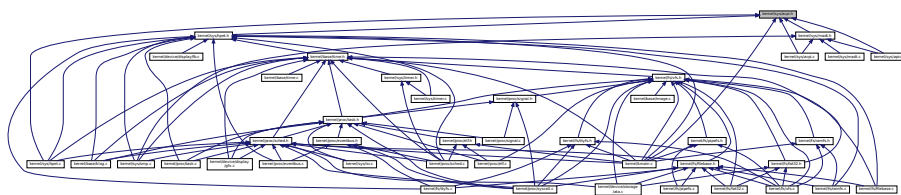
Definition of ACPI (Advanced Configuration and Power Management Interface) related data structures.

```
#include <3rd-party/boot/limine.h>
```

Include dependency graph for acpi.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [rsdp\\_t](#)
- struct [acpi\\_sdt\\_hdr\\_t](#)
- struct [acpi\\_sdt\\_t](#)
- struct [acpi\\_gas\\_t](#)

## Functions

- void **acpi\_init** (struct limine\_rsdp\_response \*)
- [acpi\\_sdt\\_t](#) \* **acpi\_get\_sdt** (const char \*sign)

### 5.64.1 Detailed Description

## Definition of ACPI (Advanced Configuration and Power Management Interface) related data structures.

ACPI (Advanced Configuration and Power Interface) is a Power Management and configuration standard for the PC, developed by Intel, Microsoft and Toshiba. ACPI allows the operating system to control the amount of power each device is given (allowing it to put certain devices on standby or power-off for example). It is also used to control and/or check thermal zones (temperature sensors, fan speeds, etc), battery levels, PCI IRQ routing, CPUs, NUMA domains and many other things.

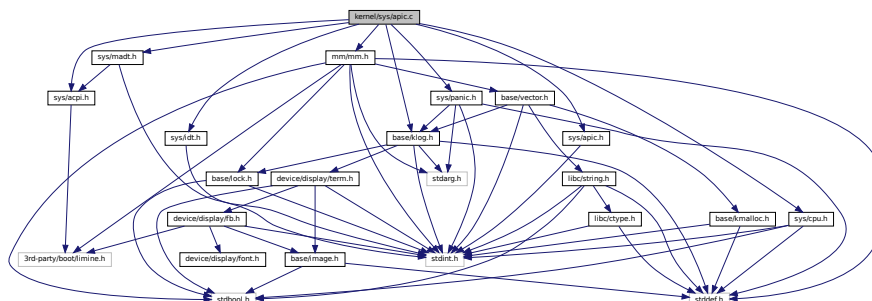
There are 2 main parts to ACPI. The first part is the tables used by the OS for configuration during boot (these include things like how many CPUs, APIC details, NUMA memory ranges, etc). The second part is the run time ACPI environment, which consists of AML code (a platform independent OOP language that comes from the BIOS and devices) and the ACPI SMM (System Management Mode) code.

## 5.65 kernel/sys/apic.c File Reference

Implementation of APIC (Advanced Programmable Interrupt Controller) functions.

```
#include <sys/acpi.h>
#include <sys/apic.h>
#include <sys/madt.h>
#include <sys/cpu.h>
#include <sys/idt.h>
#include <sys/panic.h>
#include <mm/mm.h>
#include <base/klog.h>
```

Include dependency graph for apic.c:



## Macros

- **#define IA32\_APIC\_BASE\_MSR 0x1B**
- **#define IA32\_APIC\_BASE\_MSR\_BSP 0x100 /\* Processor is a BSP \*/**
- **#define IA32\_APIC\_BASE\_MSR\_X2APIC 0x400**
- **#define IA32\_APIC\_BASE\_MSR\_ENABLE 0x800**

## Functions

- uint32\_t **apic\_read\_reg** (uint16\_t offset)
- void **apic\_write\_reg** (uint16\_t offset, uint32\_t val)
- void **apic\_send\_eoi** ()
- void **apic\_send\_ipi** (uint8\_t dest, uint8\_t vector, uint32\_t mtype)
- void **apic\_enable** ()
- void **apic\_init** ()

## Variables

- volatile void \* **lapic\_base** = NULL

### 5.65.1 Detailed Description

Implementation of APIC (Advanced Programmable Interrupt Controller) functions.

---

APIC ("Advanced Programmable Interrupt Controller") is the updated Intel standard for the older PIC. It is used in multiprocessor systems and is an integral part of all recent Intel (and compatible) processors. The APIC is used for sophisticated interrupt redirection, and for sending interrupts between processors.

Ref: <https://wiki.osdev.org/APIC>

---

### 5.65.2 Function Documentation

#### 5.65.2.1 `apic_enable()`

```
void apic_enable ()
```

Enables the Advanced Programmable Interrupt Controller (APIC).

This function writes a value to the Spurious Interrupt Vector Register (SVR) of the APIC, enabling it and setting the interrupt vector number to the `APIC_SPURIOUS_VECTOR_NUM` constant.

**Returns**

None

Here is the call graph for this function:



#### 5.65.2.2 `apic_init()`

```
void apic_init ()
```

Initializes the Advanced Programmable Interrupt Controller (APIC). This function checks if the CPU supports APIC, maps the local APIC base address, enables the APIC, and logs the APIC version.

**Returns**

None

#### 5.65.2.3 `apic_read_reg()`

```
uint32_t apic_read_reg (
 uint16_t offset)
```

Reads the value of a register from the Advanced Programmable Interrupt Controller (APIC).

**Parameters**

|               |                                     |
|---------------|-------------------------------------|
| <i>offset</i> | The offset of the register to read. |
|---------------|-------------------------------------|

**Returns**

The value of the register.

**5.65.2.4 apic\_send\_eoi()**

```
void apic_send_eoi ()
```

Sends an End of Interrupt (EOI) signal to the Advanced Programmable Interrupt Controller (APIC).

**Returns**

None

Here is the call graph for this function:

**5.65.2.5 apic\_send\_ipi()**

```
void apic_send_ipi (
 uint8_t dest,
 uint8_t vector,
 uint32_t mtype)
```

Sends an Inter-Processor Interrupt (IPI) to a specific destination processor.

**Parameters**

|               |                               |
|---------------|-------------------------------|
| <i>dest</i>   | The destination processor ID. |
| <i>vector</i> | The interrupt vector number.  |
| <i>mtype</i>  | The message type for the IPI. |

**Returns**

None

**5.65.2.6 apic\_write\_reg()**

```
void apic_write_reg (
 uint16_t offset,
 uint32_t val)
```

Writes a value to a register in the Advanced Programmable Interrupt Controller (APIC).

## Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>offset</i> | The offset of the register.         |
| <i>val</i>    | The value to write to the register. |

## Returns

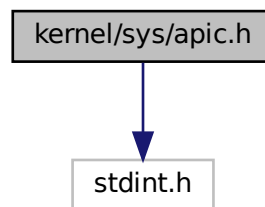
None

## 5.66 kernel/sys/apic.h File Reference

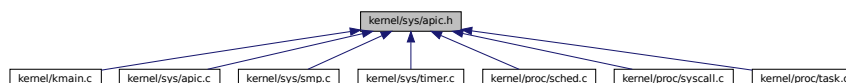
Definition of APIC (Advanced Programmable Interrupt Controller) related data structures.

```
#include <stdint.h>
```

Include dependency graph for apic.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define APIC_REG_ID 0x20`
- `#define APIC_REG_VERSION 0x30`
- `#define APIC_REG_SPURIOUS_INT 0xF0`
- `#define APIC_REG_EOI 0xB0`
- `#define APIC_REG_ICR_LOW 0x300`
- `#define APIC_REG_ICR_HIGH 0x310`
- `#define APIC_SPURIOUS_VECTOR_NUM 0xFF`
- `#define APIC_FLAG_ENABLE (1 << 8)`
- `#define APIC_IPI_TYPE_INIT 0b101`
- `#define APIC_IPI_TYPE_STARTUP 0b110`

## Functions

- void `apic_init` (void)
- void `apic_enable` (void)



- uint32\_t [apic\\_read\\_reg](#) (uint16\_t offset)
- void [apic\\_write\\_reg](#) (uint16\_t offset, uint32\_t val)
- void [apic\\_send\\_eoi](#) (void)
- void [apic\\_send\\_ipi](#) (uint8\_t dest, uint8\_t vector, uint32\_t mtype)

## Variables

- volatile void \* **lapic\_base**

### 5.66.1 Detailed Description

Definition of APIC (Advanced Programmable Interrupt Controller) related data structures.

---

APIC is used in multiprocessor systems and is an integral part of all recent Intel (and compatible) processors. The APIC is used for sophisticated interrupt redirection, and for sending interrupts between processors.

---

### 5.66.2 Function Documentation

#### 5.66.2.1 [apic\\_enable\(\)](#)

```
void apic_enable ()
```

Enables the Advanced Programmable Interrupt Controller (APIC).

This function writes a value to the Spurious Interrupt Vector Register (SVR) of the APIC, enabling it and setting the interrupt vector number to the APIC\_SPURIOUS\_VECTOR\_NUM constant.

#### Returns

None

Here is the call graph for this function:



#### 5.66.2.2 [apic\\_init\(\)](#)

```
void apic_init ()
```

Initializes the Advanced Programmable Interrupt Controller (APIC). This function checks if the CPU supports APIC, maps the local APIC base address, enables the APIC, and logs the APIC version.

#### Returns

None

### 5.66.2.3 apic\_read\_reg()

```
uint32_t apic_read_reg (
 uint16_t offset)
```

Reads the value of a register from the Advanced Programmable Interrupt Controller (APIC).

**Parameters**

|               |                                     |
|---------------|-------------------------------------|
| <i>offset</i> | The offset of the register to read. |
|---------------|-------------------------------------|

**Returns**

The value of the register.

**5.66.2.4 apic\_send\_eoi()**

```
void apic_send_eoi ()
```

Sends an End of Interrupt (EOI) signal to the Advanced Programmable Interrupt Controller (APIC).

**Returns**

None

Here is the call graph for this function:

**5.66.2.5 apic\_send\_ipi()**

```
void apic_send_ipi (
 uint8_t dest,
 uint8_t vector,
 uint32_t mtype)
```

Sends an Inter-Processor Interrupt (IPI) to a specific destination processor.

**Parameters**

|               |                               |
|---------------|-------------------------------|
| <i>dest</i>   | The destination processor ID. |
| <i>vector</i> | The interrupt vector number.  |
| <i>mtype</i>  | The message type for the IPI. |

**Returns**

None

**5.66.2.6 apic\_write\_reg()**

```
void apic_write_reg (
 uint16_t offset,
 uint32_t val)
```

Writes a value to a register in the Advanced Programmable Interrupt Controller (APIC).

## Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>offset</i> | The offset of the register.         |
| <i>val</i>    | The value to write to the register. |

## Returns

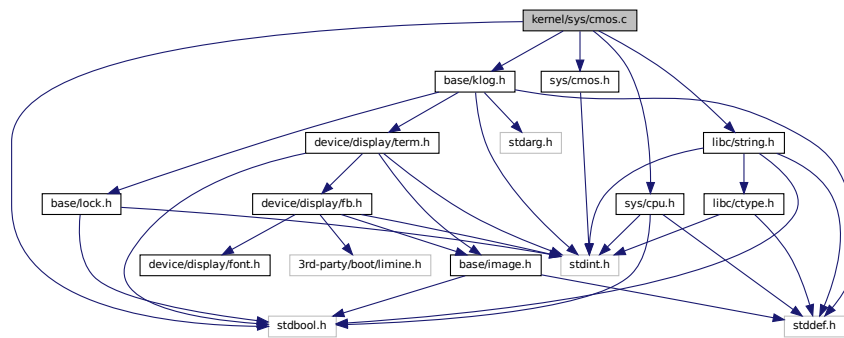
None

## 5.67 kernel/sys/cmos.c File Reference

Implementation of CMOS related functions.

```
#include <stdbool.h>
#include <libc/string.h>
#include <sys/cmos.h>
#include <sys/cpu.h>
#include <base/klog.h>
```

Include dependency graph for cmos.c:



## Macros

- `#define CURRENT_YEAR 2022 /* Change this each year! */`

## Functions

- `uint8_t get_rtc_register (uint8_t reg)`
- `bool update_in_progress ()`
- `bool rtc_values_are_not_equal (cmos_rtc_t c1, cmos_rtc_t c2)`
- `void cmos_init ()`
- `uint64_t cmos_boot_time ()`
- `uint64_t cmos_current_time ()`
- `cmos_rtc_t cmos_read_rtc ()`
- `uint64_t secs_of_years (uint64_t years)`
- `uint64_t secs_of_month (uint64_t months, uint64_t year)`

## Variables

- `int century_register = 0x00`

### 5.67.1 Detailed Description

Implementation of CMOS related functions.

"CMOS" is a tiny bit of very low power static memory that lives on the same chip as the Real-Time Clock (RTC). It was introduced to IBM PC AT in 1984 which used Motorola MC146818A RTC.

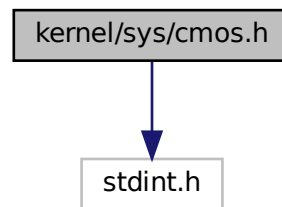
Ref: <https://wiki.osdev.org/CMOS>

## 5.68 kernel/sys/cmos.h File Reference

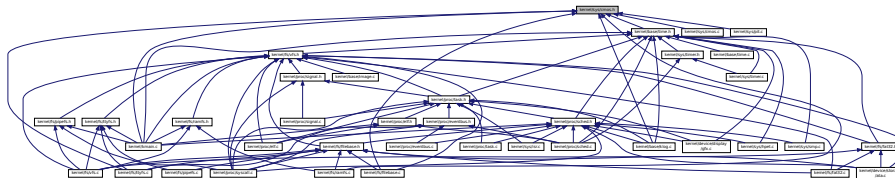
Definition of CMOS related data structures.

```
#include <stdint.h>
```

Include dependency graph for cmos.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [cmos\\_rtc\\_t](#)

### Macros

- #define **CMOS\_COMMAND\_PORT** 0x70
- #define **CMOS\_DATA\_PORT** 0x71
- #define **CMOS\_REG\_SECONDS** 0x00
- #define **CMOS\_REG\_MINUTES** 0x02
- #define **CMOS\_REG\_HOURS** 0x04
- #define **CMOS\_REG\_WEEKDAYS** 0x06
- #define **CMOS\_REG\_DAY** 0x07
- #define **CMOS\_REG\_MONTH** 0x08
- #define **CMOS\_REG\_YEAR** 0x09
- #define **CMOS\_REG\_CENTURY** 0x32
- #define **CMOS\_REG\_STATUS\_A** 0x0A
- #define **CMOS\_REG\_STATUS\_B** 0x0B

## Functions

- uint64\_t **secs\_of\_month** (uint64\_t months, uint64\_t year)
- uint64\_t **secs\_of\_years** (uint64\_t years)
- void **cmos\_init** ()
- [cmos\\_rtc\\_t](#) **cmos\_read\_rtc** ()
- uint64\_t **cmos\_boot\_time** ()
- uint64\_t **cmos\_current\_time** ()

### 5.68.1 Detailed Description

#### Definition of CMOS related data structures.

"CMOS" is a tiny bit of very low power static memory that lives on the same chip as the Real-Time Clock (RTC). It was introduced to IBM PC AT in 1984 which used Motorola MC146818A RTC.

## 5.69 kernel/sys/cpu.c File Reference

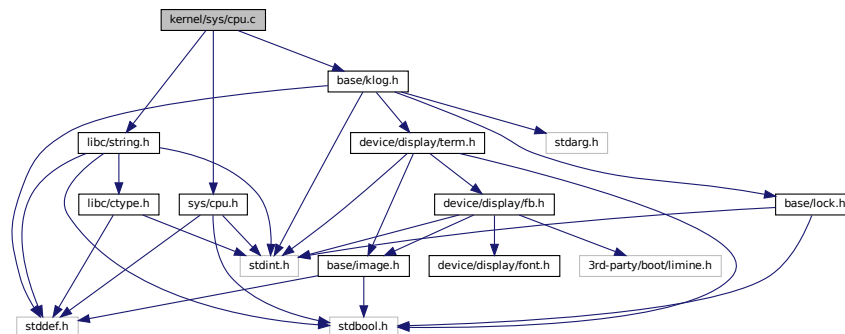
Initialize CPUs including BSP and other cores for SMP arch.

```
#include <libc/string.h>
```

```
#include <sys/cpu.h>
```

```
#include <base/klog.h>
```

Include dependency graph for cpu.c:



## Functions

- void **cpuid** (uint32\_t func, uint32\_t param, uint32\_t \*eax, uint32\_t \*ebx, uint32\_t \*ecx, uint32\_t \*edx)
- bool **cpuid\_check\_feature** ([cpuid\\_feature\\_t](#) feature)
- void **cpu\_init** (uint64\_t cpuno)
- char \* **cpu\_get\_model\_name** ()

### 5.69.1 Detailed Description

Initialize CPUs including BSP and other cores for SMP arch.

This file provides the implementation for initializing the CPUs in the HanOS kernel. It includes important CPU initializations such as enabling write combining to speed up framebuffer read/write speeds and enabling SSE and SSE2 for x86-64 architecture. The file also contains functions for checking CPU features, setting up control registers, and retrieving CPU model information.

## 5.70 kernel/sys/cpu.h File Reference

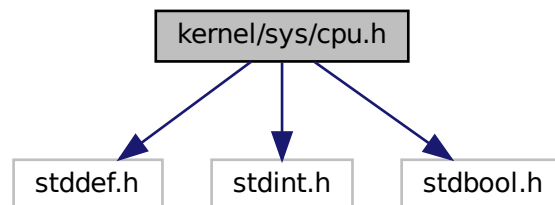
Definition of CPU related data structures and macros.

```
#include <stddef.h>
```

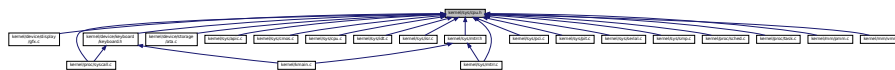
```
#include <stdint.h>
```

```
#include <stdbool.h>
```

Include dependency graph for cpu.h:



This graph shows which files directly or indirectly include this file:



### Data Structures

- struct [cpuid\\_feature\\_t](#)

### Macros

- #define **MSR\_PAT** 0x0277
- #define **MSR\_FS\_BASE** 0xC0000100
- #define **MSR\_GS\_BASE** 0xC0000101
- #define **MSR\_KERN\_GS\_BASE** 0xC0000102
- #define **MSR\_EFER** 0xC0000080
- #define **MSR\_STAR** 0xC0000081
- #define **MSR\_LSTAR** 0xC0000082
- #define **MSR\_CSTAR** 0xC0000083
- #define **MSR\_SFMASK** 0xC0000084
- #define **read\_cr**(cr, n)
- #define **write\_cr**(cr, n)
- #define **CPUID\_XSAVE** (1 << 26)
- #define **CPUID\_AVX** (1 << 28)
- #define **CPUID\_AVX512** (1 << 16)
- #define **CPUID\_INVARIANT\_TSC** (1 << 8)
- #define **CPUID\_TSC\_DEADLINE** (1 << 24)
- #define **CPUID\_MTRR** (1 << 12)

### Functions

- void **cpu\_init** (uint64\_t cpuno)
- char \* **cpu\_get\_model\_name** ()
- bool **cpuid\_check\_feature** ([cpuid\\_feature\\_t](#) feature)

### 5.70.1 Detailed Description

#### Definition of CPU related data structures and macros.

Macros and inline functions including read & write control registers, model specific registers and port input & output.

### 5.70.2 Macro Definition Documentation

#### 5.70.2.1 read\_cr

```
#define read_cr(
 cr,
 n)
```

Value:

```
asm volatile("mov %%" cr ", %%rax;" \
"mov %%rax, %0"
: "=g" (* (n))
:
: "rax");
```

#### 5.70.2.2 write\_cr

```
#define write_cr(
 cr,
 n)
```

Value:

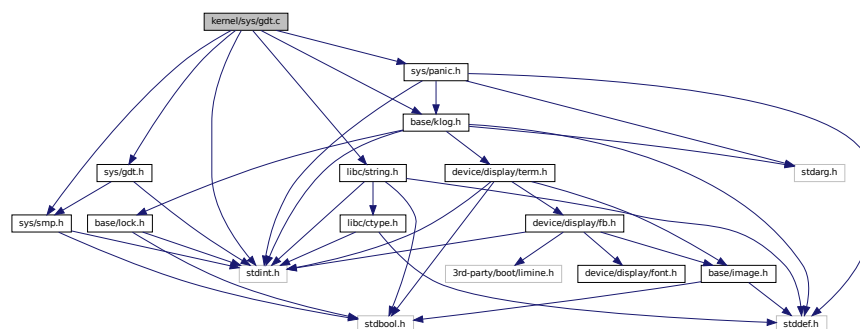
```
asm volatile("mov %0, %%rax;" \
"mov %%rax, %%" cr
:
: "g" (n)
: "rax");
```

## 5.71 kernel/sys/gdt.c File Reference

Implementation of GDT related functions.

```
#include <stdint.h>
#include <libc/string.h>
#include <base/klog.h>
#include <sys/gdt.h>
#include <sys/panic.h>
#include <sys/smp.h>
```

Include dependency graph for gdt.c:





## Functions

- void **gdt\_init** ([cpu\\_t](#) \*cpuinfo)
- void **gdt\_install\_tss** ([cpu\\_t](#) \*cpuinfo)

### 5.71.1 Detailed Description

Implementation of GDT related functions.

The Global Descriptor Table (GDT) contains entries telling the CPU about memory segments.

In HanOS, GDT initialization is very simple. Only memory protection is used. Two ring-0 and two ring-3 segment descriptor are defined. The memory regions are from 0 to 4GB.

Jan 14, 2023: To make sure that we can call write() in limine bootloader, the initializaton function is refined according to limine protocol.

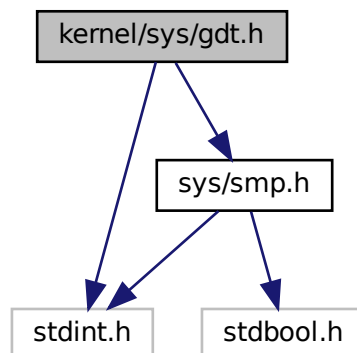
## 5.72 kernel/sys/gdt.h File Reference

Definition of GDT related data structures.

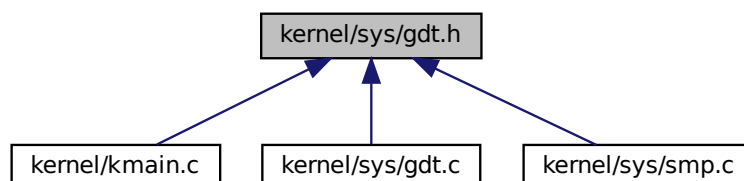
```
#include <stdint.h>
```

```
#include <sys/smp.h>
```

Include dependency graph for gdt.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [gdt\\_entry\\_t](#)
- struct [sys\\_seg\\_desc\\_t](#)
- struct [gdt\\_table\\_t](#)
- struct [gdt\\_register\\_t](#)

## Macros

- `#define AC_AC 0x1 /* access */`
- `#define AC_RW 0x2 /* readable for code & writeable for data selector */`
- `#define AC_DC 0x4 /* direction */`
- `#define AC_EX 0x8 /* executable, code segment */`
- `#define AC_ST`
- `#define AC_PR`
- `#define AC_DPL_KERN 0x0 /* RING 0 kernel level */`
- `#define AC_DPL_USER 0x60 /* RING 3 user level - 01100000b */`
- `#define GDT_GR`
- `#define GDT_SZ 0x4 /* size bt, 32 bit protect mode */`
- `#define GDT_LM`
- `#define SEL_KCODE 0x1`
- `#define SEL_KDATA 0x2`
- `#define SEL_UCODE 0x3`
- `#define SEL_UDATA 0x4`
- `#define SEL_TSS 0x5`
- `#define RPL_KERN 0x0`
- `#define RPL_USER 0x3`
- `#define CPL_KERN 0x0`
- `#define CPL_USER 0x3`

## Functions

- void [gdt\\_init](#) ([cpu\\_t](#) \*cpuinfo)
- void [gdt\\_install\\_tss](#) ([cpu\\_t](#) \*cpuinfo)

### 5.72.1 Detailed Description

#### Definition of GDT related data structures.

---

The Global Descriptor Table (GDT) contains entries telling the CPU about memory segments.

---

### 5.72.2 Macro Definition Documentation

#### 5.72.2.1 AC\_PR

```
#define AC_PR
```

**Value:**

```
0x80 /* percent in memory, Must be set (1) for any valid
 * segment.
 */
```



- void **hpet\_nanosleep** (uint64\_t nanos)
- void **hpet\_init** ()

## Variables

- **hpet\_t** \* **hpet** = NULL

### 5.73.1 Detailed Description

#### Implementation of HPET (High Precision Event Timer) functions.

This module includes implementation of HPET initialization, getting nanos and nanos sleep.

HPET consists of (usually 64-bit) main counter (which counts up), as well as from 3 to 32 32-bit or 64-bit wide comparators. HPET is programmed using memory mapped IO, and the base address of HPET can be found using ACPI.

General initialization:

1. Find HPET base address in 'HPET' ACPI table.
2. Calculate HPET frequency ( $f = 10^{15} / \text{period}$ ).
3. Save minimal tick (either from ACPI table or configuration register).
4. Initialize comparators.
5. Set ENABLE\_CNF bit.

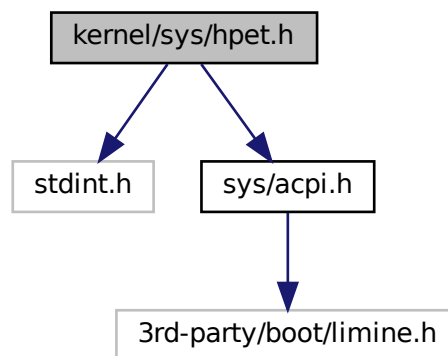
## 5.74 kernel/sys/hpet.h File Reference

Definition of HPET (High Precision Event Timer) related data structures.

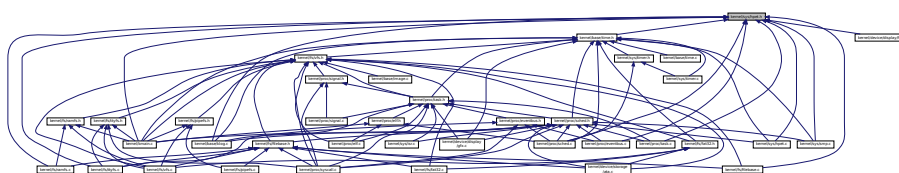
```
#include <stdint.h>
```

```
#include <sys/acpi.h>
```

Include dependency graph for hpet.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [hpet\\_sdt\\_t](#)
- struct [hpet\\_timer\\_t](#)
- struct [hpet\\_t](#)

## Functions

- void [hpet\\_init](#) ()
- uint64\_t [hpet\\_get\\_nanos](#) ()
- uint64\_t [hpet\\_get\\_millis](#) ()
- void [hpet\\_nanosleep](#) (uint64\_t nanos)

## Variables

- [hpet\\_t](#) \* [hpet](#)

### 5.74.1 Detailed Description

Definition of HPET (High Precision Event Timer) related data structures.

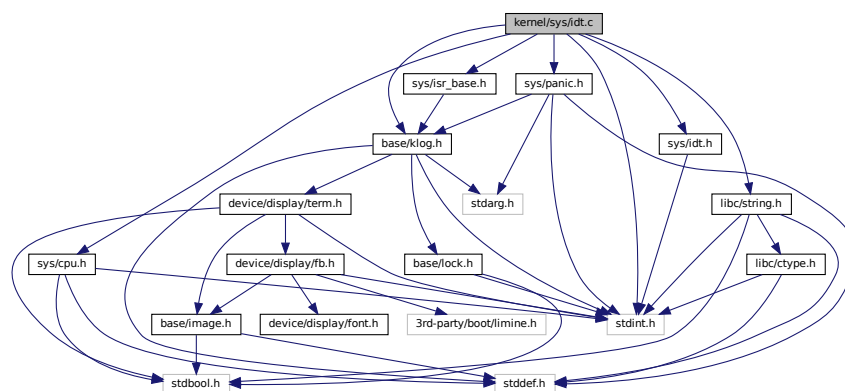
HPET, or High Precision Event Timer, is a piece of hardware designed by Intel and Microsoft to replace older PIT and RTC. It consists of (usually 64-bit) main counter (which counts up), as well as from 3 to 32 32-bit or 64-bit wide comparators. HPET is programmed using memory mapped IO, and the base address of HPET can be found using ACPI.

## 5.75 kernel/sys/idt.c File Reference

Implementation of idt related functions.

```
#include <stdint.h>
#include <libc/string.h>
#include <base/klog.h>
#include <sys/isr_base.h>
#include <sys/idt.h>
#include <sys/cpu.h>
#include <sys/panic.h>
```

Include dependency graph for idt.c:



## Functions

- void **idt\_set\_handler** (uint8\_t vector, void \*handler)
- uint8\_t **idt\_get\_available\_vector** (void)
- void **irq\_set\_mask** (uint8\_t line)
- void **irq\_clear\_mask** (uint8\_t line)
- void **idt\_init** ()

### 5.75.1 Detailed Description

Implementation of idt related functions.

The Interrupt Descriptor Table (idt) telling the CPU where the Interrupt Service Routines (ISR) are located (one per interrupt vector). The idt entries are called gates. It can contain Interrupt Gates, Task Gates and Trap Gates. As the first step, only trap gates (exceptions) are implemented.

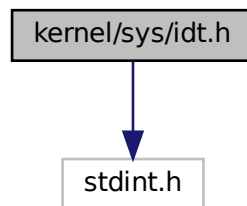
Ref: [https://wiki.osdev.org/Interrupt\\_Descriptor\\_Table](https://wiki.osdev.org/Interrupt_Descriptor_Table)

## 5.76 kernel/sys/idt.h File Reference

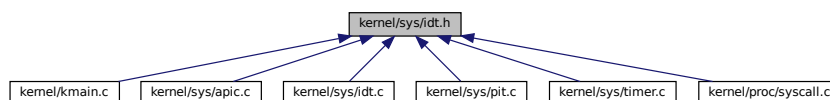
Definition of IDT related data structures.

```
#include <stdint.h>
```

Include dependency graph for idt.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [idt\\_entry\\_t](#)
- struct [idt\\_register\\_t](#)

## Macros

- #define **IDT\_ENTRIES** 256
- #define **IDT\_DEFAULT\_TYPE\_ATTRIBUTES** 0b10001110

## Functions

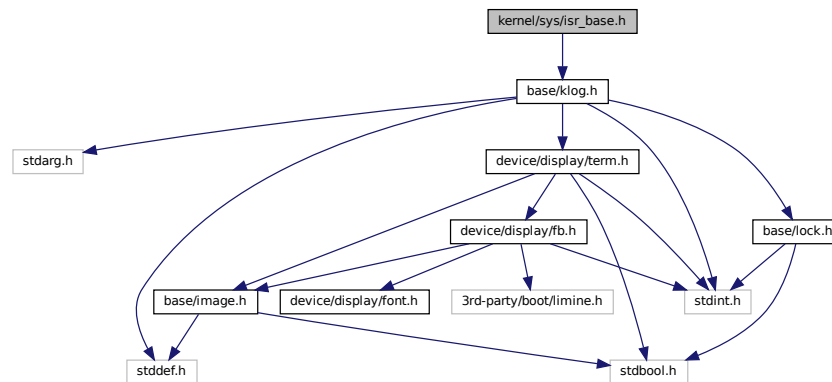
- void **idt\_init** ()
- void **idt\_set\_handler** (uint8\_t vector, void \*handler)
- uint8\_t **idt\_get\_available\_vector** (void)
- void **irq\_set\_mask** (uint8\_t line)
- void **irq\_clear\_mask** (uint8\_t line)

## 5.78 kernel/sys/isr\_base.h File Reference

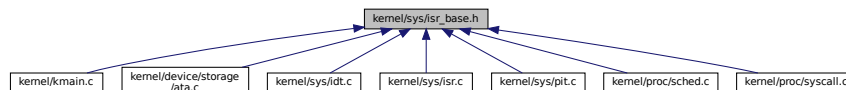
Definition of ISR related data structures.

```
#include <base/klog.h>
```

Include dependency graph for isr\_base.h:



This graph shows which files directly or indirectly include this file:



### Macros

- **#define PIC1** 0x20 /\* Master PIC \*/
- **#define PIC2** 0xA0 /\* Slave PIC \*/
- **#define PIC1\_DATA** (PIC1 + 1)
- **#define PIC2\_DATA** (PIC2 + 1)
- **#define PIC\_EOI** 0x20 /\* end of interrupt \*/
- **#define IRQ\_BASE** 0x20
- **#define IRQ0** 32
- **#define IRQ1** 33
- **#define IRQ2** 34
- **#define IRQ3** 35
- **#define IRQ4** 36
- **#define IRQ5** 37
- **#define IRQ6** 38
- **#define IRQ7** 39
- **#define IRQ8** 40
- **#define IRQ9** 41
- **#define IRQ10** 42
- **#define IRQ11** 43
- **#define IRQ12** 44
- **#define IRQ128** 128
- **#define isr\_enable\_interrupts()**
- **#define isr\_disable\_interrupts()**



## Typedefs

- typedef void(\* **exc\_handler\_t**) ()

## Functions

- void **exc\_register\_handler** (uint64\_t id, exc\_handler\_t handler)
- void **exc0** (void \*p)
- void **exc1** (void \*p)
- void **exc2** (void \*p)
- void **exc3** (void \*p)
- void **exc4** (void \*p)
- void **exc5** (void \*p)
- void **exc6** (void \*p)
- void **exc7** (void \*p)
- void **exc8** (void \*p)
- void **exc10** (void \*p)
- void **exc11** (void \*p)
- void **exc12** (void \*p)
- void **exc13** (void \*p)
- void **exc14** (void \*p)
- void **exc16** (void \*p)
- void **exc17** (void \*p)
- void **exc18** (void \*p)
- void **exc19** (void \*p)
- void **exc20** (void \*p)
- void **exc30** (void \*p)
- void **irq0** (void \*p)
- void **irq1** (void \*p)
- void **irq2** (void \*p)
- void **irq3** (void \*p)
- void **irq4** (void \*p)
- void **irq5** (void \*p)
- void **irq6** (void \*p)
- void **irq7** (void \*p)
- void **irq8** (void \*p)
- void **irq9** (void \*p)
- void **irq10** (void \*p)
- void **irq11** (void \*p)
- void **irq12** (void \*p)
- void **irq128** (void \*p)

### 5.78.1 Detailed Description

Definition of ISR related data structures.

---

The x86 architecture is an interrupt driven system.

---

### 5.78.2 Macro Definition Documentation

#### 5.78.2.1 isr\_disable\_interrupts

```
#define isr_disable_interrupts()
```

**Value:**

```
{
 asm volatile("cli");
}
```

### 5.78.2.2 isr\_enable\_interrupts

```
#define isr_enable_interrupts()
```

**Value:**

```
{
 asm volatile("sti");
}
```

## 5.79 kernel/sys/madt.c File Reference

Implementation of ACPI MADT (Multiple APIC Description Table) functions.

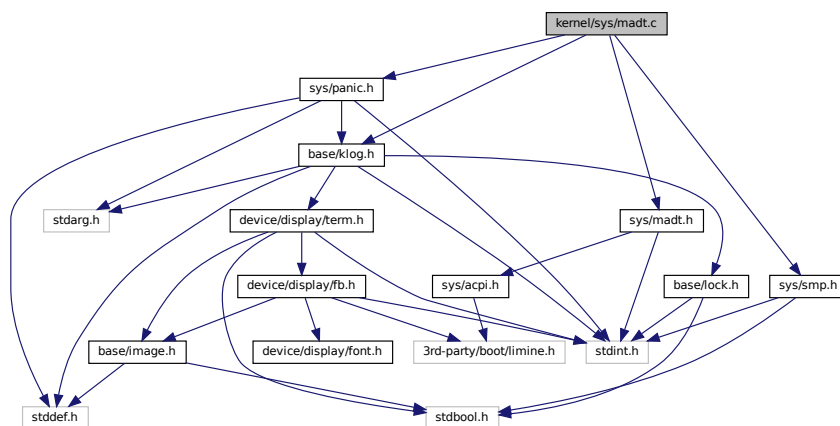
```
#include <sys/madt.h>
```

```
#include <sys/panic.h>
```

```
#include <sys/smp.h>
```

```
#include <base/klog.h>
```

Include dependency graph for madt.c:



## Functions

- uint32\_t madt\_get\_num\_ioapic ()
- uint32\_t madt\_get\_num\_lapic ()
- madt\_record\_ioapic\_t \*\* madt\_get\_ioapics ()
- madt\_record\_lapic\_t \*\* madt\_get\_lapics ()
- uint64\_t madt\_get\_lapic\_base ()
- void madt\_init ()

### 5.79.1 Detailed Description

Implementation of ACPI MADT (Multiple APIC Description Table) functions.

The MADT describes all of the interrupt controllers in the system. It can be used to enumerate the processors currently available.

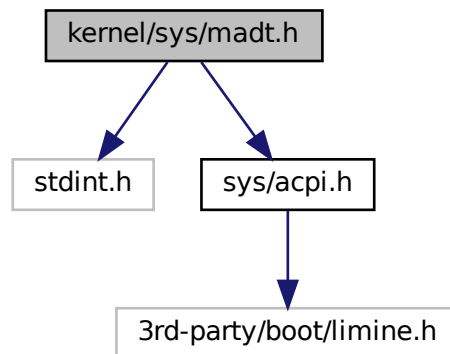
## 5.80 kernel/sys/madt.h File Reference

Definition of ACPI MADT (Multiple APIC Description Table) related data structures.

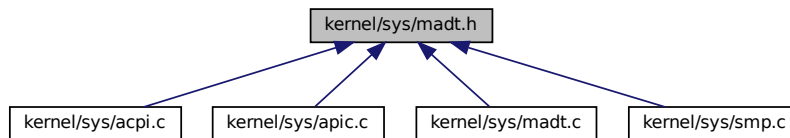
```
#include <stdint.h>
```

```
#include <sys/acpi.h>
```

Include dependency graph for madt.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [madt\\_record\\_hdr\\_t](#)
- struct [madt\\_record\\_lapic\\_t](#)
- struct [madt\\_record\\_ioapic\\_t](#)
- struct [madt\\_record\\_iso\\_t](#)
- struct [madt\\_record\\_nmi\\_t](#)
- struct [madt\\_t](#)

## Macros

- #define **MADT\_RECORD\_TYPE\_LAPIC** 0
- #define **MADT\_RECORD\_TYPE\_IOAPIC** 1
- #define **MADT\_RECORD\_TYPE\_ISO** 2
- #define **MADT\_RECORD\_TYPE\_NMI** 4
- #define **MADT\_RECORD\_TYPE\_LAPIC\_AO** 5
- #define **MADT\_LAPIC\_FLAG\_ENABLED** (1 << 0)
- #define **MADT\_LAPIC\_FLAG\_ONLINE\_CAPABLE** (1 << 1)

## Functions

- void **madt\_init** ()
- uint32\_t **madt\_get\_num\_ioapic** ()

- `uint32_t madt_get_num_lapic ()`
- `madt_record_ioapic_t ** madt_get_ioapics ()`
- `madt_record_lapic_t ** madt_get_lapics ()`
- `uint64_t madt_get_lapic_base ()`

### 5.80.1 Detailed Description

Definition of ACPI MADT (Multiple APIC Description Table) related data structures.

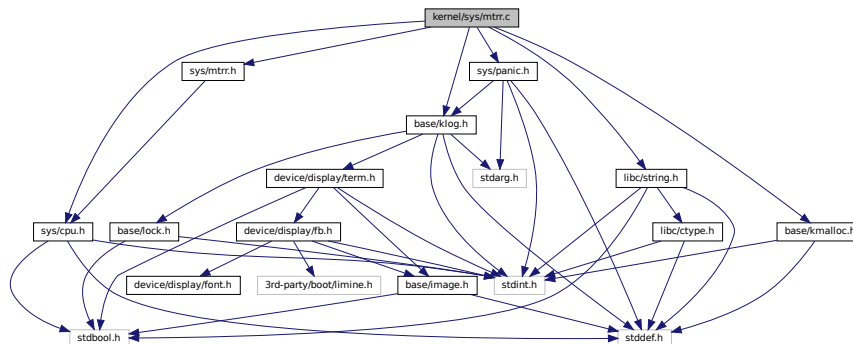
The MADT describes all of the interrupt controllers in the system. It can be used to enumerate the processors currently available.

## 5.81 kernel/sys/mtrr.c File Reference

MTRR configuration to speed up memory reading & writing operations.

```
#include <libc/string.h>
#include <sys/cpu.h>
#include <sys/mtrr.h>
#include <sys/panic.h>
#include <base/klog.h>
#include <base/kmalloc.h>
```

Include dependency graph for mtrr.c:



### Functions

- `void mtrr_save (uint16_t cpu_id, void *framebuffer)`
- `void mtrr_restore (uint16_t cpu_id)`

### Variables

- `uint64_t * saved_mtrrs = NULL`

### 5.81.1 Detailed Description

MTRR configuration to speed up memory reading & writing operations.

MTRR, or Memory-Type Range Registers are a group of x86 Model Specific Registers providing a way to control access and cacheability of physical memory regions. These were introduced in the Intel Pentium Pro (P6) processor, intended to extend and enhance the memory type information provided by page tables (i.e. the write-through and cache disable bits).

The set of registers is provided in two groups: 11 registers for 88 fixed ranges and a number of base-mask pairs for custom range configuration. The exact number of the latter can be known by reading the capabilities register.

There are 5 memory types defined for use in MTRRs:

| Number | Name                  | Description                                                                                                                                                                            |
|--------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| =====  | =====                 | =====                                                                                                                                                                                  |
| 0      | UC -- Uncacheable     | All accesses are uncacheable. Write combining is not allowed. Speculative accesses are not allowed.                                                                                    |
| 1      | WC -- Write-Combining | All accesses are uncacheable. Write combining is allowed. Speculative reads are allowed.                                                                                               |
| 4      | WT -- Writethrough    | Reads allocate cache lines on a cache miss. Cache lines are not allocated on a write miss. Write hits update the cache and main memory.                                                |
| 5      | WP -- Write-Protect   | Reads allocate cache lines on a cache miss. All writes update main memory. Cache lines are not allocated on a write miss. Write hits invalidate the cache line and update main memory. |
| 6      | WB -- Writeback       | Reads allocate cache lines on a cache miss, and can allocate to either the shared, exclusive, or modified state. Writes allocate to the modified state on a cache miss.                |

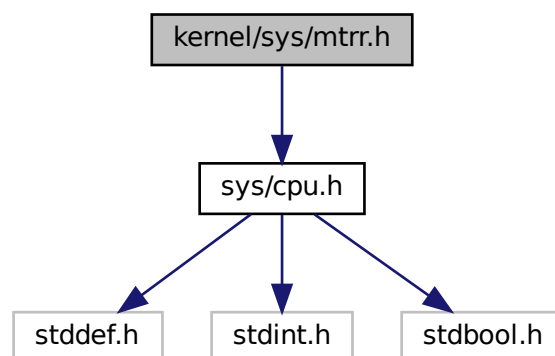
Reference: <https://wiki.osdev.org/MTRR>

## 5.82 kernel/sys/mtrr.h File Reference

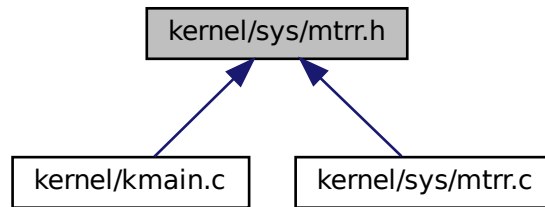
Definition of MTRR macros and save/restore functions.

```
#include <sys/cpu.h>
```

Include dependency graph for mtrr.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define MTRR_CACHE_UNCACHEABLE 0`
- `#define MTRR_CACHE_WRITE_COMBINING 1`
- `#define MTRR_CACHE_WRITE_THROUGH 4`
- `#define MTRR_CACHE_WRITE_PROTECTED 5`
- `#define MTRR_CACHE_WRITE_BACK 6`

## Functions

- `void mtrr_save (uint16_t cpu_id, void *framebuffer)`
- `void mtrr_restore (uint16_t cpu_id)`

### 5.82.1 Detailed Description

Definition of MTRR macros and save/restore functions.

Save and modify MTRR settings when booting system, and restore every core by modified values.

## 5.83 kernel/sys/panic.h File Reference

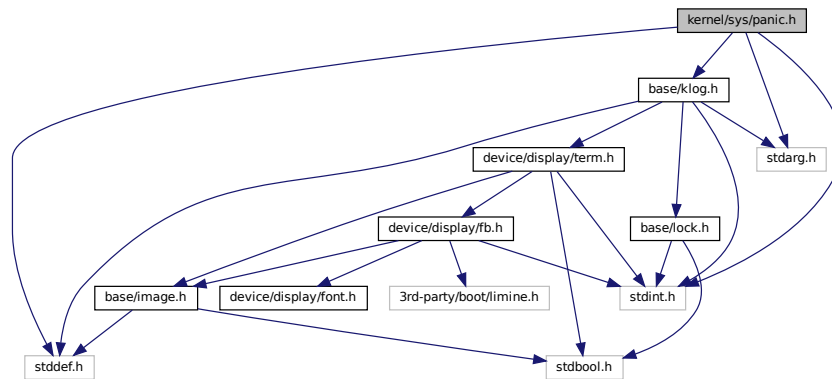
Implementation of panic related functions.

```

#include <stdint.h>
#include <stddef.h>
#include <stdarg.h>
#include <base/klog.h>

```

Include dependency graph for panic.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define kpanic(s, ...)`
- `#define panic_unless(c)`
- `#define panic_if(c)`

## Functions

- `void dump_backtrace()`

### 5.83.1 Detailed Description

Implementation of panic related functions.

Definition of panic related data structures.

A kernel panic is one of several boot issues. In basic terms, it is a situation when the kernel can't load properly and therefore the system fails to boot.

### 5.83.2 Macro Definition Documentation

#### 5.83.2.1 kpanic

```
#define kpanic(
 s,
 ...)
```

Value:

```
{
 asm volatile("cli");
 if (term_get_mode() != TERM_MODE_INFO) {
 term_switch(TERM_MODE_INFO);
 }
 klog_vprintf(KLOG_LEVEL_ERROR, "Enter kernel panic...\n"); \
 klog_vprintf(KLOG_LEVEL_ERROR, s, ##__VA_ARGS__); \
}
```

```

 dump_backtrace();
 while (true)
 asm volatile("hlt");
}

```

### 5.83.2.2 panic\_if

```

#define panic_if(
 c)

```

**Value:**

```

({ \
 if((c)) \
 kpanic("panic_if(" #c ") triggered in " \
 "%s:%d", __FILE__, __LINE__); \
})

```

### 5.83.2.3 panic\_unless

```

#define panic_unless(
 c)

```

**Value:**

```

({ \
 if(!(c)) \
 kpanic("panic_unless(" #c ") triggered in " \
 "%s:%d", __FILE__, __LINE__); \
})

```

## 5.84 kernel/sys/pci.c File Reference

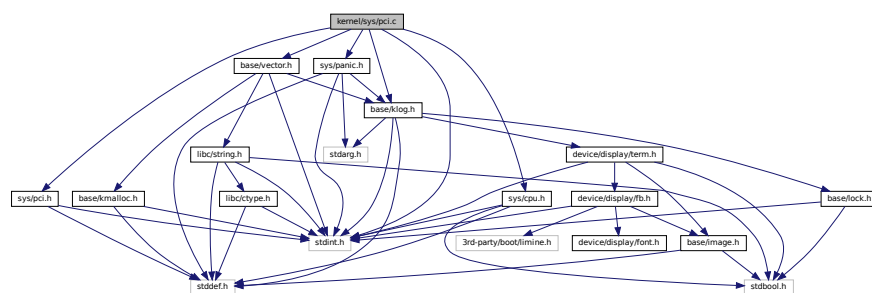
Implementation of PCI related functions.

```

#include <stdint.h>
#include <sys/pci.h>
#include <sys/cpu.h>
#include <sys/panic.h>
#include <base/klog.h>
#include <base/vector.h>

```

Include dependency graph for pci.c:



## Macros

- `#define MAX_BUS 1`
- `#define MAX_DEVICE 32`
- `#define MAX_FUNCTION 8`
- `#define pci_func_exist(dev)`
- `#define pci_read_vendor_id(dev) (pci_ind(PCI_MAKE_DEVICE_ID(dev), PCI_CLASS_LEGACY) & 0xFF ← FFF)`
- `#define pci_read_device_id(dev) (pci_ind(PCI_MAKE_DEVICE_ID(dev), PCI_CLASS_LEGACY) >> 16)`



- `#define pci_read_class(dev) (pci_ind(PCI_MAKE_DEVICE_ID(dev), PCI_CLASS_PERIPHERALS) >> 24)`
- `#define pci_read_subclass(dev) ((pci_ind(PCI_MAKE_DEVICE_ID(dev), PCI_CLASS_PERIPHERALS) >> 16) & 0xFF)`
- `#define pci_read_prog_if(dev) ((pci_ind(PCI_MAKE_DEVICE_ID(dev), PCI_CLASS_PERIPHERALS) >> 8) & 0xFF)`
- `#define pci_read_header(dev) (((pci_ind(PCI_MAKE_DEVICE_ID(dev), PCI_CLASS_PERIPHERALS) >> 16) & ~(1 << 7)) & 0xFF)`
- `#define pci_read_sub_bus(dev) ((pci_ind(PCI_MAKE_DEVICE_ID(dev), 0x18) >> 8) & 0xFF)`
- `#define pci_is_bridge(dev) (pci_read_header(dev) == 0x1 && pci_read_class(dev) == 0x6)`
- `#define pci_has_multi_func(dev) (((pci_ind(PCI_MAKE_DEVICE_ID(dev), PCI_CLASS_SERIAL_BUS) >> 16) & (1 << 7)) & 0xFF)`

## Functions

- `vec_new (pci_device_t, pci_devices)`
- `const char * pci_device_id_to_string (pci_device_t *device)`
- `void pci_get_bar (pci_bar_t *bar, uint32_t id, uint32_t index)`
- `uint8_t pci_inb (uint32_t id, uint32_t offset)`
- `void pci_outb (uint32_t id, uint32_t offset, uint8_t data)`
- `uint16_t pci_inw (uint32_t id, uint32_t offset)`
- `void pci_outw (uint32_t id, uint32_t offset, uint16_t data)`
- `uint32_t pci_ind (uint32_t id, uint32_t offset)`
- `void pci_outd (uint32_t id, uint32_t offset, uint32_t data)`
- `void pci_init (void)`
- `void pci_list (void)`

### 5.84.1 Detailed Description

Implementation of PCI related functions.

---

There are two functions in this file:

1. `pci_init()` which can scan all PCI devices and store information into an array.
2. `pci_list()` which can be called by command line and display PCI device list.

The PCI device can be determined by vendor id and device id:

<https://devicehunt.com/view/type/pci/vendor/1052#search-results-table>

Ref: <https://wiki.osdev.org/PCI> for technical document <https://devicehunt.com/view/type/pci/vendor/8086> for device name

---

### 5.84.2 Macro Definition Documentation

#### 5.84.2.1 pci\_func\_exist

```
#define pci_func_exist(
 dev)
```

Value:

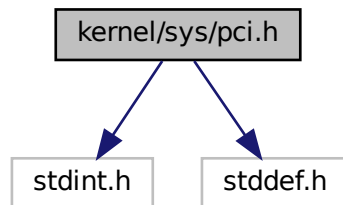
```
((uint16_t) (pci_ind(PCI_MAKE_DEVICE_ID(dev), PCI_CLASS_LEGACY) & 0xFFFF) \
!= 0xFFFF)
```

## 5.85 kernel/sys/pci.h File Reference

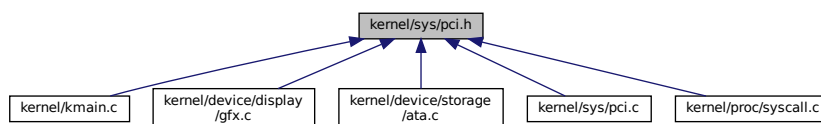
Definition of PCI related data structures and functions.

---

```
#include <stdint.h>
#include <stddef.h>
Include dependency graph for pci.h:
```



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [pci\\_device\\_t](#)
- struct [pci\\_device\\_desc\\_t](#)
- struct [pci\\_bar\\_t](#)

## Macros

- #define **PCI\_MAKE\_ID**(b, d, f) ((b) << 16) | ((d) << 11) | ((f) << 8)
- #define **PCI\_MAKE\_DEVICE\_ID**(dev)
- #define **PCI\_PORT\_ADDR** 0xCF8
- #define **PCI\_PORT\_DATA** 0xCFC
- #define **PCI\_TYPE\_MULTIFUNC** 0x80
- #define **PCI\_TYPE\_GENERIC** 0x00
- #define **PCI\_TYPE\_PCI\_BRIDGE** 0x01
- #define **PCI\_TYPE\_CARDBUS\_BRIDGE** 0x02
- #define **PCI\_CONFIG\_VENDOR\_ID** 0x00
- #define **PCI\_CONFIG\_DEVICE\_ID** 0x02
- #define **PCI\_CONFIG\_COMMAND** 0x04
- #define **PCI\_CONFIG\_STATUS** 0x06
- #define **PCI\_CONFIG\_REVISION\_ID** 0x08
- #define **PCI\_CONFIG\_PROG\_INTF** 0x09
- #define **PCI\_CONFIG\_SUBCLASS** 0x0a
- #define **PCI\_CONFIG\_CLASS\_CODE** 0x0b
- #define **PCI\_CONFIG\_CACHELINE\_SIZE** 0x0c
- #define **PCI\_CONFIG\_LATENCY** 0x0d
- #define **PCI\_CONFIG\_HEADER\_TYPE** 0x0e

- `#define PCI_CONFIG_BIST 0x0f`
- `#define PCI_CONFIG_BAR0 0x10`
- `#define PCI_CONFIG_BAR1 0x14`
- `#define PCI_CONFIG_BAR2 0x18`
- `#define PCI_CONFIG_BAR3 0x1c`
- `#define PCI_CONFIG_BAR4 0x20`
- `#define PCI_CONFIG_BAR5 0x24`
- `#define PCI_CONFIG_CARDBUS_CIS 0x28`
- `#define PCI_CONFIG_SUBSYSTEM_VENDOR_ID 0x2c`
- `#define PCI_CONFIG_SUBSYSTEM_DEVICE_ID 0x2e`
- `#define PCI_CONFIG_EXPANSION_ROM 0x30`
- `#define PCI_CONFIG_CAPABILITIES 0x34`
- `#define PCI_CONFIG_INTERRUPT_LINE 0x3c`
- `#define PCI_CONFIG_INTERRUPT_PIN 0x3d`
- `#define PCI_CONFIG_MIN_GRANT 0x3e`
- `#define PCI_CONFIG_MAX_LATENCY 0x3f`
- `#define PCI_BAR_IO 0x01`
- `#define PCI_BAR_LOWMEM 0x02`
- `#define PCI_BAR_64 0x04`
- `#define PCI_BAR_PREFETCH 0x08`
- `#define VENDOR_INTEL 0x8086`
- `#define PCI_CLASS_LEGACY 0x00`
- `#define PCI_CLASS_STORAGE 0x01`
- `#define PCI_CLASS_NETWORK 0x02`
- `#define PCI_CLASS_DISPLAY 0x03`
- `#define PCI_CLASS_MULTIMEDIA 0x04`
- `#define PCI_CLASS_MEMORY 0x05`
- `#define PCI_CLASS_BRIDGE_DEVICE 0x06`
- `#define PCI_CLASS_COMMUNICATION 0x07`
- `#define PCI_CLASS_PERIPHERALS 0x08`
- `#define PCI_CLASS_INPUT_DEVICES 0x09`
- `#define PCI_CLASS_DOCKING_STATION 0x0a`
- `#define PCI_CLASS_PROCESSOR 0x0b`
- `#define PCI_CLASS_SERIAL_BUS 0x0c`
- `#define PCI_CLASS_WIRELESS 0x0d`
- `#define PCI_CLASS_INTELLIGENT_IO 0x0e`
- `#define PCI_CLASS_SATELLITE 0x0f`
- `#define PCI_CLASS_CRYPT 0x10`
- `#define PCI_CLASS_SIGNAL_PROCESSING 0x11`
- `#define PCI_CLASS_UNDEFINED 0xff`
- `#define PCI_UNCLASSIFIED 0x0000`
- `#define PCI_VGA_COMPATIBLE 0x0001`
- `#define PCI_STORAGE_SCSI 0x0100`
- `#define PCI_STORAGE_IDE 0x0101`
- `#define PCI_STORAGE_FLOPPY 0x0102`
- `#define PCI_STORAGE_IPI 0x0103`
- `#define PCI_STORAGE_RAID 0x0104`
- `#define PCI_STORAGE_ATA 0x0105`
- `#define PCI_STORAGE_SATA 0x0106`
- `#define PCI_STORAGE_OTHER 0x0180`
- `#define PCI_NETWORK_ETHERNET 0x0200`
- `#define PCI_NETWORK_TOKEN_RING 0x0201`
- `#define PCI_NETWORK_FDDI 0x0202`
- `#define PCI_NETWORK_ATM 0x0203`
- `#define PCI_NETWORK_ISDN 0x0204`

- #define **PCI\_NETWORK\_WORLDIFIP** 0x0205
- #define **PCI\_NETWORK\_PICGMG** 0x0206
- #define **PCI\_NETWORK\_OTHER** 0x0280
- #define **PCI\_DISPLAY\_VGA** 0x0300
- #define **PCI\_DISPLAY\_XGA** 0x0301
- #define **PCI\_DISPLAY\_3D** 0x0302
- #define **PCI\_DISPLAY\_OTHER** 0x0380
- #define **PCI\_MULTIMEDIA\_VIDEO** 0x0400
- #define **PCI\_MULTIMEDIA\_AUDIO** 0x0401
- #define **PCI\_MULTIMEDIA\_PHONE** 0x0402
- #define **PCI\_MULTIMEDIA\_AUDIO\_DEVICE** 0x0403
- #define **PCI\_MULTIMEDIA\_OTHER** 0x0480
- #define **PCI\_MEMORY\_RAM** 0x0500
- #define **PCI\_MEMORY\_FLASH** 0x0501
- #define **PCI\_MEMORY\_OTHER** 0x0580
- #define **PCI\_BRIDGE\_HOST** 0x0600
- #define **PCI\_BRIDGE\_ISA** 0x0601
- #define **PCI\_BRIDGE\_EISA** 0x0602
- #define **PCI\_BRIDGE\_MCA** 0x0603
- #define **PCI\_BRIDGE\_PCI** 0x0604
- #define **PCI\_BRIDGE\_PCMCIA** 0x0605
- #define **PCI\_BRIDGE\_NUBUS** 0x0606
- #define **PCI\_BRIDGE\_CARDBUS** 0x0607
- #define **PCI\_BRIDGE\_RACEWAY** 0x0608
- #define **PCI\_BRIDGE\_OTHER** 0x0680
- #define **PCI\_COMM\_SERIAL** 0x0700
- #define **PCI\_COMM\_PARALLEL** 0x0701
- #define **PCI\_COMM\_MULTIPOINT** 0x0702
- #define **PCI\_COMM\_MODEM** 0x0703
- #define **PCI\_COMM\_GPIB** 0x0704
- #define **PCI\_COMM\_SMARTCARD** 0x0705
- #define **PCI\_COMM\_OTHER** 0x0780
- #define **PCI\_SYSTEM\_PIC** 0x0800
- #define **PCI\_SYSTEM\_DMA** 0x0801
- #define **PCI\_SYSTEM\_TIMER** 0x0802
- #define **PCI\_SYSTEM\_RTC** 0x0803
- #define **PCI\_SYSTEM\_PCI\_HOTPLUG** 0x0804
- #define **PCI\_SYSTEM\_SD** 0x0805
- #define **PCI\_SYSTEM\_OTHER** 0x0880
- #define **PCI\_INPUT\_KEYBOARD** 0x0900
- #define **PCI\_INPUT\_PEN** 0x0901
- #define **PCI\_INPUT\_MOUSE** 0x0902
- #define **PCI\_INPUT\_SCANNER** 0x0903
- #define **PCI\_INPUT\_GAMEPORT** 0x0904
- #define **PCI\_INPUT\_OTHER** 0x0980
- #define **PCI\_DOCKING\_GENERIC** 0x0a00
- #define **PCI\_DOCKING\_OTHER** 0x0a80
- #define **PCI\_PROCESSOR\_386** 0x0b00
- #define **PCI\_PROCESSOR\_486** 0x0b01
- #define **PCI\_PROCESSOR\_PENTIUM** 0x0b02
- #define **PCI\_PROCESSOR\_ALPHA** 0x0b10
- #define **PCI\_PROCESSOR\_POWERPC** 0x0b20
- #define **PCI\_PROCESSOR\_MIPS** 0x0b30
- #define **PCI\_PROCESSOR\_CO** 0x0b40
- #define **PCI\_SERIAL\_FIREWIRE** 0x0c00

- #define **PCI\_SERIAL\_ACCESS** 0x0c01
- #define **PCI\_SERIAL\_SSA** 0x0c02
- #define **PCI\_SERIAL\_USB** 0x0c03
- #define **PCI\_SERIAL\_FIBER** 0x0c04
- #define **PCI\_SERIAL\_SMBUS** 0x0c05
- #define **PCI\_SERIAL\_USB\_UHCI** 0x00
- #define **PCI\_SERIAL\_USB\_OHCI** 0x10
- #define **PCI\_SERIAL\_USB\_EHCI** 0x20
- #define **PCI\_SERIAL\_USB\_XHCI** 0x30
- #define **PCI\_SERIAL\_USB\_OTHER** 0x80
- #define **PCI\_WIRELESS\_IRDA** 0x0d00
- #define **PCI\_WIRELESS\_IR** 0x0d01
- #define **PCI\_WIRELESS\_RF** 0x0d10
- #define **PCI\_WIRELESS\_BLUETOOTH** 0x0d11
- #define **PCI\_WIRELESS\_BROADBAND** 0x0d12
- #define **PCI\_WIRELESS\_ETHERNET\_A** 0x0d20
- #define **PCI\_WIRELESS\_ETHERNET\_B** 0x0d21
- #define **PCI\_WIRELESS\_OTHER** 0x0d80
- #define **PCI\_INTELLIGENT\_I2O** 0x0e00
- #define **PCI\_SATELLITE\_TV** 0x0f00
- #define **PCI\_SATELLITE\_AUDIO** 0x0f01
- #define **PCI\_SATELLITE\_VOICE** 0x0f03
- #define **PCI\_SATELLITE\_DATA** 0x0f04
- #define **PCI\_CRYPT\_NETWORK** 0x1000
- #define **PCI\_CRYPT\_ENTERTAINMENT** 0x1001
- #define **PCI\_CRYPT\_OTHER** 0x1080
- #define **PCI\_SP\_DPIO** 0x1100
- #define **PCI\_SP\_OTHER** 0x1180

## Functions

- void **pci\_init** (void)
- void **pci\_list** (void)
- const char \* **pci\_device\_id\_to\_string** ([pci\\_device\\_t](#) \*device)
- uint8\_t **pci\_inb** (uint32\_t id, uint32\_t offset)
- void **pci\_outb** (uint32\_t id, uint32\_t offset, uint8\_t data)
- uint16\_t **pci\_inw** (uint32\_t id, uint32\_t offset)
- void **pci\_outw** (uint32\_t id, uint32\_t offset, uint16\_t data)
- uint32\_t **pci\_ind** (uint32\_t id, uint32\_t offset)
- void **pci\_outd** (uint32\_t id, uint32\_t offset, uint32\_t data)
- void **pci\_get\_bar** ([pci\\_bar\\_t](#) \*bar, uint32\_t id, uint32\_t index)

### 5.85.1 Detailed Description

Definition of PCI related data structures and functions.

---

The PCI (Peripheral Component Interconnect) bus was defined to establish a high performance and low cost local bus that would remain through several generations of products.

---

### 5.85.2 Macro Definition Documentation

### 5.85.2.1 PCI\_MAKE\_DEVICE\_ID

```
#define PCI_MAKE_DEVICE_ID(
 dev)
```

Value:

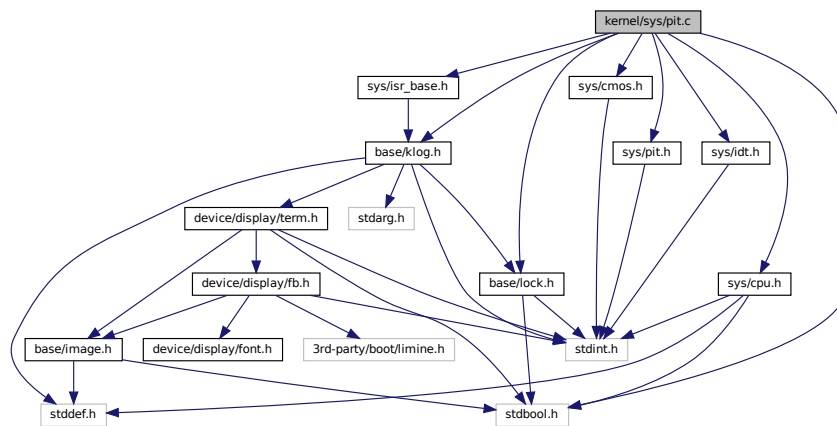
```
((dev->bus) << 16) |
((dev->device) << 11) |
((dev->func) << 8))
```

## 5.86 kernel/sys/pit.c File Reference

Implementation of PIT related functions.

```
#include <stdbool.h>
#include <sys/pit.h>
#include <sys/cpu.h>
#include <sys/idt.h>
#include <sys/cmos.h>
#include <sys/isr_base.h>
#include <base/lock.h>
#include <base/klog.h>
```

Include dependency graph for pit.c:



### Macros

- `#define PIT_FREQ_HZ 1000`

### Functions

- `uint64_t pit_get_ticks (void)`
- `void pit_init (void)`
- `void pit_wait (uint64_t ms)`

### 5.86.1 Detailed Description

Implementation of PIT related functions.

This PIT wait function should be rewritten based on IRQ mechanism, then it can be used in different processes at the same time.

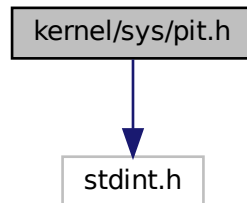
The PIT wait should only be used in system initialization stage. It will not be re-written after re-considering it's usage.

## 5.87 kernel/sys/pit.h File Reference

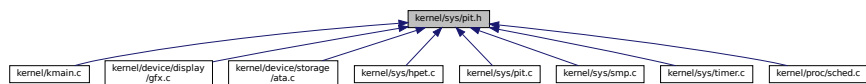
Definition of PIT related data structures.

```
#include <stdint.h>
```

Include dependency graph for pit.h:



This graph shows which files directly or indirectly include this file:



### Functions

- void **pit\_init** (void)
- void **pit\_wait** (uint64\_t ms)
- uint64\_t **pit\_get\_ticks** (void)

#### 5.87.1 Detailed Description

Definition of PIT related data structures.

---

The Programmable Interval Timer (PIT) chip (Intel 8253/8254) basically consists of an oscillator, a prescaler and 3 independent frequency dividers. Each frequency divider has an output, which is used to allow the timer to control external circuitry (for example, IRQ 0).

---

## 5.88 kernel/sys/serial.c File Reference

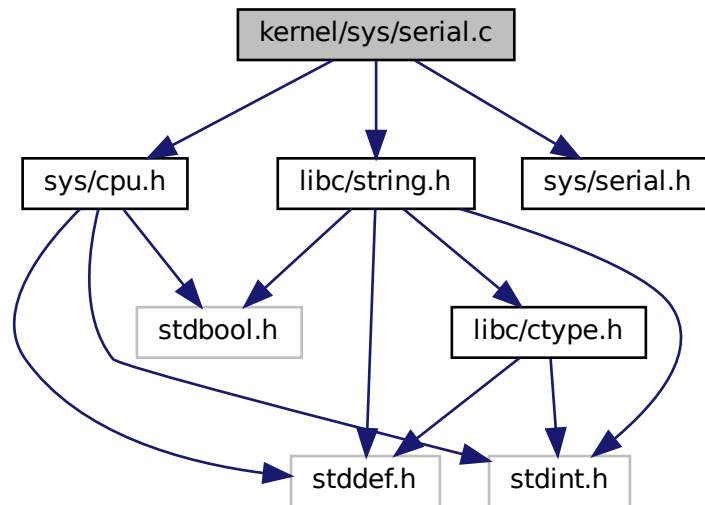
Implementation of serial port communication functions.

```
#include <sys/cpu.h>
```

```
#include <sys/serial.h>
```

```
#include <libc/string.h>
```

Include dependency graph for serial.c:



## Macros

- `#define BAUD_RATE 38400`

## Functions

- `void serial_init ()`
- `void serial_write (char a)`
- `void serial_puts (char *s)`

### 5.88.1 Detailed Description

Implementation of serial port communication functions.

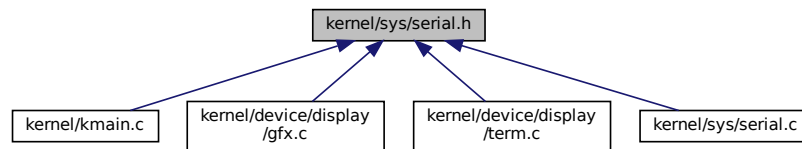
This file contains the implementation of functions to initialize and use the serial port for communication in the HanOS kernel. It sets up the serial port with the specified baud rate and configuration, and provides functions to write individual characters and strings to the serial port. The initialization function ensures the serial port is properly configured before any data is sent.

## 5.89 kernel/sys/serial.h File Reference

Definitions for serial port communication.



This graph shows which files directly or indirectly include this file:



## Macros

- `#define SERIAL_PORT 0x3F8`

## Functions

- void **serial\_init** ()
- void **serial\_write** (char a)
- void **serial\_puts** (char \*s)

### 5.89.1 Detailed Description

Definitions for serial port communication.

---

This file contains the function declarations and constants used for serial port communication within the HanOS kernel. It includes the initialization function for the serial port, and functions for writing individual characters and strings to the serial port. The serial port base address is also defined.

---

## 5.90 kernel/sys/smp.c File Reference

SMP initialization and other support functions.

```

#include <stddef.h>
#include <kconfig.h>
#include <libc/string.h>
#include <base/klog.h>
#include <base/kmalloc.h>
#include <base/time.h>
#include <mm/mm.h>
#include <sys/cpu.h>
#include <sys/smp.h>
#include <sys/gdt.h>
#include <sys/hpet.h>
#include <sys/madt.h>
#include <sys/apic.h>
#include <sys/pit.h>
#include <proc/syscall.h>
#include <proc/sched.h>

```

---



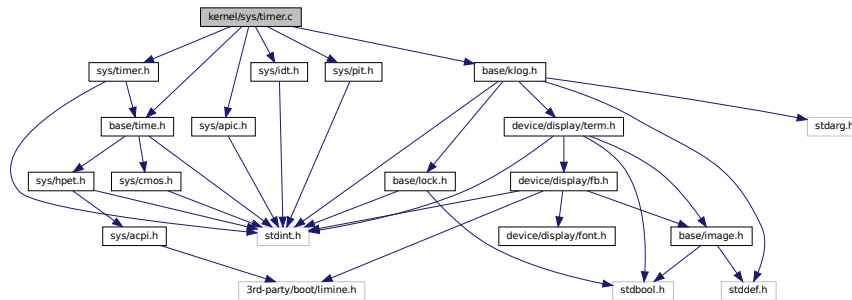


## 5.92 kernel/sys/timer.c File Reference

Implementation of APIC timer related functions.

```
#include <sys/timer.h>
#include <sys/apic.h>
#include <sys/idt.h>
#include <sys/pit.h>
#include <base/klog.h>
#include <base/time.h>
```

Include dependency graph for timer.c:



### Functions

- void **apic\_timer\_handler** (void \*v)
- void **apic\_timer\_stop** (void)
- void **apic\_timer\_start** (void)
- void **apic\_timer\_set\_handler** (void(\*h)(void \*))
- void **apic\_timer\_set\_frequency** (uint64\_t freq)
- void **apic\_timer\_set\_period** (time\_t tv)
- uint8\_t **apic\_timer\_get\_vector** (void)
- void **apic\_timer\_set\_mode** (apic\_timer\_mode\_t mode)
- void **apic\_timer\_enable** (void)
- void **apic\_timer\_init** (void)

### 5.92.1 Detailed Description

Implementation of APIC timer related functions.

The timer has 2 or 3 modes. The first 2 modes (periodic and one-shot) are supported by all local APICs. The third mode (TSC-Deadline mode) is an extension that is only supported on recent CPUs.

Periodic Mode:

- Generate IRQs at a fixed rate depending on the initial count.

One-Shot Mode:

- Decrement the current count (and generates a timer IRQ when the count reaches zero) in the same way as in periodic mode. However it doesn't reset the current count to the initial count when the current count reaches zero.

TSC-Deadline mode:

- Similar with one-shot mode but using CPU's time stamp counter instead to get higher precision.

Ref: [https://wiki.osdev.org/APIC\\_timer](https://wiki.osdev.org/APIC_timer)

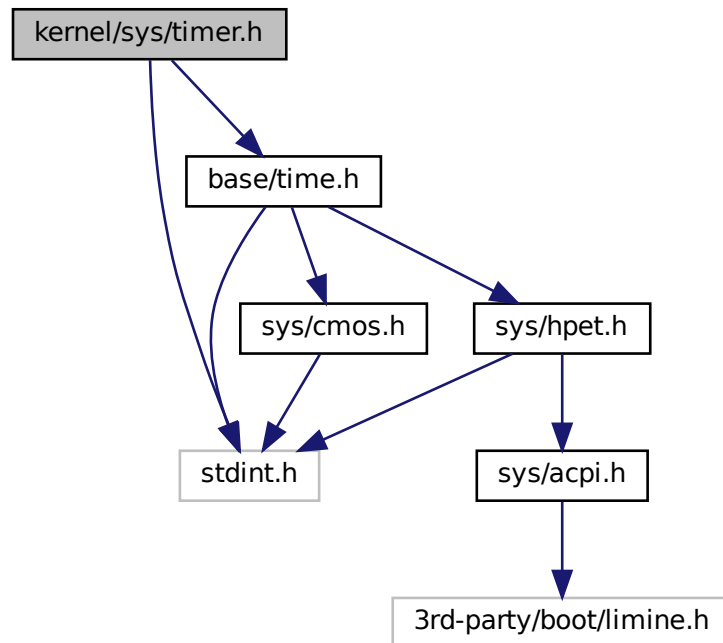
## 5.93 kernel/sys/timer.h File Reference

Definition of APIC timer related macros, data structures, functions.

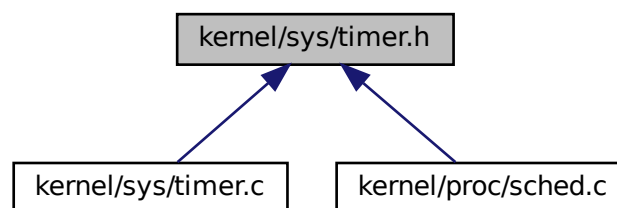
```
#include <stdint.h>
```

```
#include <base/time.h>
```

Include dependency graph for timer.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define APIC_REG_TIMER_LVT 0x320`
- `#define APIC_REG_TIMER_ICR 0x380`
- `#define APIC_REG_TIMER_CCR 0x390`
- `#define APIC_REG_TIMER_DCR 0x3e0`

- `#define APIC_TIMER_FLAG_PERIODIC (1 << 17)`
- `#define APIC_TIMER_FLAG_MASKED (1 << 16)`

## Enumerations

- `enum apic_timer_mode_t { APIC_TIMER_MODE_PERIODIC, APIC_TIMER_MODE_ONESHOT }`

## Functions

- `void apic_timer_init (void)`
- `void apic_timer_enable (void)`
- `void apic_timer_stop (void)`
- `void apic_timer_start (void)`
- `void apic_timer_set_handler (void(*h)(void *))`
- `void apic_timer_set_frequency (uint64_t freq)`
- `void apic_timer_set_period (time_t tv)`
- `void apic_timer_set_mode (apic_timer_mode_t mode)`
- `uint8_t apic_timer_get_vector (void)`

### 5.93.1 Detailed Description

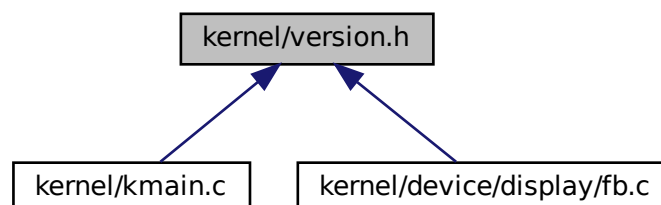
Definition of APIC timer related macros, data structures, functions.

The great benefit of the Local APIC timer is that it is hardwired to each CPU core, unlike the Programmable Interval Timer which is a separate circuit. Because of this, there is no need for any resource management, which makes things easier. The downside is that it's oscillating at (one of) the CPU's frequencies, which varies from machine to machine, while the PIT uses a standard frequency (1,193,182 Hz). To make use of it, you have to know how many interrupts/sec it's capable of.

## 5.94 kernel/version.h File Reference

Definition of version information.

This graph shows which files directly or indirectly include this file:



## Macros

- `#define major 0`
- `#define minor 1`
- `#define patch 2`
- `#define ver(arg) #arg`
- `#define ver2(arg) ver(arg)`

- `#define major2 ver2(major)`
- `#define minor2 ver2(minor)`
- `#define patch2 ver2(patch)`
- `#define dot "."`
- `#define VERSION (major2 dot minor2 dot patch2)`

### 5.94.1 Detailed Description

#### Definition of version information.

---

This file defines the version information for the HanOS Kernel. The version information is broken down into major, minor, and patch levels. These values are combined to form a version string that can be used to identify the specific version of the kernel.

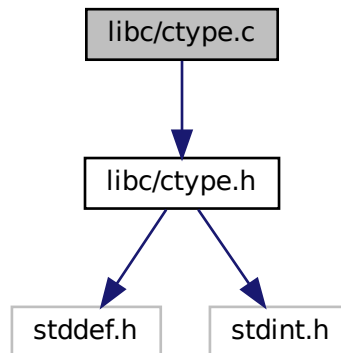
---

## 5.95 libc/ctype.c File Reference

Implementation of character type and conversion functions.

```
#include <libc/ctype.h>
```

Include dependency graph for ctype.c:



### Functions

- `int isalnum (int c)`
- `int isalpha (int c)`
- `int isblank (int c)`
- `int iscntrl (int c)`
- `int isdigit (int c)`
- `int isgraph (int c)`
- `int islower (int c)`
- `int isprint (int c)`
- `int ispunct (int c)`
- `int isspace (int c)`
- `int isupper (int c)`
- `int isxdigit (int c)`
- `int tolower (int c)`
- `int toupper (int c)`

### 5.95.1 Detailed Description

Implementation of character type and conversion functions.

This file contains the implementation of functions for character type checking and conversion in the HanOS standard library. The functions include checks for alphanumeric characters, alphabetic characters, blank characters, control characters, digits, graphical characters, lowercase characters, printable characters, punctuation, whitespace characters, uppercase characters, and hexadecimal digits. Additionally, it provides functions to convert characters to lowercase and uppercase.

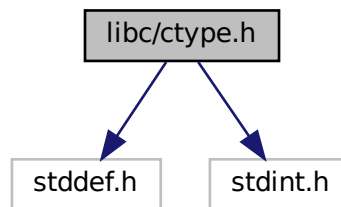
## 5.96 libc/ctype.h File Reference

Character type and conversion functions for HanOS standard library.

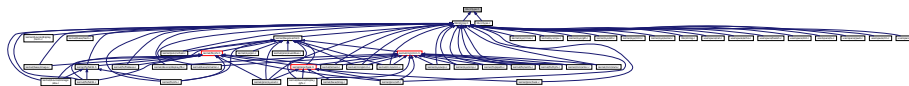
```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for ctype.h:



This graph shows which files directly or indirectly include this file:



### Macros

- `#define KB ((uint64_t)1024)`
- `#define MB (((uint64_t)1024 * KB))`
- `#define GB (((uint64_t)1024 * MB))`
- `#define TB (((uint64_t)1024 * GB))`

### Functions

- `int isalnum (int c)`
- `int isalpha (int c)`
- `int isblank (int c)`
- `int iscntrl (int c)`
- `int isdigit (int c)`
- `int isgraph (int c)`
- `int islower (int c)`
- `int isprint (int c)`
- `int ispunct (int c)`



- int **isspace** (int c)
- int **isupper** (int c)
- int **isxdigit** (int c)
- int **tolower** (int c)
- int **toupper** (int c)

### 5.96.1 Detailed Description

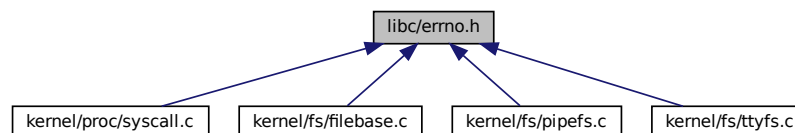
Character type and conversion functions for HanOS standard library.

This file contains the declarations of functions used for character type checking and conversion in the HanOS standard library. It includes functions to check if a character is alphanumeric, alphabetic, blank, control, digit, graphical, lowercase, printable, punctuation, whitespace, uppercase, or hexadecimal digit. Additionally, it provides functions to convert characters to lowercase and uppercase.

## 5.97 libc/errno.h File Reference

Error number definitions for HanOS standard library.

This graph shows which files directly or indirectly include this file:



### Macros

- **#define EDOM** 1 /\* Math argument out of domain of func \*/
- **#define EILSEQ** 2 /\* Illegal byte sequence \*/
- **#define ERANGE** 3 /\* Math result not representable \*/
- **#define E2BIG** 1001 /\* Arg list too long \*/
- **#define EACCES** 1002 /\* Permission denied \*/
- **#define EADDRINUSE** 1003 /\* Address already in use \*/
- **#define EADDRNOTAVAIL** 1004 /\* Cannot assign requested address \*/
- **#define EAFNOSUPPORT** 1005 /\* Address family not supported \*/
- **#define EAGAIN** 1006 /\* Try again \*/
- **#define EALREADY** 1007 /\* Operation already in progress \*/
- **#define EBADF** 1008 /\* Bad file descriptor \*/
- **#define EBADMSG** 1009 /\* Not a data message \*/
- **#define EBUSY** 1010 /\* Device or resource busy \*/
- **#define ECANCELED** 1011
- **#define ECHILD** 1012 /\* No child processes \*/
- **#define ECONNABORTED** 1013 /\* Software caused connection abort \*/
- **#define ECONNREFUSED** 1014 /\* Connection refused \*/
- **#define ECONNRESET** 1015 /\* Connection reset by peer \*/
- **#define EDEADLK** 1016 /\* Resource deadlock would occur \*/
- **#define EDESTADDRREQ** 1017 /\* Destination address required \*/
- **#define EDQUOT** 1018 /\* Quota exceeded \*/
- **#define EEXIST** 1019 /\* File already exists \*/
- **#define EFAULT** 1020 /\* Bad address \*/

- `#define EFBIG 1021 /* File too large */`
- `#define EHOSTUNREACH 1022 /* No route to host */`
- `#define EIDRM 1023 /* Identifier removed */`
- `#define EINPROGRESS 1024 /* Operation now in progress */`
- `#define EINTR 1025 /* Interrupted system call */`
- `#define EINVAL 1026 /* Invalid argument */`
- `#define EIO 1027 /* I/O error */`
- `#define EISCONN 1028 /* Transport endpoint is already connected */`
- `#define EISDIR 1029 /* Is a directory */`
- `#define ELOOP 1030 /* Too many symbolic links encountered */`
- `#define EMFILE 1031 /* Too many open files */`
- `#define EMLINK 1032 /* Too many links */`
- `#define EMSGSIZE 1034 /* Message too long */`
- `#define EMULTIHOP 1035 /* Multihop attempted */`
- `#define ENAMETOOLONG 1036 /* File name too long */`
- `#define ENETDOWN 1037 /* Network is down */`
- `#define ENETRESET`
- `#define ENETUNREACH 1039 /* Network is unreachable */`
- `#define ENFILE 1040 /* File table overflow */`
- `#define ENOBUFS 1041 /* No buffer space available */`
- `#define ENODEV 1042 /* No such device */`
- `#define ENOENT 1043 /* No such file or directory */`
- `#define ENOEXEC 1044 /* Exec format error */`
- `#define ENOLCK 1045 /* No record locks available */`
- `#define ENOLINK 1046 /* No links */`
- `#define ENOMEM 1047 /* Out of memory */`
- `#define ENOMSG 1048 /* No message of desired type */`
- `#define ENOPROTOPT 1049 /* Protocol not available */`
- `#define ENOSPC 1050 /* No space left on device */`
- `#define ENOSYS 1051 /* Function not implemented */`
- `#define ENOTCONN 1052 /* Transport endpoint is not connected */`
- `#define ENOTDIR 1053 /* Not a directory */`
- `#define ENOTEMPTY 1054 /* Directory not empty */`
- `#define ENOTRECOVERABLE 1055`
- `#define ENOTSOCK 1056 /* Argument is not a socket */`
- `#define ENOTSUP 1057`
- `#define ENOTTY 1058 /* Not a typewriter */`
- `#define ENXIO 1059 /* No such device or address */`
- `#define EOPNOTSUPP`
- `#define EOVERFLOW 1061 /* Value too large for defined data type */`
- `#define EOWNERDEAD 1062`
- `#define EPERM 1063 /* Operation not permitted */`
- `#define EPIPE 1064 /* Broken pipe */`
- `#define EPROTO 1065 /* Protocol error */`
- `#define EPROTONOSUPPORT 1066 /* Protocol not supported */`
- `#define EPROTOTYPE 1067 /* Protocol wrong type for socket */`
- `#define EROFS 1068 /* Read-only file system */`
- `#define ESPIPE 1069 /* Illegal seek */`
- `#define ESRCH 1070 /* No such process */`
- `#define ESTALE 1071 /* Stale NFS file handle */`
- `#define ETIMEDOUT 1072 /* Connection timed out */`
- `#define ETXTBSY 1073 /* Text file busy */`
- `#define EWOULDBLOCK EAGAIN /* Operation would block */`
- `#define EXDEV 1075 /* Cross-device link */`
- `#define ENODATA 1076 /* No data available */`

- `#define ETIME 1077 /* Timer expired */`
- `#define ENOKEY 1078`
- `#define ESHUTDOWN 1079`
- `#define EHOSTDOWN 1080`
- `#define EBADFD 1081 /* File descriptor in bad state */`
- `#define ENOMEDIUM 1082 /* No medium found */`
- `#define ENOTBLK 1083 /* Block device required */`

### 5.97.1 Detailed Description

Error number definitions for HanOS standard library.

---

This file defines a set of error numbers (`errno`) used in the HanOS standard library. These error numbers represent various error conditions that might be encountered during system and library calls. Each error number is associated with a specific type of error, such as file not found, permission denied, or out of memory. These definitions help standardize error handling across the operating system and its libraries.

---

### 5.97.2 Macro Definition Documentation

#### 5.97.2.1 ENETRESET

```
#define ENETRESET
```

**Value:**

```
1038 /* Network dropped connection because of
 * reset */
```

#### 5.97.2.2 EOPNOTSUPP

```
#define EOPNOTSUPP
```

**Value:**

```
1060 /* Operation not supported on transport
 * endpoint */
```

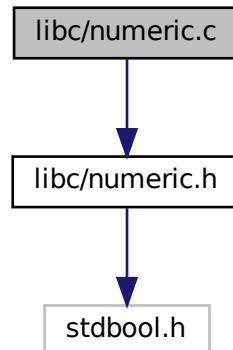
## 5.98 libc/numeric.c File Reference

Implementation of numeric utility functions.

---

```
#include <libc/numeric.h>
```

Include dependency graph for numeric.c:



## Functions

- bool **itoa** (int num, char \*str, int len, int base)
- int **rand** (int seed, int min, int max)

### 5.98.1 Detailed Description

Implementation of numeric utility functions.

---

This file contains the implementation of utility functions for numeric operations in the HanOS standard library. It includes the implementation of the function to convert an integer to a string representation (`itoa`), a helper function to reverse a string, and a function to generate a random integer within a specified range based on a seed value using a linear congruential generator (`rand`).

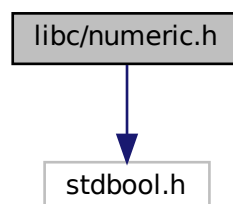
---

## 5.99 libc/numeric.h File Reference

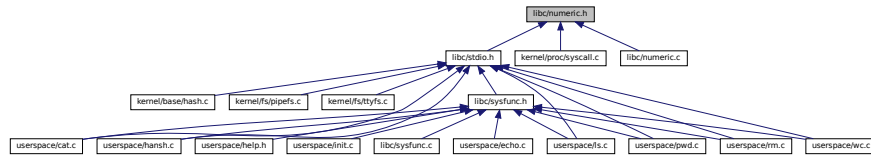
Numeric utility functions for HanOS standard library.

```
#include <stdbool.h>
```

Include dependency graph for numeric.h:



This graph shows which files directly or indirectly include this file:



## Functions

- bool **itoa** (int num, char \*str, int len, int base)
- int **rand** (int seed, int min, int max)

### 5.99.1 Detailed Description

## Numeric utility functions for HanOS standard library.

This file contains the declarations of utility functions for numeric operations in the HanOS standard library. It includes the function to convert an integer to a string representation (itoa) and a function to generate a random integer within a specified range based on a seed value.

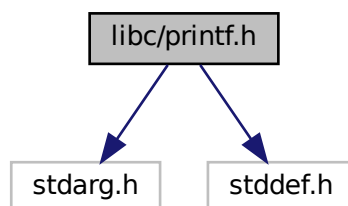
## 5.100 libc/printf.h File Reference

printf, sprintf and snprintf implementation for HanOS standard library

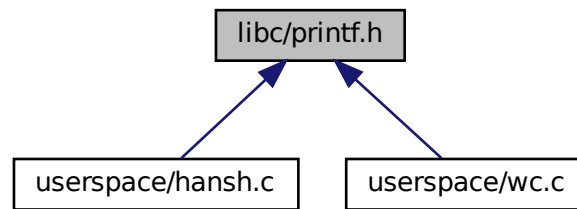
```
#include <stdarg.h>
```

```
#include <stddef.h>
```

Include dependency graph for printf.h:



This graph shows which files directly or indirectly include this file:



## Macros

- `#define printf_ printf`
- `#define sprintf_ sprintf_`
- `#define snprintf_ snprintf_`
- `#define vsnprintf_ vsnprintf_`
- `#define vprintf_ vprintf_`

## Functions

- `void _putchar` (char character)
- `int printf_` (const char \*format,...)
- `int sprintf_` (char \*buffer, const char \*format,...)
- `int snprintf_` (char \*buffer, uint64\_t count, const char \*format,...)
- `int vsnprintf_` (char \*buffer, uint64\_t count, const char \*format, va\_list va)
- `int vprintf_` (const char \*format, va\_list va)
- `int fctprintf` (void(\*out)(char character, void \*arg), void \*arg, const char \*format,...)

### 5.100.1 Detailed Description

printf, sprintf and snprintf implementation for HanOS standard library

---

### 5.100.2 Macro Definition Documentation

#### 5.100.2.1 printf\_

```
#define printf_ printf
```

Tiny printf implementation You have to implement `putchar` if you use `printf()` To avoid conflicts with the regular `printf()` API it is overridden by macro defines and internal underscore-appended functions like `printf_()` are used

#### Parameters

|               |                                                  |
|---------------|--------------------------------------------------|
| <i>format</i> | A string that specifies the format of the output |
|---------------|--------------------------------------------------|

#### Returns

The number of characters that are written into the array, not counting the terminating null character

### 5.100.2.2 snprintf

```
#define snprintf snprintf_
Tiny snprintf/vsnprintf implementation
```

#### Parameters

|               |                                                                                                 |
|---------------|-------------------------------------------------------------------------------------------------|
| <i>buffer</i> | A pointer to the buffer where to store the formatted string                                     |
| <i>count</i>  | The maximum number of characters to store in the buffer, including a terminating null character |
| <i>format</i> | A string that specifies the format of the output                                                |
| <i>va</i>     | A value identifying a variable arguments list                                                   |

#### Returns

The number of characters that COULD have been written into the buffer, not counting the terminating null character. A value equal or larger than count indicates truncation. Only when the returned value is non-negative and less than count, the string has been completely written.

### 5.100.2.3 sprintf

```
#define sprintf sprintf_
Tiny sprintf implementation Due to security reasons (buffer overflow) YOU SHOULD CONSIDER USING (V)SNP↔
PRINTF INSTEAD!
```

#### Parameters

|               |                                                                                                      |
|---------------|------------------------------------------------------------------------------------------------------|
| <i>buffer</i> | A pointer to the buffer where to store the formatted string. MUST be big enough to store the output! |
| <i>format</i> | A string that specifies the format of the output                                                     |

#### Returns

The number of characters that are WRITTEN into the buffer, not counting the terminating null character

### 5.100.2.4 vprintf

```
#define vprintf vprintf_
Tiny vprintf implementation
```

#### Parameters

|               |                                                  |
|---------------|--------------------------------------------------|
| <i>format</i> | A string that specifies the format of the output |
| <i>va</i>     | A value identifying a variable arguments list    |

**Returns**

The number of characters that are WRITTEN into the buffer, not counting the terminating null character

**5.100.3 Function Documentation****5.100.3.1 `_putchar()`**

```
void _putchar (
 char character)
```

Output a character to a custom device like UART, used by the `printf()` function This function is declared here only. You have to write your custom implementation somewhere

**Parameters**

|                  |                     |
|------------------|---------------------|
| <i>character</i> | Character to output |
|------------------|---------------------|

**5.100.3.2 `fctprintf()`**

```
int fctprintf (
 void(*) (char character, void *arg) out,
 void * arg,
 const char * format,
 ...)
```

`printf` with output function You may use this as dynamic alternative to `printf()` with its fixed `_putchar()` output

**Parameters**

|               |                                                                      |
|---------------|----------------------------------------------------------------------|
| <i>out</i>    | An output function which takes one character and an argument pointer |
| <i>arg</i>    | An argument pointer for user data passed to output function          |
| <i>format</i> | A string that specifies the format of the output                     |

**Returns**

The number of characters that are sent to the output function, not counting the terminating null character

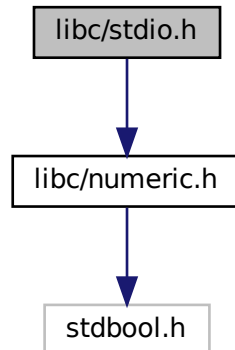
**5.101 `libc/stdio.h` File Reference**

Standard I/O definitions and declarations for HanOS standard library.

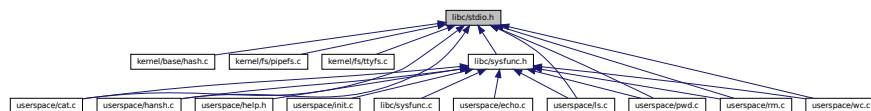


```
#include <libc/numeric.h>
```

Include dependency graph for stdio.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [timespec\\_t](#)
- struct [dirent\\_t](#)
- struct [stat\\_t](#)

## Macros

- #define **STDIN** 0
- #define **STDOUT** 1
- #define **STDERR** 2
- #define **EOF** -1
- #define **DT\_UNKNOWN** 0
- #define **DT\_FIFO** 1
- #define **DT\_CHR** 2
- #define **DT\_DIR** 4
- #define **DT\_BLK** 6
- #define **DT\_REG** 8
- #define **DT\_LNK** 10
- #define **DT SOCK** 12
- #define **DT\_WHT** 14
- #define **S\_IFMT** 0170000 /\* bit mask for the file type bit field \*/
- #define **S\_IFSOCK** 0140000 /\* socket \*/
- #define **S\_IFLNK** 0120000 /\* symbolic link \*/
- #define **S\_IFREG** 0100000 /\* regular file \*/

- `#define S_IFBLK 0060000 /* block device */`
- `#define S_IFDIR 0040000 /* directory */`
- `#define S_IFCHR 0020000 /* character device */`
- `#define S_IFIFO 0010000 /* FIFO */`

## Functions

- `int fprintf (int fd, const char *fmt,...)`
- `int printf (const char *fmt,...)`

### 5.101.1 Detailed Description

Standard I/O definitions and declarations for HanOS standard library.

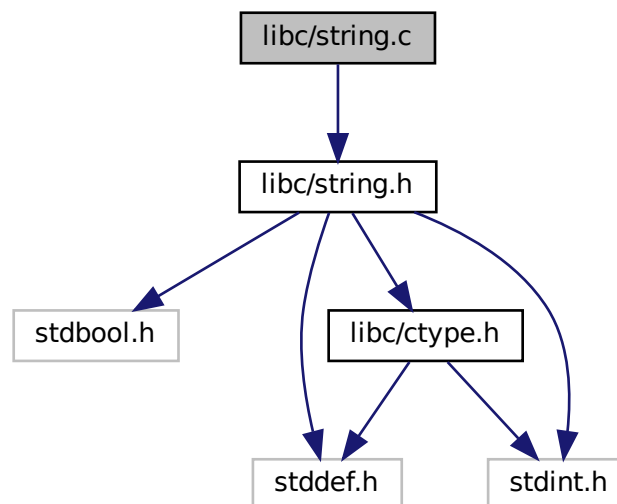
This file provides the declarations for standard input/output functions and definitions for file system structures and constants used in HanOS. It includes definitions for standard file descriptors (STDIN, STDOUT, STDERR), the end-of-file marker (EOF), and file system structures such as `timespec_t`, `dirent_t`, and `stat_t`. Additionally, it defines various file types and modes.

## 5.102 libc/string.c File Reference

Implementation of various string manipulation functions.

`#include <libc/string.h>`

Include dependency graph for string.c:



## Functions

- `void * memcpy (void *dest, const void *src, uint64_t len)`
- `void memset (void *addr, uint8_t val, uint64_t len)`
- `bool memcmp (const void *s1, const void *s2, uint64_t len)`
- `int64_t strlen (const char *s)`
- `int64_t strcmp (const char *a, const char *b)`
- `int64_t strncmp (const char *a, const char *b, uint64_t len)`

- char \* **strcpy** (char \*\_\_restrict dest, const char \*src)
- char \* **strncpy** (char \*\_\_restrict dest, const char \*src, uint64\_t len)
- int64\_t **strcat** (char \*dest, const char \*src)
- int64\_t **strncat** (char \*dest, const char \*src, uint64\_t dest\_size)
- uint64\_t **strtol** (char \*s, num\_sys\_t type)
- char \* **strchr** (const char \*s, int c)
- char \* **strlwr** (char \*s)
- char \* **strupr** (char \*s)

### 5.102.1 Detailed Description

Implementation of various string manipulation functions.

String functions including length, comparison, copy etc.

## 5.103 libc/string.h File Reference

Definition of string operation related functions.

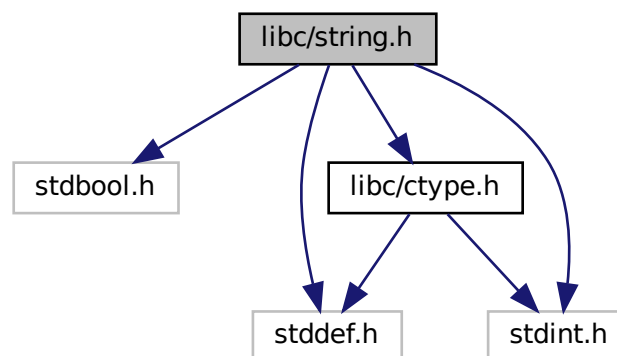
```
#include <stdbool.h>
```

```
#include <stddef.h>
```

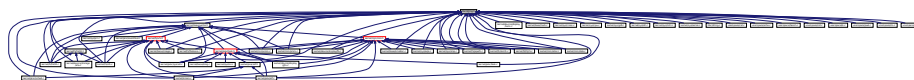
```
#include <stdint.h>
```

```
#include <libc/ctype.h>
```

Include dependency graph for string.h:



This graph shows which files directly or indirectly include this file:



### Enumerations

- enum **num\_sys\_t** { **OCT**, **DEC** }

## Functions

- bool **memcmp** (const void \*s1, const void \*s2, uint64\_t len)
- void **memset** (void \*addr, uint8\_t val, uint64\_t len)
- void \* **memcpy** (void \*dest, const void \*src, uint64\_t len)
- char \* **strcpy** (char \*\_\_restrict dest, const char \*src)
- char \* **strncpy** (char \*\_\_restrict dest, const char \*src, uint64\_t len)
- int64\_t **strlen** (const char \*s)
- int64\_t **strcmp** (const char \*a, const char \*b)
- int64\_t **strncmp** (const char \*a, const char \*b, uint64\_t len)
- int64\_t **strcat** (char \*dest, const char \*src)
- int64\_t **strncat** (char \*dest, const char \*src, uint64\_t dest\_size)
- char \* **strchr** (const char \*s, int c)
- uint64\_t **strtol** (char \*s, num\_sys\_t type)
- char \* **strlwr** (char \*s)
- char \* **strupr** (char \*s)

### 5.103.1 Detailed Description

Definition of string operation related functions.

---

String functions including copying, concatenation, comparison, search etc.

---

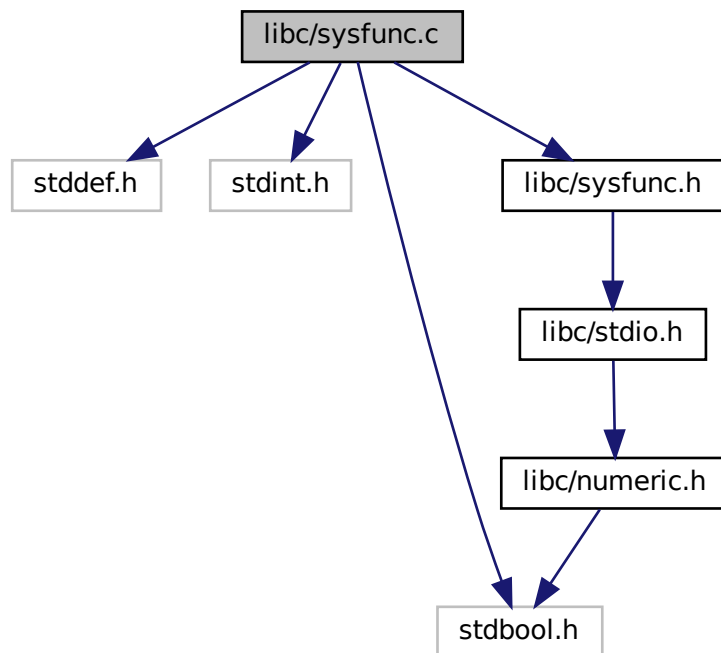
## 5.104 libc/sysfunc.c File Reference

Implementation of system call functions.

```
#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>
#include <libc/sysfunc.h>
```

---

Include dependency graph for sysfunc.c:



## Macros

- `#define SYSCALL0(NUM)`
- `#define SYSCALL1(NUM, ARG0)`
- `#define SYSCALL2(NUM, ARG0, ARG1)`
- `#define SYSCALL3(NUM, ARG0, ARG1, ARG2)`
- `#define SYSCALL4(NUM, ARG0, ARG1, ARG2, ARG3)`
- `#define SYSCALL6(NUM, ARG0, ARG1, ARG2, ARG3, ARG4, ARG5)`
- `#define SYSCALL_DEBUGLOG 0`
- `#define SYSCALL_MMAP 1`
- `#define SYSCALL_OPENAT 2`
- `#define SYSCALL_READ 3`
- `#define SYSCALL_WRITE 4`
- `#define SYSCALL_SEEK 5`
- `#define SYSCALL_CLOSE 6`
- `#define SYSCALL_SET_FS_BASE 7`
- `#define SYSCALL_IOCTL 8`
- `#define SYSCALL_GETPID 9`
- `#define SYSCALL_CHDIR 10`
- `#define SYSCALL_MKDIRAT 11`
- `#define SYSCALL_SOCKET 12`
- `#define SYSCALL_BIND 13`
- `#define SYSCALL_FORK 14`
- `#define SYSCALL_EXECVE 15`
- `#define SYSCALL_FACCESSAT 16`
- `#define SYSCALL_FSTATAT 17`

- `#define SYSCALL_FSTAT` 18
- `#define SYSCALL_GETPPID` 19
- `#define SYSCALL_FCNTL` 20
- `#define SYSCALL_DUP3` 21
- `#define SYSCALL_WAITPID` 22
- `#define SYSCALL_EXIT` 23
- `#define SYSCALL_READDIR` 24
- `#define SYSCALL_MUNMAP` 25
- `#define SYSCALL_GETCWD` 26
- `#define SYSCALL_GETCLOCK` 27
- `#define SYSCALL_READLINK` 28
- `#define SYSCALL_GETRUSAGE` 29
- `#define SYSCALL_GETRLIMIT` 30
- `#define SYSCALL_UNAME` 31
- `#define SYSCALL_FUTEX_WAIT` 32
- `#define SYSCALL_FUTEX_WAKE` 33
- `#define SYSCALL_MEMINFO` 34
- `#define SYSCALL_PIPE` 35
- `#define SYSCALL_UNLINK` 36
- `#define SYSCALL_RUNCMD` 40

## Functions

- `void sys_libc_log (const char *message)`
- `int sys_runcmd (const char *cmd)`
- `int sys_fork ()`
- `int sys_meminfo ()`
- `int sys_openat (int dirfd, const char *path, int flags)`
- `int sys_getcwd (char *buffer, uint64_t size)`
- `int sys_chdir (const char *path)`
- `int sys_unlink (const char *path)`
- `int sys_pipe (int *fd)`
- `int sys_open (const char *path, int flags)`
- `int sys_close (int fd)`
- `int sys_read (int fd, void *buf, uint64_t count)`
- `int sys_write (int fd, const void *buf, uint64_t count)`
- `int sys_exec (const char *path, char *const argv[])`
- `void sys_exit (int status)`
- `int sys_wait (int pid)`
- `void sys_panic (const char *message)`
- `void * sys_malloc (int size)`
- `int sys_mkdirat (const char *path)`
- `int sys_dup (int fd, int flags, int newfd)`
- `int sys_fstat (int fd, stat\_t *statbuf)`
- `int sys_stat (const char *path, stat\_t *statbuf)`
- `int sys_readdir (int fd, void *buffer)`

### 5.104.1 Detailed Description

#### Implementation of system call functions.

---

This file contains the implementation of system call functions for HanOS. It includes various system call wrappers that provide interfaces for common operations such as file handling, process management, memory allocation, and logging. These functions utilize inline assembly to perform the actual system calls.

---

## 5.104.2 Macro Definition Documentation

### 5.104.2.1 SYSCALL0

```
#define SYSCALL0(
 NUM)
```

**Value:**

```
{
 asm volatile ("syscall"
 : "=a"(ret), "=d"(errno)
 : "a"(NUM)
 : "rcx", "r11", "memory");
}
```

### 5.104.2.2 SYSCALL1

```
#define SYSCALL1(
 NUM,
 ARG0)
```

**Value:**

```
{
 asm volatile ("syscall"
 : "=a"(ret), "=d"(errno)
 : "a"(NUM), "D"(ARG0)
 : "rcx", "r11", "memory");
}
```

### 5.104.2.3 SYSCALL2

```
#define SYSCALL2(
 NUM,
 ARG0,
 ARG1)
```

**Value:**

```
{
 asm volatile ("syscall"
 : "=a"(ret), "=d"(errno)
 : "a"(NUM), "D"(ARG0), "S"(ARG1)
 : "rcx", "r11", "memory");
}
```

### 5.104.2.4 SYSCALL3

```
#define SYSCALL3(
 NUM,
 ARG0,
 ARG1,
 ARG2)
```

**Value:**

```
{
 asm volatile ("syscall"
 : "=a"(ret), "=d"(errno)
 : "a"(NUM), "D"(ARG0), "S"(ARG1), "d"(ARG2)
 : "rcx", "r11", "memory");
}
```

### 5.104.2.5 SYSCALL4

```
#define SYSCALL4(
 NUM,
 ARG0,
```

```

 ARG1,
 ARG2,
 ARG3)

```

**Value:**

```

({
 register typeof(ARG3) arg3 asm("r10") = ARG3;
 asm volatile ("syscall"
 : "=a"(ret), "=d"(errno)
 : "a"(NUM), "D"(ARG0), "S"(ARG1), "d"(ARG2),
 : "r"(arg3)
 : "rcx", "r11", "memory");
})

```

**5.104.2.6 SYSCALL6**

```

#define SYSCALL6(
 NUM,
 ARG0,
 ARG1,
 ARG2,
 ARG3,
 ARG4,
 ARG5)

```

**Value:**

```

({
 register typeof(ARG3) arg3 asm("r10") = ARG3;
 register typeof(ARG4) arg4 asm("r8") = ARG4;
 register typeof(ARG5) arg5 asm("r9") = ARG5;
 asm volatile ("syscall"
 : "=a"(ret), "=d"(errno)
 : "a"(NUM), "D"(ARG0), "S"(ARG1), "d"(ARG2),
 : "r"(arg3), "r"(arg4), "r"(arg5)
 : "rcx", "r11", "memory");
})

```

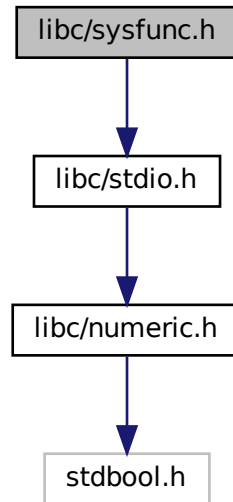
**5.105 libc/sysfunc.h File Reference**

System call function declarations and related macros.

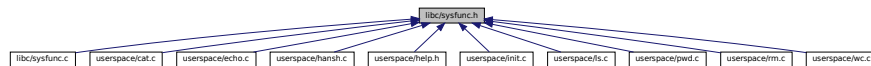


```
#include <libc/stdio.h>
```

Include dependency graph for sysfunc.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct [command\\_help\\_t](#)

## Macros

- #define **AT\_FDCWD** -100
- #define **STDIN** 0
- #define **STDOUT** 1
- #define **STDERR** 2
- #define **O\_ACCMODE** 0x0007
- #define **O\_EXEC** 1
- #define **O\_RDONLY** 2
- #define **O\_RDWR** 3
- #define **O\_SEARCH** 4
- #define **O\_WRONLY** 5
- #define **O\_APPEND** 0x0008
- #define **O\_CREAT** 0x0010
- #define **O\_DIRECTORY** 0x0020
- #define **O\_EXCL** 0x0040
- #define **O\_NOCTTY** 0x0080
- #define **O\_NOFOLLOW** 0x0100

- `#define O_TRUNC 0x0200`
- `#define O_NONBLOCK 0x0400`
- `#define O_DSYNC 0x0800`
- `#define O_RSYNC 0x1000`
- `#define O_SYNC 0x2000`
- `#define O_CLOEXEC 0x4000`
- `#define O_PATH 0x8000`

## Functions

- void **sys\_libc\_log** (const char \*message)
- int **sys\_meminfo** ()
- int **sys\_fork** ()
- int **sys\_openat** (int dirfd, const char \*path, int flags)
- int **sys\_getcwd** (char \*buffer, uint64\_t size)
- int **sys\_chdir** (const char \*path)
- int **sys\_open** (const char \*path, int flags)
- int **sys\_close** (int fd)
- int **sys\_read** (int fd, void \*buf, uint64\_t count)
- int **sys\_write** (int fd, const void \*buf, uint64\_t count)
- int **sys\_exec** (const char \*path, char \*const argv[])
- void **sys\_exit** (int status)
- int **sys\_wait** (int pid)
- void **sys\_panic** (const char \*message)
- void \* **sys\_malloc** (int size)
- int **sys\_mkdirat** (const char \*path)
- int **sys\_dup** (int fd, int flags, int newfd)
- int **sys\_fstat** (int fd, [stat\\_t](#) \*statbuf)
- int **sys\_stat** (const char \*path, [stat\\_t](#) \*statbuf)
- int **sys\_readdir** (int fd, void \*buffer)
- int **sys\_pipe** (int \*fd)
- int **sys\_unlink** (const char \*path)
- int **sys\_runcmd** (const char \*cmd)

### 5.105.1 Detailed Description

System call function declarations and related macros.

---

This file contains the declarations of system call functions and related macros used in the HanOS operating system. It includes definitions for various file descriptor constants, access modes, and flags. The system call functions provide interfaces for common operations such as file handling, process management, and memory allocation.

---

## 5.106 userspace/cat.c File Reference

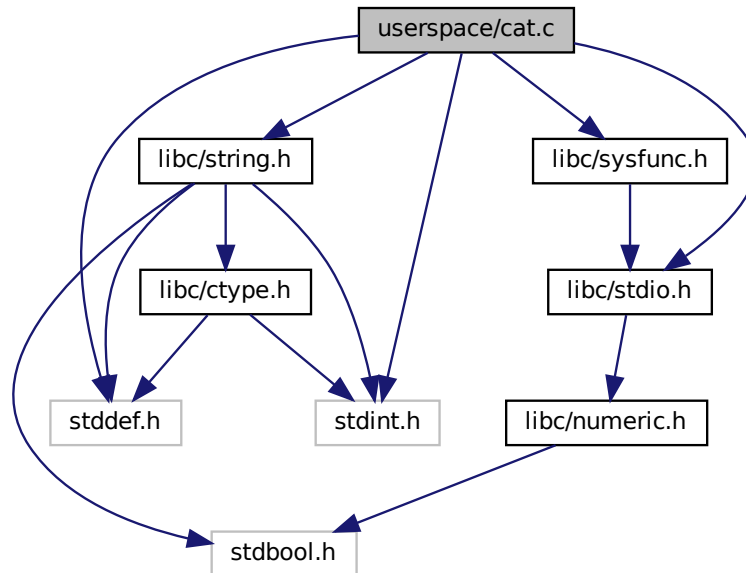
Implementation of the 'cat' command for HanOS userspace.

```
#include <stddef.h>
#include <stdint.h>
#include <libc/stdio.h>
#include <libc/string.h>
```

---

```
#include <libc/sysfunc.h>
```

Include dependency graph for cat.c:



## Functions

- void **cat** (int fd)
- int **main** (int argc, char \*argv[])

## Variables

- char **buf** [512] = {0}

### 5.106.1 Detailed Description

Implementation of the 'cat' command for HanOS userspace.

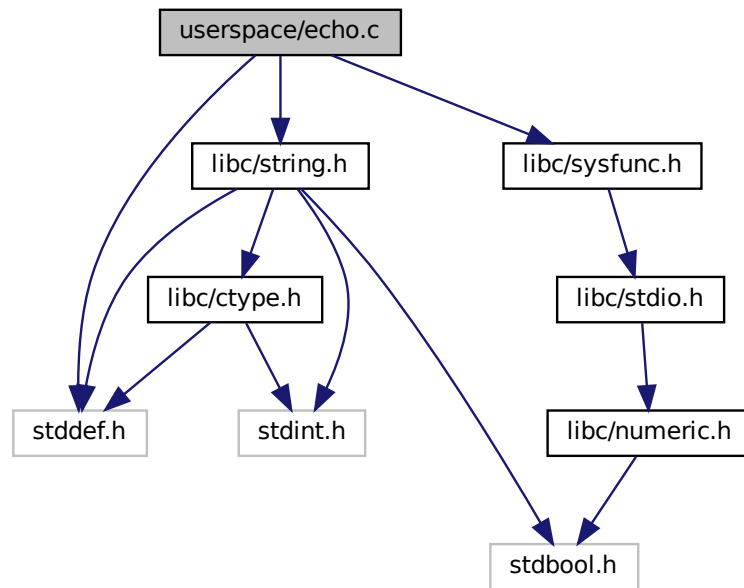
This file provides the implementation of the 'cat' command which is used to concatenate and display files on the standard output. It includes the main function to handle command-line arguments and the cat function for reading from a file descriptor and writing to standard output. Error handling is included for read and write operations.

## 5.107 userspace/echo.c File Reference

Implementation of the 'echo' command for HanOS userspace.

```
#include <stddef.h>
#include <libc/string.h>
#include <libc/sysfunc.h>
```

Include dependency graph for echo.c:



## Functions

- `int main (int argc, char *argv[])`

### 5.107.1 Detailed Description

Implementation of the 'echo' command for HanOS userspace.

---

This file provides the implementation of the 'echo' command which is used to display a specified string. It includes the main function to handle command-line arguments and output them to the standard output, separated by spaces and followed by a newline. This command is useful for displaying messages or concatenating command-line arguments.

---

## 5.108 userspace/hansh.c File Reference

Implementation of the HanOS shell (hansh) for userspace.

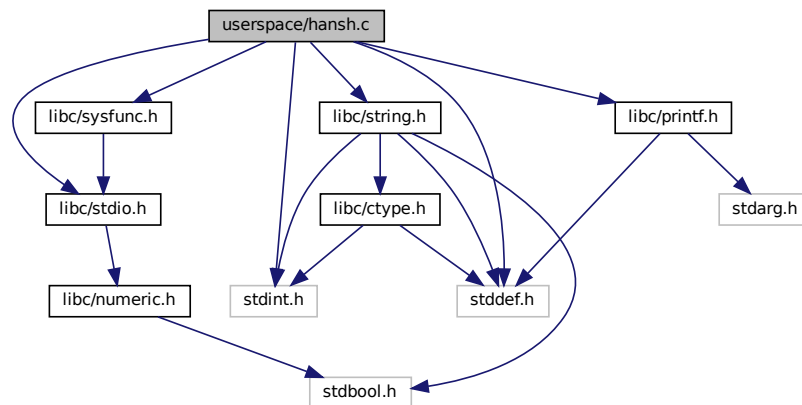
```

#include <stddef.h>
#include <stdint.h>
#include <libc/stdio.h>
#include <libc/string.h>
#include <libc/sysfunc.h>
#include <libc/printf.h>

```

---

Include dependency graph for hansh.c:



## Data Structures

- struct [cmd\\_t](#)
- struct [execcmd\\_t](#)
- struct [redircmd\\_t](#)
- struct [pipecmd\\_t](#)
- struct [listcmd\\_t](#)
- struct [backcmd\\_t](#)

## Macros

- `#define EXEC 1`
- `#define REDIR 2`
- `#define PIPE 3`
- `#define LIST 4`
- `#define BACK 5`
- `#define MAXARGS 10`
- `#define CMD_MAX_LEN 100`
- `#define CMD_PROMPT "\033[36m$ \033[0m"`

## Functions

- int **fork1** (void)
- [cmd\\_t](#) \* **parsecmd** (char \*)
- void **runcmd** ([cmd\\_t](#) \*cmd)
- int **getcmd** (char \*buf, int nbuf)
- int **main** (void)
- [cmd\\_t](#) \* **execcmd** (void)
- [cmd\\_t](#) \* **redircmd** ([cmd\\_t](#) \*subcmd, char \*file, char \*efile, int mode, int fd)
- [cmd\\_t](#) \* **pipecmd** ([cmd\\_t](#) \*left, [cmd\\_t](#) \*right)
- [cmd\\_t](#) \* **listcmd** ([cmd\\_t](#) \*left, [cmd\\_t](#) \*right)
- [cmd\\_t](#) \* **backcmd** ([cmd\\_t](#) \*subcmd)
- int **gettoken** (char \*\*ps, char \*es, char \*\*q, char \*\*eq)
- int **peek** (char \*\*ps, char \*es, char \*toks)
- [cmd\\_t](#) \* **parseline** (char \*\*, char \*)
- [cmd\\_t](#) \* **parsepipe** (char \*\*, char \*)

- `cmd_t * parseexec (char **, char *)`
- `cmd_t * nulterminate (cmd_t *)`
- `cmd_t * parseredirs (cmd_t *cmd, char **ps, char *es)`
- `cmd_t * parseblock (char **ps, char *es)`

## Variables

- char `whitespace []` = `"\t\r\n\v"`
- char `symbols []` = `"<|>&:()"`

### 5.108.1 Detailed Description

Implementation of the HanOS shell (hansh) for userspace.

---

This file provides the implementation of the HanOS shell (hansh) which is used to execute user commands in the HanOS operating system. It includes functions for parsing and executing commands, handling input and output redirection, and managing processes. The shell supports command execution, piping, background processes, and command lists.

---

### 5.108.2 Function Documentation

#### 5.108.2.1 execcmd()

```
cmd_t* execcmd (
 void)
```

---

### 5.108.3 Variable Documentation

#### 5.108.3.1 whitespace

```
char whitespace[] = "\t\r\n\v"
```

---

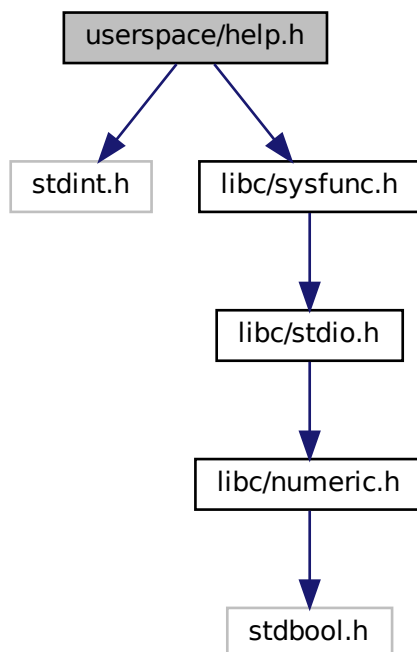
## 5.109 userspace/help.h File Reference

Definition of help command related data structures.

```
#include <stdint.h>
#include <libc/sysfunc.h>
```

---

Include dependency graph for help.h:



## Variables

- const `command_help_t_shell_helptab` []

### 5.109.1 Detailed Description

Definition of help command related data structures.

---

The help list will be generated by genhelp from sourcec codes.

---

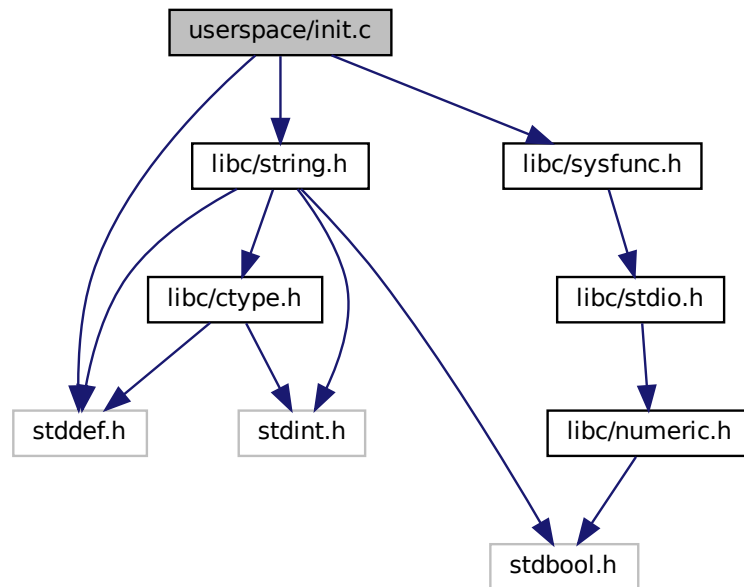
## 5.110 userspace/init.c File Reference

Initialization process for HanOS userspace.

```
#include <stddef.h>
#include <libc/string.h>
#include <libc/sysfunc.h>
```

---

Include dependency graph for init.c:



## Functions

- int **main** (void)

## Variables

- char \* **argv** []

### 5.110.1 Detailed Description

Initialization process for HanOS userspace.

This file provides the implementation of the initialization process for the HanOS userspace. It includes the main function that continuously starts the shell (hansh) by forking and executing the shell program. The init process is responsible for ensuring that the shell is always running and handles the necessary process management.

### 5.110.2 Variable Documentation

#### 5.110.2.1 argv

```
char* argv[]
```

**Initial value:**

```
= {
 "hansh",
 NULL
}
```

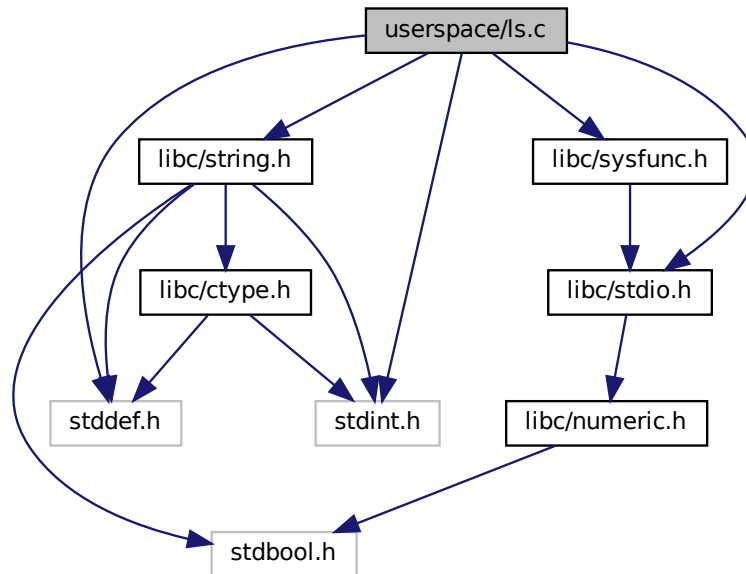


## 5.111 userspace/lc.c File Reference

Implementation of the 'ls' command for HanOS userspace.

```
#include <stddef.h>
#include <stdint.h>
#include <libc/stdio.h>
#include <libc/string.h>
#include <libc/sysfunc.h>
```

Include dependency graph for lc.c:



### Macros

- `#define DIRSIZE 1024`

### Functions

- `char * fmtname (char *path, char *buf)`
- `void ls (char *path)`
- `int main (int argc, char *argv[])`

#### 5.111.1 Detailed Description

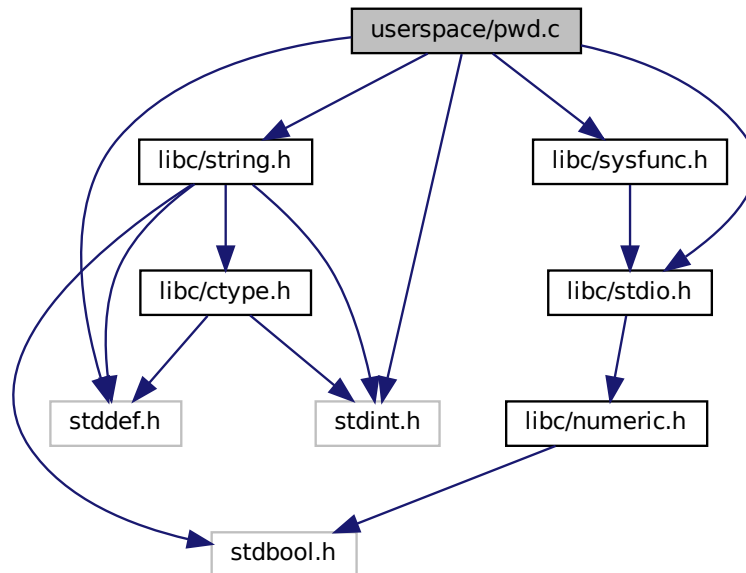
Implementation of the 'ls' command for HanOS userspace.

This file provides the implementation of the 'ls' command which is used to list the contents of a specified directory. It includes functions for formatting directory names and reading directory entries. The command displays file names, types, inode numbers, and sizes. Error handling is included for directory operations.

## 5.112 userspace/pwd.c File Reference

Implementation of the 'pwd' command for HanOS userspace.

```
#include <stddef.h>
#include <stdint.h>
#include <libc/stdio.h>
#include <libc/string.h>
#include <libc/sysfunc.h>
Include dependency graph for pwd.c:
```



## Macros

- `#define DIRSIZE 256`

## Functions

- `int main (int argc, char *argv[])`

### 5.112.1 Detailed Description

Implementation of the 'pwd' command for HanOS userspace.

This file provides the implementation of the 'pwd' command which is used to print the current working directory. It includes the main function that calls `sys_getcwd` to retrieve the current directory path and then prints it to the standard output. Error handling is included for the `getcwd` operation.

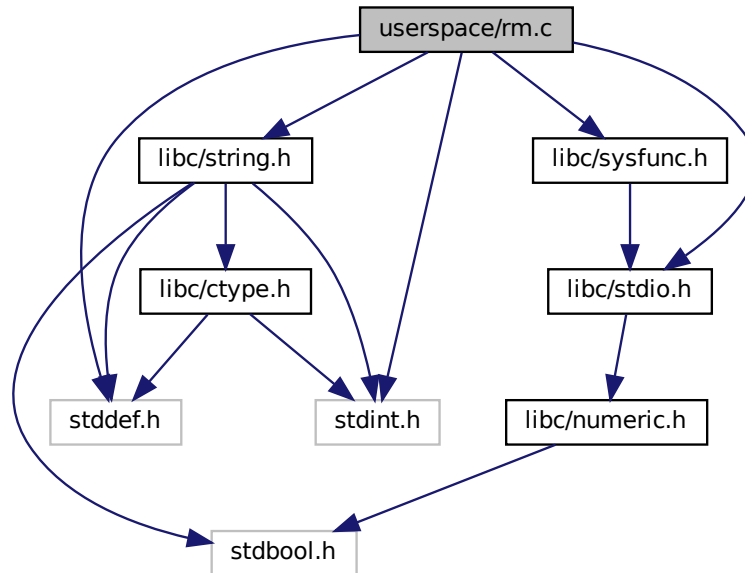
## 5.113 userspace/rm.c File Reference

Implementation of the 'rm' command for HanOS userspace.

```
#include <stddef.h>
#include <stdint.h>
#include <libc/stdio.h>
#include <libc/string.h>
```

```
#include <libc/sysfunc.h>
```

Include dependency graph for rm.c:



## Functions

- `int main (int argc, char *argv[])`

### 5.113.1 Detailed Description

Implementation of the 'rm' command for HanOS userspace.

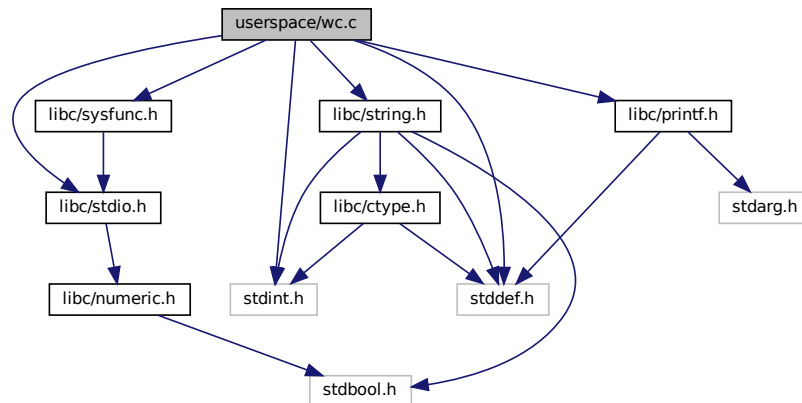
This file provides the implementation of the 'rm' command which is used to remove files or directories in the HanOS operating system. It includes the main function that processes command-line arguments and calls the `sys_unlink` system function to delete specified files. Error handling is included for file deletion operations.

## 5.114 userspace/wc.c File Reference

Implementation of the 'wc' command for HanOS userspace.

```
#include <stddef.h>
#include <stdint.h>
#include <libc/stdio.h>
#include <libc/string.h>
#include <libc/sysfunc.h>
#include <libc/printf.h>
```

Include dependency graph for wc.c:



## Functions

- void **wc** (int fd, char \*name)
- int **main** (int argc, char \*argv[])

## Variables

- char **buf** [512] = {0}
- char **out** [512] = {0}
- bool **outmore** = false

### 5.114.1 Detailed Description

Implementation of the 'wc' command for HanOS userspace.

This file provides the implementation of the 'wc' command which is used to print newline, word, and byte counts for files or standard input in the HanOS operating system. It includes functions for reading and counting characters, words, and lines. The command processes input and outputs the counts to the standard output. Error handling is included for file operations.

# Index

`_putchar`  
`printf.h`, 202

`AC_PR`  
`gdt.h`, 164

`AC_ST`  
`gdt.h`, 164

`acpi_gas_t`, 13

`acpi_sdt_hdr_t`, 13

`acpi_sdt_t`, 14

`addrspace_t`, 14

`alloc.c`

`CAPACITY_SIZE`, 123

`apic.c`

`apic_enable`, 152

`apic_init`, 152

`apic_read_reg`, 152

`apic_send_eoi`, 153

`apic_send_ipi`, 153

`apic_write_reg`, 153

`apic.h`

`apic_enable`, 155

`apic_init`, 155

`apic_read_reg`, 155

`apic_send_eoi`, 157

`apic_send_ipi`, 157

`apic_write_reg`, 157

`apic_enable`

`apic.c`, 152

`apic.h`, 155

`apic_init`

`apic.c`, 152

`apic.h`, 155

`apic_read_reg`

`apic.c`, 152

`apic.h`, 155

`apic_send_eoi`

`apic.c`, 153

`apic.h`, 157

`apic_send_ipi`

`apic.c`, 153

`apic.h`, 157

`apic_write_reg`

`apic.c`, 153

`apic.h`, 157

`argv`

`init.c`, 218

`ASSERT`

`klib.h`, 75

`ata_device_t`, 15

`ata_identify_t`, 16

`atapi_command_t`, 16

`auxval_t`, 16

`backcmd_t`, 17

`bmp_header_t`, 17

`CAPACITY_SIZE`

`alloc.c`, 123

`cmd_t`, 18

`cmos_rtc_t`, 18

`command_help_t`, 18

`computer_info_t`, 19

`cpu.h`

`read_cr`, 162

`write_cr`, 162

`cpu_t`, 19

`cpuid_feature_t`, 20

`dirent_t`, 20

`edid_info_t`, 20

`elf_hdr_t`, 21

`elf_nhdr_t`, 22

`elf_phdr_t`, 22

`elf_shdr_t`, 23

`elf_sym_t`, 23

`ENETRESET`

`errno.h`, 197

`EOPNOTSUPP`

`errno.h`, 197

`errno.h`

`ENETRESET`, 197

`EOPNOTSUPP`, 197

`event_t`, 23

`execcmd`

`hansh.c`, 216

`execcmd_t`, 24

`fat32`

`fat32.c`, 103

`fat32.c`

`fat32`, 103

`fat32_bs_info_t`, 24

`fat32_entry_t`, 24

`fat32_ident_item_t`, 25

`fat32_ident_t`, 25

`fat_bs_t`, 26

`fat_dir_entry_t`, 26

`fat_extbs_16_t`, 27

`fat_extbs_32_t`, 27

fat\_lfn\_entry\_t, 27  
 fb\_info\_t, 28  
 fctprintf  
     printf.h, 202  
 file\_dup\_t, 28  
 font\_psf1\_t, 29  
 font\_psf2\_t, 29  
  
 gdt.h  
     AC\_PR, 164  
     AC\_ST, 164  
     GDT\_GR, 165  
     GDT\_LM, 165  
 gdt\_entry\_t, 29  
 GDT\_GR  
     gdt.h, 165  
 GDT\_LM  
     gdt.h, 165  
 gdt\_register\_t, 30  
 gdt\_table\_t, 30  
 GET\_SLAB  
     slab.c, 128  
 gfx\_gtt\_t, 31  
 gfx\_mem\_manager\_t, 31  
 gfx\_mem\_range\_t, 32  
 gfx\_object\_t, 32  
 gfx\_pci\_t, 32  
  
 hansh.c  
     execcmd, 216  
     whitespace, 216  
 hpet\_sdt\_t, 33  
 hpet\_t, 33  
 hpet\_timer\_t, 34  
 ht\_item\_t, 34  
 ht\_t, 35  
  
 ide\_channel\_regs\_t, 35  
 ide\_device\_t, 36  
 idt\_entry\_t, 36  
 idt\_register\_t, 36  
 image\_t, 37  
 init.c  
     argv, 218  
 isr\_base.h  
     isr\_disable\_interrupts, 171  
     isr\_enable\_interrupts, 171  
 isr\_disable\_interrupts  
     isr\_base.h, 171  
 isr\_enable\_interrupts  
     isr\_base.h, 171  
  
 kernel/base/hash.c, 71  
 kernel/base/hash.h, 72  
 kernel/base/image.c, 72  
 kernel/base/image.h, 73  
 kernel/base/klib.h, 74  
 kernel/base/klog.c, 75  
 kernel/base/klog.h, 76  
  
 kernel/base/kmalloc.c, 77  
 kernel/base/kmalloc.h, 78  
 kernel/base/lock.c, 80  
 kernel/base/lock.h, 80  
 kernel/base/time.c, 81  
 kernel/base/time.h, 82  
 kernel/base/vector.h, 84  
 kernel/device/display/edid.h, 86  
 kernel/device/display/fb.c, 86  
 kernel/device/display/fb.h, 88  
 kernel/device/display/font.h, 89  
 kernel/device/display/gfx.c, 90  
 kernel/device/display/gfx.h, 91  
 kernel/device/display/gfx\_reg.h, 92  
 kernel/device/display/term.c, 94  
 kernel/device/display/term.h, 95  
 kernel/device/keyboard/keyboard.h, 97  
 kernel/device/keyboard/keycode.h, 98  
 kernel/device/storage/ata.c, 99  
 kernel/device/storage/ata.h, 100  
 kernel/fs/fat32.c, 102  
 kernel/fs/fat32.h, 103  
 kernel/fs/filebase.c, 105  
 kernel/fs/filebase.h, 106  
 kernel/fs/pipefs.c, 107  
 kernel/fs/pipefs.h, 108  
 kernel/fs/ramfs.c, 110  
 kernel/fs/ramfs.h, 111  
 kernel/fs/ttyfs.c, 112  
 kernel/fs/ttyfs.h, 115  
 kernel/fs/vfs.c, 116  
 kernel/fs/vfs.h, 117  
 kernel/kconfig.h, 120  
 kernel/kmain.c, 121  
 kernel/mm/alloc.c, 122  
 kernel/mm/alloc.h, 123  
 kernel/mm/mm.h, 124  
 kernel/mm/pmm.c, 126  
 kernel/mm/slab.c, 127  
 kernel/mm/slab.h, 129  
 kernel/mm/vmm.c, 130  
 kernel/proc/elf.c, 131  
 kernel/proc/elf.h, 132  
 kernel/proc/eventbus.c, 134  
 kernel/proc/eventbus.h, 134  
 kernel/proc/sched.c, 135  
 kernel/proc/sched.h, 137  
 kernel/proc/signal.c, 138  
 kernel/proc/signal.h, 139  
 kernel/proc/syscall.c, 141  
 kernel/proc/syscall.h, 143  
 kernel/proc/task.c, 145  
 kernel/proc/task.h, 146  
 kernel/proc/wait.h, 148  
 kernel/symbols.h, 148  
 kernel/sys/acpi.c, 149  
 kernel/sys/acpi.h, 150  
 kernel/sys/apic.c, 151

- kernel/sys/apic.h, 154
- kernel/sys/cmos.c, 158
- kernel/sys/cmos.h, 159
- kernel/sys/cpu.c, 160
- kernel/sys/cpu.h, 161
- kernel/sys/gdt.c, 162
- kernel/sys/gdt.h, 163
- kernel/sys/hpet.c, 165
- kernel/sys/hpet.h, 166
- kernel/sys/idt.c, 167
- kernel/sys/idt.h, 168
- kernel/sys/isr.c, 169
- kernel/sys/isr\_base.h, 170
- kernel/sys/madt.c, 172
- kernel/sys/madt.h, 172
- kernel/sys/mtrr.c, 174
- kernel/sys/mtrr.h, 175
- kernel/sys/panic.h, 176
- kernel/sys/pci.c, 178
- kernel/sys/pci.h, 179
- kernel/sys/pit.c, 184
- kernel/sys/pit.h, 185
- kernel/sys/serial.c, 185
- kernel/sys/serial.h, 186
- kernel/sys/smp.c, 187
- kernel/sys/smp.h, 188
- kernel/sys/timer.c, 190
- kernel/sys/timer.h, 191
- kernel/version.h, 192
- keyboard\_t, 37
- klib.h
  - ASSERT, 75
- klog\_info\_t, 38
- kpanic
  - panic.h, 177
- libc/ctype.c, 193
- libc/ctype.h, 194
- libc/errno.h, 195
- libc/numeric.c, 197
- libc/numeric.h, 198
- libc/printf.h, 199
- libc/stdio.h, 202
- libc/string.c, 204
- libc/string.h, 205
- libc/sysfunc.c, 206
- libc/sysfunc.h, 210
- listcmd\_t, 39
- lock\_t, 39
- madt\_record\_hdr\_t, 39
- madt\_record\_ioapic\_t, 40
- madt\_record\_iso\_t, 40
- madt\_record\_lapic\_t, 41
- madt\_record\_nmi\_t, 42
- madt\_t, 42
- mbr\_t, 43
- mem\_info\_t, 43
- mem\_map\_t, 44
- memory\_metadata\_t, 44
- mm.h
  - VMM\_FLAGS\_MMIO, 126
- out\_fct\_wrap\_type, 44
- panic.h
  - kpanic, 177
  - panic\_if, 178
  - panic\_unless, 178
- panic\_if
  - panic.h, 178
- panic\_unless
  - panic.h, 178
- partition\_t, 45
- pci.c
  - pci\_func\_exist, 179
- pci.h
  - PCI\_MAKE\_DEVICE\_ID, 183
- pci\_bar\_t, 45
- pci\_device\_desc\_t, 45
- pci\_device\_t, 46
- pci\_func\_exist
  - pci.c, 179
- PCI\_MAKE\_DEVICE\_ID
  - pci.h, 183
- pipecmd\_t, 46
- pipefs
  - pipefs.c, 108
- pipefs.c
  - pipefs, 108
- pipefs\_file\_t, 47
- pipefs\_ident\_item\_t, 47
- pipefs\_ident\_t, 48
- prdt\_t, 48
- printf.h
  - \_putchar, 202
  - fctprintf, 202
  - printf\_, 200
  - snprintf, 200
  - sprintf, 201
  - vprintf, 201
- printf\_
  - printf.h, 200
- ramfs
  - ramfs.c, 111
- ramfs.c
  - ramfs, 111
- ramfs\_file\_t, 48
- ramfs\_ident\_item\_t, 49
- ramfs\_ident\_t, 49
- read\_cr
  - cpu.h, 162
- redircmd\_t, 50
- rsdp\_t, 50
- rusage\_t, 51
- S\_IRWXG

- vfs.h, 120
- S\_IRWXO
  - vfs.h, 120
- S\_IRWXU
  - vfs.h, 120
- scache\_t, 52
- sigaction\_t, 53
- siginfo\_t, 53
- sigset\_t, 55
- sigval, 55
- slab.c
  - GET\_SLAB, 128
- slab\_t, 55
- smp\_info\_t, 56
- snprintf
  - printf.h, 200
- sprintf
  - printf.h, 201
- stack\_t, 56
- stat\_t, 57
- symbol\_t, 57
- sys\_seg\_desc\_t, 58
- SYSCALL0
  - sysfunc.c, 209
- SYSCALL1
  - sysfunc.c, 209
- SYSCALL2
  - sysfunc.c, 209
- SYSCALL3
  - sysfunc.c, 209
- SYSCALL4
  - sysfunc.c, 209
- SYSCALL6
  - sysfunc.c, 210
- sysfunc.c
  - SYSCALL0, 209
  - SYSCALL1, 209
  - SYSCALL2, 209
  - SYSCALL3, 209
  - SYSCALL4, 209
  - SYSCALL6, 210
- task\_regs\_t, 58
- task\_t, 59
- term\_info\_t, 60
- termios\_t, 61
- timespec\_t, 61
- timeval\_t, 61
- timezone\_t, 62
- tm\_t, 62
- tss\_t, 62
- ttyfs
  - ttyfs.c, 114
- ttyfs.c
  - ttyfs, 114
- ttyfs\_file\_t, 63
- ttyfs\_ident\_item\_t, 63
- ttyfs\_ident\_t, 64
- userspace/cat.c, 212
- userspace/echo.c, 213
- userspace/hansh.c, 214
- userspace/help.h, 216
- userspace/init.c, 217
- userspace/lis.c, 219
- userspace/pwd.c, 219
- userspace/rm.c, 220
- userspace/wc.c, 221
- ustar\_file\_t, 64
- vec\_erase
  - vector.h, 85
- vec\_erase\_all
  - vector.h, 85
- vec\_erase\_val
  - vector.h, 85
- vec\_push\_back
  - vector.h, 85
- vec\_struct
  - vector.h, 86
- vector.h
  - vec\_erase, 85
  - vec\_erase\_all, 85
  - vec\_erase\_val, 85
  - vec\_push\_back, 85
  - vec\_struct, 86
- vfs.h
  - S\_IRWXG, 120
  - S\_IRWXO, 120
  - S\_IRWXU, 120
- vfs\_dirent\_t, 65
- vfs\_fsinfo\_t, 65
- vfs\_inode\_t, 66
- vfs\_node\_desc\_t, 67
- vfs\_stat\_t, 68
- vfs\_timespec\_t, 69
- vfs\_tnode\_t, 69
- VMM\_FLAGS\_MMIO
  - mm.h, 126
- vprintf
  - printf.h, 201
- whitespace
  - hansh.c, 216
- winsize\_t, 70
- write\_cr
  - cpu.h, 162